ENGINEERING MASTERPLAN · BACSE202 DATABASE SYSTEMS

VOLTHUB CSMS

A portfolio-grade Electric Vehicle Charging Station Management System: Oracle for transactional truth, TimescaleDB for charger telemetry analytics, driven end-to-end by an OCPP 1.6J charger simulator — with domain research, ER/BCNF database design, PL/SQL implementation, DA3 comparative analysis, and a week-by-week execution plan in one document.

Prepared for a three-person project team

Digital Assignments DA1 · DA2 · DA3 — 51 sections

EV CHARGING STATION MANAGEMENT · SEPTEMBER 2026

Table of Contents

PART I

Executive Summary, Research Findings, and Domain Analysis

> Masterplan sections 1–3. This part establishes what VoltHub is, why every major decision in it is defensible, and how the real EV charging industry actually works. Facts are labeled **[Fact]** with citations; judgment calls are labeled **[Recommendation]** or **[Assumption]**.

---

1. Executive Summary

VoltHub CSMS is a Charging Station Management System for a mid-size electric-vehicle charging network operator (a CPO — Charge Point Operator). It manages stations, charging points, connectors, drivers, reservations, charging sessions, metering, tariffs, billing, payments, faults, maintenance, and analytics. It is being built by a three-person team as the assigned mini-project for BACSE202 (Database Systems), and it deliberately satisfies the university's DA1 → DA2 → DA3 progression with a single continuous codebase instead of three disconnected submissions.

The core architectural thesis is a **two-engine database design**: Oracle remains the system of record for everything transactional (users, stations, connectors, reservations, sessions, tariffs, payments, invoices, maintenance — the OLTP workload), while **TimescaleDB** is introduced in DA3 as the purpose-built store for high-frequency charger telemetry and time-series analytics (the OLAP workload). This is not "Oracle plus another database for the resume." It is the same split real charging networks make: industry references describe EV charging platforms as mixing 15–30 second telemetry writes with strictly consistent billing data [8], and Timescale's own CSMS architecture guide identifies meter values, transaction records, status events, and configuration as four distinct data workloads with different storage requirements [7]. DA1 designs the relational core, DA2 implements it in Oracle SQL/PL-SQL, and DA3 streams the telemetry slice into TimescaleDB with hypertables, continuous aggregates, compression, and retention policies — a genuine OLTP-versus-OLAP separation that can be defended line-by-line in a viva.

The application layer is a **modular monolith** written in **TypeScript**: a NestJS REST API plus an OCPP 1.6J WebSocket gateway with a built-in charger simulator, backed by Oracle (via `node-oracledb` with hand-written SQL and calls to PL/SQL packages). The frontend is a **Next.js (App Router)** application with a custom design system called **Grid Current** — typography-led, dark, data-dense, inspired by the engineering discipline of acmvit.in (Astro, Tailwind, oversized display type, cream-on-near-black palette) but an original identity: Space Grotesk display type, IBM Plex Mono for tabular numerals, a carbon/cream/electric-lime palette, and hairline-bordered data surfaces instead of glassmorphism and gradient decoration.

The eight recruiter-facing "wow moments" this plan is engineered to produce:

1. **A BCNF-honest relational model** with worked functional-dependency proofs and two intentional, justified denormalizations.
2. **Race-condition-proof reservation logic** — `SELECT ... FOR UPDATE` plus a uniqueness constraint that makes double-booking physically impossible, demonstrated live with two parallel requests.

3. **A realistic charging-session lifecycle** driven by a simulated OCPP 1.6J charger speaking JSON over WebSocket (BootNotification, StatusNotification, Authorize, StartTransaction, MeterValues, StopTransaction) [4][23].
4. **Business logic in the right place**: billing, tariff resolution, and connector-state invariants implemented as Oracle packages and triggers, not scattered across app code.
5. **An OLTP/OLAP split with evidence**: Oracle for transactions, TimescaleDB hypertables + continuous aggregates for utilization and load analytics, with compression and retention policies [7][19].
6. **A query portfolio that reads like an analyst wrote it**: window functions, CTEs, gaps-and-islands utilization, revenue trends, keyset pagination.
7. **A frontend that does not look AI-generated**: an original design system, data-dense operator dashboards, live telemetry charts, and a map-based station discovery experience.
8. **Production hygiene**: Docker Compose, CI on GitHub Actions, versioned SQL migrations, seeded deterministic data, structured logs, health checks, and a real test suite that includes concurrency tests.

What this project is not: it is not a microservices platform, not a real hardware deployment, not a payments processor (no card data is ever stored), and not a machine-learning product. Every exclusion is documented in Section 45 with the reasoning, because knowing what not to build is part of the engineering maturity this project is meant to demonstrate.

---

2. Research Findings

This section is the evidence base for every later decision. Each finding is marked **[Fact]** (verifiable, cited), **[Recommendation]** (our judgment based on facts), or **[Assumption]** (stated belief we could not fully verify).

2.1 OCPP — the protocol that makes this project real

[Fact] The Open Charge Point Protocol (OCPP) is the open, vendor-neutral protocol connecting charging stations (charge points) to central management systems. It is developed by the Open Charge Alliance and is the de-facto global standard [1].

[Fact] OCPP 1.6J (JSON over WebSocket) remains the most widely deployed version in the field [4]. OCPP 2.0.1 replaced the classic `StartTransaction`/`StopTransaction`/`MeterValues` flow with a unified `TransactionEvent` message, added a structured Device Model, and introduced formal security profiles [2][3]. OCPP 2.1 has since been published as IEC 63584, extending 2.0.1 compatibly [1].

[Fact] A charging session in OCPP 1.6 follows a recognizable message choreography: the charger sends `BootNotification` when it comes online, `StatusNotification` as its connector state changes, `Authorize` when a driver presents an idTag, `StartTransaction` when energy flow begins, periodic `MeterValues` while charging, and `StopTransaction` with a final meter value when it ends [1][23]. Remote operations (`RemoteStartTransaction`, `RemoteStopTransaction`, `Reset`, `ChangeAvailability`) flow from the central system to the charger.

[Recommendation] We implement an **OCPP 1.6J subset** in both directions: our backend acts as the central system, and a Node.js **charger simulator** acts as the charge point. This is the single highest-leverage research-based decision in the project. It converts every database feature from an abstract CRUD row into the observable outcome of a protocol conversation — connector states change because `StatusNotification` arrived, sessions and meter readings exist because the charger reported them, and reservations end in a real state machine. OCPP 1.6J is chosen over 2.0.1 because it is simpler, dramatically better documented for implementers, most deployed in the real world, and fully sufficient to demonstrate the database concepts DA1–DA3 grade on. The simulator replaces physical chargers; we make no claims about real hardware behavior beyond what the protocol specifies.

[Assumption] The professor will treat an OCPP simulator as a legitimate "realistic workload generator." SteVe — a mature open-source OCPP server maintained since 2013 [10] — establishes strong precedent that this is exactly how CSMS software is built and tested without hardware.

2.2 CSMS architecture in industry

[Fact] A Charging Station Management System (CSMS) is the backend platform that handles OCPP communication, user authentication, billing, and network management for a charging network [5][7].

[Fact] Industry engineering guidance states that at low scale, a monolithic CSMS with a relational database is the appropriate architecture, with microservices reserved for networks scaling to very large station counts [6]. AWS's reference architecture similarly decomposes the domain into command/control of chargers, session processing, and billing rather than into microservices [5].

[Recommendation] Modular monolith: one NestJS application with strict module boundaries (auth, stations, sessions, billing, telemetry, admin), one Oracle schema, one TimescaleDB store, plus a small worker process for the telemetry pipeline. Microservices would add distributed-transaction complexity that would *weaken* the database story this course grades. Section 8 contains the full decision record.

2.3 The data workload evidence — why two engines

[Fact] Timescale's CSMS guide states that EV charging databases must handle four distinct OCPP data types — meter values, charge detail records (CDRs), status events, and configuration — and that meter values are high-volume, time-ordered, and analytics-heavy, which is a poor fit for a row-store OLTP engine tuned for transactions [7].

[Fact] Managed-database providers serving EV charging platforms describe workloads that "mix 15–30 second telemetry writes with PCI-DSS billing data" [8]. Billing data demands strict ACID guarantees; telemetry demands cheap, append-heavy ingestion with time-windowed analytical reads.

[Fact] OLTP and OLAP are fundamentally different workloads: OLTP optimizes many small fast operations with concurrency control; OLAP optimizes large scans and aggregations over historical data [22]. Forcing both into one engine forces compromises in indexing, storage layout, and vacuum/maintenance behavior.

[Recommendation] Oracle holds the money-path (reservations, sessions, payments, invoices — strictly ACID), and TimescaleDB holds the telemetry path (meter ticks, connector state events — insert-heavy, time-ordered, aggregated with continuous aggregates [19]). This is the DA3 thesis, and it is the same thesis industry sources describe [7][8].

2.4 Tariffs and pricing

[Fact] Real networks price charging through several mechanisms: per-kWh energy pricing, time-based pricing, per-session fees, idle fees after charging completes, and time-of-use (ToU) schedules where the per-kWh price depends on time of day [12][13][14]. India's CEA framework for public charging stations is built around a single-part energy tariff per kWh (a billing simplification mandated by regulation) [17].

[Recommendation] Model tariffs as **versioned tariff plans with time-of-use rate bands**: a tariff has a fixed session fee (nullable), and a set of rate bands each defining a price per kWh that applies within time windows. This exercises versioning, temporal validity intervals, and non-trivial joins — all excellent database-design material — while remaining honest to how Indian CPOs actually bill [17].

2.5 The Indian deployment context (assumed market)

[Fact] India standardizes on Type 2 AC connectors (7.4–22 kW), CCS2 DC fast charging (25–150 kW), and the indigenous Bharat AC001/DC001 standards for low-cost charging [15][16]. National guidelines target one public charging station per 3x3 km grid in cities and one every 25 km on highways [18].

[Recommendation] Seed data reflects this reality: stations in Chennai with mixed Type 2 / CCS2 / Bharat AC001 connectors, ToU tariffs in INR per kWh, and coordinates around VIT Chennai and OMR. Local realism makes the demo memorable and defensible.

2.6 Time-series database landscape (DA3 candidates)

[Fact] TimescaleDB extends PostgreSQL with hypertables (automatic time/space chunking), continuous aggregates (incrementally-maintained materialized views refreshed in the background), native compression (commonly 10–20x), and retention policies [19][28]. InfluxDB 3 rebuilds the product on the FDAP stack (Flight, DataFusion, Arrow, Parquet) with SQL support and no cardinality limit; it wins on raw write throughput for pure metrics, while TimescaleDB wins when SQL semantics and hybrid relational workloads matter [20][21].

[Fact] Distributed SQL engines (CockroachDB, YugabyteDB, TiDB) solve horizontal scalability and geo-distribution; practitioners note they are niche compared to a well-tuned single-node relational database at small scale [27].

[Recommendation] TimescaleDB is the DA3 choice; the full 12-criteria decision matrix and the honest counterarguments are in Section 23–24 (file `13-da3-database-decision.md`).

2.7 acmvit.in — design forensics and extracted principles

[Fact] (Direct observation of `https://www.acmvit.in/` HTML, fetched 2026-09.) The site is built with **Astro** (`.astro` component scripts, scoped `data-astro-cid` styles), styled with **Tailwind CSS** (arbitrary-value utilities such as `text-[8rem]`, `leading-[0.85]`, `tracking-tighter`), self-hosts the **PolySans** family (Bold, Bulky Wide, Slim) as display type with **Inter** 400/700/900 as body type, uses a cream-on-dark palette (headline color `#FFFDD0`), full-screen video background sections, hairline `border-white/10` dividers, and oversized uppercase headings [25].

[Recommendation] What we take from ACM VIT is not their code or their look — it is their **design discipline**: (1) typography carries the identity, not decoration; (2) a near-monochrome palette with one warm accent; (3) enormous, tightly-tracked display type used sparingly for scale; (4) hairline borders instead of shadows-and-glass; (5) section-per-view narrative scrolling. What we deliberately do *not* copy: PolySans (paid), the cassette/retro motif, and Astro (wrong tool for a data-dense application). Our frontend uses Next.js and an original identity called **Grid Current**, specified in Section 21. For dark data-dense UIs we additionally apply the elevation-contrast lesson: without distinct surface steps, adjacent panels bleed together [26].

2.8 KPIs operators actually track

[Fact] Charging-network operators track uptime, charging success rate, energy delivered, utilization, session duration, and downtime [29][30]. A 2026 arXiv paper formalizes **Fault Time, Fault-Reason Time, and Unreachable Time** as actionable, computable performance metrics for charging sites [9].

[Recommendation] Our analytics schema computes exactly these: connector uptime %, utilization % (occupied time / sellable time), energy delivered per station/day, mean session duration, fault counts by reason, and peak-hour load curves. The arXiv metric definitions directly shape our `connector_state_event` model in DA3 — status transitions with timestamps make Fault/Unreachable Time a simple window computation [9].

---

3. Real-World EV Charging Domain Analysis

This section defines the domain vocabulary the entire database design is built on. Every entity in DA1 traces back to a concept defined here.

3.1 Actors and organizations

A **Charge Point Operator (CPO)** owns/operates physical charging infrastructure. An **eMobility Service Provider (EMSP)** owns the customer relationship with drivers [11]. In VoltHub these are simplified into one platform with three human roles: **Drivers** (discover, reserve, charge, pay), **Station Operators** (operate stations: monitor, maintain, manage faults), and **Admins** (manage users, operators, tariffs, and the audit trail). The fourth actor is not human: the **charger itself**, which speaks OCPP and is the source of truth for connector state and metering [1].

3.2 The physical hierarchy — Station, Charge Point (EVSE), Connector

[Fact] Charging infrastructure follows a strict three-level hierarchy. A **station** (location) contains one or more **charge points** (the physical cabinet; OCPP 2.x calls this an EVSE — Electric Vehicle Supply Equipment), each offering one or more **connectors** (the physical socket/cable: Type 2, CCS2, Bharat AC001, CHAdeMO) [1][15][16].

This hierarchy is not negotiable in the data model because OCPP itself identifies devices as `chargePointId / connectorId`, availability is a property of a *connector*, power capability is a property of a *connector's standard*, and a station's "has available chargers" is a *derived* aggregate over its connectors. Getting this hierarchy right is the difference between a toy schema and one that mirrors reality.

3.3 The charging session lifecycle

[Fact] Across OCPP versions, a session moves through an operational lifecycle: a driver authorizes (RFID tag / app / Plug-and-Charge in ISO 15118 [24]); the connector transitions to preparing; energy flow begins (charging); it may suspend (EV full, EVSE paused, fault); it finishes with a final meter reading; then the connector returns to available (or into fault) [1][2][23].

[Recommendation] VoltHub models the session lifecycle as an explicit, database-enforced state machine:

```
RESERVED -> PREPARING -> CHARGING -> SUSPENDED -> CHARGING -> COMPLETED
                  |                      |
                  +--> CANCELLED         +--> FAILED  (fault, payment failure)
COMPLETED -> BILLED -> PAID   (billing pipeline, separate from energy flow)
```

Session states live in a lookup table; PL/SQL package procedures are the *only* code path allowed to transition states; a trigger rejects illegal transitions; every transition is audit-logged. This turns "we thought about correctness" into a demoable, testable fact.

3.4 Reservations

Drivers reserve a **connector** for a future time window. Real systems constrain reservations to available connectors, prevent overlapping holds, expire no-shows, and convert a reservation into a session when the driver plugs in. Reservations are VoltHub's flagship concurrency problem: two drivers racing for the last CCS2 connector at 6 PM must be impossible to double-book, enforced by the database rather than by hope. Section 31 specifies the mechanism (`SELECT FOR UPDATE` + an exclusion constraint pattern); Section 15 implements it in PL/SQL.

3.5 Metering, energy, and billing

[Fact] Chargers report meter values (energy, power, current, voltage) periodically during a session — typically every 15–60 seconds in real deployments [7][8]. A session's bill derives from final meter deltas priced under the tariff plan active during the session, plus session/idle fees [13][14].

[Recommendation] Every `MeterValues` message becomes a `METER_READING` row in Oracle (session-scoped, the *billing* record of record: reading at start, at end, and periodic deltas) *and* a telemetry tick streamed to TimescaleDB in DA3 (the *analytics* record: high-frequency power/voltage/current/SOC for load curves). Same physical event, two purposeful representations at different resolutions — the clearest possible demonstration that OLTP and OLAP schemas differ by design, not by accident.

3.6 Faults, maintenance, and the operator loop

Connectors fail: hardware faults, cable breaks, payment terminal errors, or the charger simply becoming unreachable. Operators need a loop — fault reported or detected, maintenance ticket opened, work performed, connector restored — and managers need fault-time metrics per station [9]. VoltHub models `FAULT` (reported by OCPP `StatusNotification` with error codes, or manually) and `MAINTENANCE_RECORD` (inspection, repair, replacement with parts cost), linked to connectors and resolved by operators.

3.7 KPI definitions (used across DA2 queries and DA3 analytics)

KPI	Definition	Where computed
Connector uptime %	1 - (fault time / sellable time) over a period	DA3 cagg + DA2 MV
Utilization %	occupied session time / sellable connector time	DA3 cagg + DA2 MV
Energy delivered	sum of session kWh	Oracle sessions + DA3 ticks
Revenue	sum of invoice totals	Oracle (system of record)
Charging success rate	completed sessions / started sessions	Oracle
Mean session duration	avg(completed_at - started_at)	Oracle
Fault time	sum of fault-state durations by reason	DA3 state events [9]
Peak load	max avg power (kW) per time bucket	DA3 only

The split is deliberate: money and session KPIs are Oracle-native (they need transactional truth); telemetry KPIs are TimescaleDB-native (they need time-series aggregation). Neither engine is asked to do the other's job.

PART II

Functional Requirements, Non-Functional Requirements, Scope, and Roles

> Masterplan sections 4–7. Requirements are numbered so that every entity, query, and screen later in the plan traces back to one. MoSCoW: **M** = must, **S** = should, **C** = could (stretch).

4. Functional Requirements

4.1 Identity and access

ID	Requirement	MoSCoW	DA
FR-AUTH-01	Users register as Driver with email + password (Argon2-hashed) and profile (name, phone)	M	DA2
FR-AUTH-02	Users authenticate and receive a JWT; roles: DRIVER, OPERATOR, ADMIN	M	DA2
FR-AUTH-03	Admin creates/edits Operator accounts and assigns them to stations	M	DA2
FR-AUTH-04	RBAC enforced at API layer (guards) and DB layer (grants); every mutation audit-logged	M	DA2

4.2 Vehicles and drivers

ID	Requirement	MoSCoW	DA
FR-DRV-01	Driver registers EVs: model, battery capacity kWh, connector standard(s)	M	DA2
FR-DRV-02	Driver sets a default vehicle used for reservation compatibility checks	S	DA2
FR-DRV-03	Driver views charging history with energy, cost, duration per session	M	DA2

4.3 Stations, charge points, connectors

ID	Requirement	MoSCoW	DA
FR-STN-01	Admin/Operator creates stations: name, geocode (lat/lng), address, operator owner	M	DA2
FR-STN-02	Station contains charge points (EVSE); each has OCPP identity and vendor/model	M	DA2
FR-STN-03	Charge points contain connectors; connector has standard (Type 2 / CCS2 / Bharat AC001 / CHAdeMO), max power kW, and live status	M	DA2

ID	Requirement	MoSCoW	DA
FR-STN-04	Driver searches stations by text, filters by connector standard and min power, sorts by distance	M	DA2
FR-STN-05	Map view renders stations; station detail shows connectors with live availability	M	DA2/3

4.4 Reservations

ID	Requirement	MoSCoW	DA
FR-RSV-01	Driver reserves an available connector for a 15–120 min future window	M	DA2
FR-RSV-02	Overlapping reservations on the same connector are impossible (DB-enforced)	M	DA2
FR-RSV-03	No-show reservations auto-expire and release the connector	M	DA2
FR-RSV-04	Driver can cancel a reservation before it starts	M	DA2
FR-RSV-05	Reservation converts to a session when the simulated charger reports plug-in with the reserving driver's idTag	S	DA2

4.5 Charging sessions and metering

ID	Requirement	MoSCoW	DA
FR-SES-01	Charger simulator performs full OCPP 1.6J flow: BootNotification, StatusNotification, Authorize, StartTransaction, MeterValues (5s ticks), StopTransaction	M	DA2
FR-SES-02	Backend creates/updates session rows from OCPP events; connector status mirrors charger-reported state	M	DA2
FR-SES-03	Every MeterValues tick is persisted: billing-grade readings in Oracle, telemetry ticks to the analytics store (DA3)	M	DA2/3
FR-SES-04	Driver sees live session: elapsed time, kWh delivered, current kW, estimated cost	M	DA2
FR-SES-05	Driver can remote-stop a session (OCPP RemoteStopTransaction)	S	DA2
FR-SES-06	Session state machine transitions validated in PL/SQL; illegal transitions rejected and logged	M	DA2

4.6 Tariffs, billing, payments

ID	Requirement	MoSCoW	DA
FR-TAR-01	Admin defines tariff plans: name, currency, optional fixed session fee, optional idle-fee per 30 min	M	DA2
FR-TAR-02	Tariff plans carry time-of-use rate bands: price per kWh valid within day-of-week + time windows	M	DA2
FR-TAR-03	Tariff plans are versioned; sessions record the tariff version used (price changes never rewrite history)	M	DA2

ID	Requirement	MoSCoW	DA
FR-BIL-01	On session completion, PL/SQL bills the session: kWh x band price + fees, creating an INVOICE	M	DA2
FR-BIL-02	Invoice is itemized (energy charge, session fee, idle fee, tax line)	S	DA2
FR-PAY-01	Driver "pays" invoices through a mocked wallet (prepaid balance); no real card data ever stored	M	DA2
FR-PAY-02	Failed wallet payment marks invoice PENDING and session billing state FAILED for retry	S	DA2

4.7 Faults and maintenance

ID	Requirement	MoSCoW	DA
FR-FLT-01	OCPP StatusNotification error codes create FAULT records automatically; operator can file faults manually	M	DA2
FR-FLT-02	Faulted connectors become unavailable for reservation immediately	M	DA2
FR-MNT-01	Operator opens maintenance records: type, description, parts cost, start/end timestamps, resolution	M	DA2
FR-MNT-02	Resolving maintenance on a fault restores connector to AVAILABLE (state machine enforced)	S	DA2

4.8 Reviews and notifications

ID	Requirement	MoSCoW	DA
FR-REV-01	Driver rates a completed session's station (1–5 + comment); one review per session	S	DA2
FR-NTF-01	In-app notifications: reservation confirmation/expiry, session completion, invoice due	S	DA2/3

4.9 Analytics and observability

ID	Requirement	MoSCoW	DA
FR-ANL-01	Operator dashboard: revenue, energy, utilization, active sessions, faults per station	M	DA2/3
FR-ANL-02	DA3: live network load curve, hourly utilization heatmap, per-connector power traces from TimescaleDB	M	DA3
FR-ANL-03	Admin system dashboard: user growth, network KPIs, audit log browser	S	DA2

4.10 Charger/protocol (system actor)

ID	Requirement	MoSCoW	DA
FR-OCPP-01	Simulator fleet runs scripted scenarios: normal charge, simultaneous race for one connector, fault mid-session, no-show reservation, out-of-order meter ticks	M	DA2/3
FR-OCPP-02	OCPP gateway accepts authenticated WebSocket connections per charge point identity	M	DA2

5. Non-Functional Requirements

ID	Category	Requirement	Acceptance criterion
NFR-01	Correctness	No double reservation, no negative energy, no invoice without completed session, ever	Concurrency test suite passes; DB constraints reject violations
NFR-02	Consistency	Reservation overlap prevention is enforced by the database, not app logic	Two concurrent requests: exactly one succeeds (demoed live)
NFR-03	Performance	Interactive API p95 < 300 ms on laptop-scale data (10k sessions)	k6 smoke script report
NFR-04	Ingestion	Simulator fleet of 50 chargers x 5 s ticks ingests without unbounded lag	Pipeline lag metric stays < 30 s in 10-min run
NFR-05	Auditability	Every state-changing mutation has an audit trail row (who, what, when, old/new)	Spot-check in demo; AUDIT_LOG row exists for every mutation
NFR-06	Security	Passwords Argon2-hashed; JWT RBAC; parameterized SQL only; no card data	Security checklist in Section 30 fully ticked
NFR-07	Testability	Core flows covered by automated tests runnable in CI	`npm run test` green in GitHub Actions
NFR-08	Reproducibility	`docker compose up` produces a fully seeded working system on any laptop	Fresh-clone setup documented and verified
NFR-09	Demo-ability	Every showcase feature reachable in <= 2 clicks from a dashboard	Demo rehearsal checklist
NFR-10	Observability	Structured JSON logs with request IDs; health endpoints for API and both DBs	`/health` reports Oracle + Timescale status
NFR-11	Honesty	No claims of production scale without benchmark numbers shown in-repo	README states tested scale explicitly

6. Scope Definition

6.1 Core MVP (the system must ship with all of this)

Stations / charge points / connectors hierarchy; driver/operator/admin roles with JWT auth; station search + map + connector availability; reservations with DB-enforced overlap prevention; full OCPP 1.6J simulator loop producing sessions and meter readings; tariff plans with ToU bands and versioning; PL/SQL billing into itemized invoices; wallet payments; faults + maintenance; operator dashboard with core KPIs; audit logging; Docker Compose with seeded data.

Rationale: this set alone fully satisfies DA1 (rich ER model), DA2 (SQL + PL/SQL + objects + sample data + live demo), and the transactional core of DA3 (telemetry outbox streaming into TimescaleDB).

6.2 Strong portfolio features (high signal, moderate cost)

Live telemetry charts on the operator dashboard (TimescaleDB continuous aggregates); utilization heatmap; keyset-paginated history endpoints; race-condition test suite demonstrated in CI; versioned SQL migrations; OpenAPI docs; LTTB-downsampled power traces; reservation expiry background job; notifications.

6.3 Stretch features (build only if ahead of schedule)

Grafana dashboard over TimescaleDB (1 hour of work, huge demo payoff); CSV export; GraphQL read endpoint beside REST (only with explicit "why both" answer prepared); multi-currency tariffs; ISO 15118 Plug-and-Charge *simulation* of the authorize step (concept discussed, not implemented).

6.4 Explicitly excluded (with reasons)

Excluded	Reason
Real payment gateway (Stripe/Razorpay)	Storing/handling real money flows adds compliance burden (PCI) with zero database-learning value. Mocked wallet keeps the billing model honest.
Microservices / Kafka / RabbitMQ	One node, one team. An outbox table + worker gives the event-driven lesson without the operational tax.
Kubernetes	Docker Compose is the honest deployment for the scale; K8s would be resume-keyword collecting.
Mobile app	Responsive web covers the demo; a second frontend halves quality.
ML (demand prediction, dynamic pricing)	No defensible dataset at student scale; would look like technology collecting.
OCPI roaming	Real multi-party protocol requiring a partner CPO; out of scope, discussed in docs only [11].
Real charger hardware	Impossible in scope; the OCPP simulator is the accepted approach in CSMS testing [10].
Redis caching layer (early)	At our scale Oracle + materialized views + keyset pagination suffice; Redis appears only as an optional stretch with a measured reason.

7. User Roles

7.1 Role definitions

Driver — the customer. Registers EVs, discovers stations on map/list, checks live connector availability, reserves, charges (via the simulated charger assigned their idTag), watches the live session, pays invoices from wallet, reviews stations, browses history and personal energy/cost analytics.

Station Operator — runs assigned stations. Sees station health at a glance (connector states, active sessions), revenue and energy analytics, opens/manages faults and maintenance, forces connector availability changes (OCPP ChangeAvailability), monitors live network load (DA3).

Admin — runs the platform. Manages users and operators, owns stations and tariff plans, reads the audit log, sees network-wide KPIs and growth analytics.

System (OCPP actor) — the charger fleet (simulated). Not a UI user: authenticates per charge point over WebSocket, reports state and metering, receives remote commands. Treating the charger as a first-class actor is what keeps the schema honest.

7.2 RBAC permission matrix

Capability	Driver	Operator	Admin	OCPP actor
Search stations / view availability	yes	yes	yes	n/a
Create/edit own EVs	yes	-	-	n/a
Create reservations	yes	-	-	n/a
Cancel own reservation	yes	operator (station scope)	yes	n/a
Start/stop own session	via charger	remote stop (station scope)	-	start/stop (protocol)
View own invoices/history	yes	-	-	n/a
Pay invoices (wallet)	yes	-	-	n/a
Review stations (own session)	yes	-	-	n/a
View station analytics	-	yes (assigned stations)	yes (network)	n/a
Manage faults/maintenance	-	yes (station scope)	yes	report (protocol)
Manage users/operators	-	-	yes	n/a
Manage stations/charge points	-	-	yes	n/a
Manage tariff plans	-	-	yes	n/a
Read audit log	-	-	yes	n/a
Report meter values / status	-	-	-	yes (protocol identity)

Authorization is enforced twice: NestJS guards for API ergonomics, Oracle grants and `SECURE_PKG.CHECK_ACCESS` calls inside PL/SQL for defense in depth — a nice viva point showing defense is layered, not cosmetic.

PART III

System Architecture (and North Star)

> Masterplan sections 8 and 48. Each decision is written as Decision → Alternatives → Evaluation → Recommendation → Reason (D/A/E/R/R), so every choice can be defended as a trade-off rather than a preference.

---

8. System Architecture

8.1 Decision record: overall architecture style

Decision: Modular monolith (single NestJS deployable) + one worker process + a strict two-database data layer.

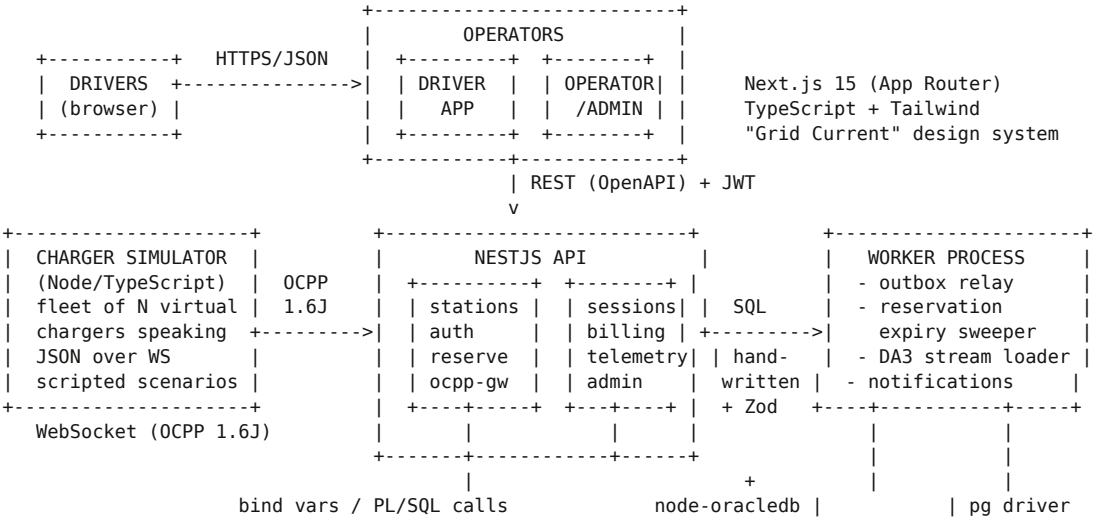
Alternatives: (a) microservices; (b) serverless functions; (c) layered monolith without modules; (d) modular monolith with worker.

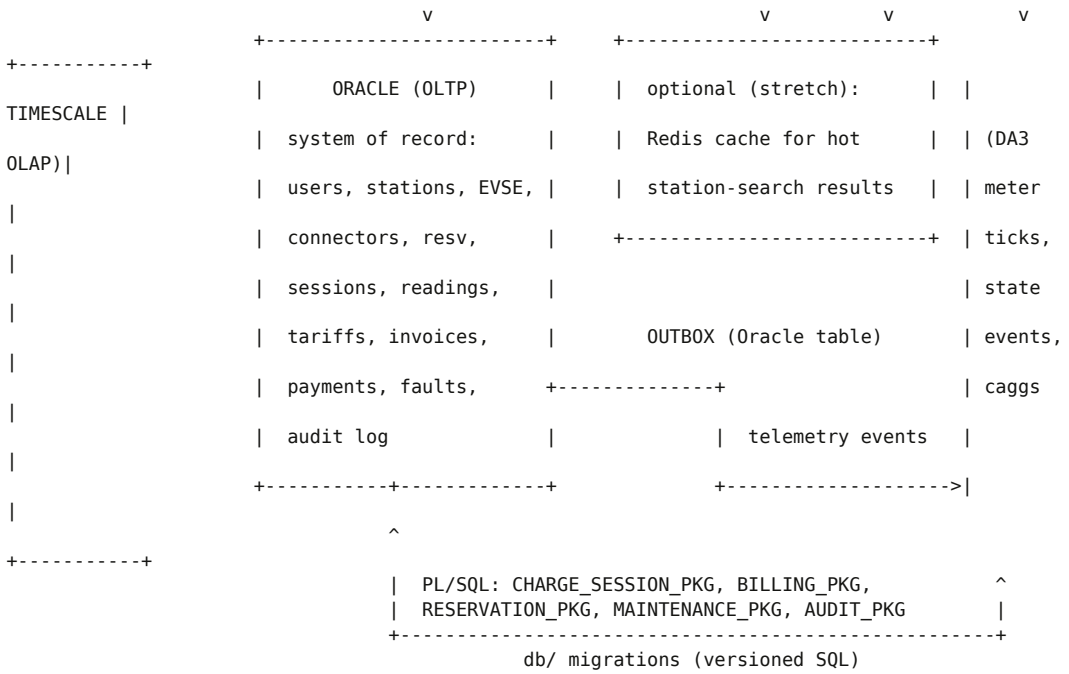
Evaluation: Microservices would split the reservation/billing flow across processes, forcing distributed transactions (sagas, outboxes at the service level) that *obscure* exactly the database-level correctness this course grades; industry guidance is explicit that at low station counts a monolithic CSMS with a relational database is the right call, with microservices earned later by scale [6]. Serverless complicates WebSocket (OCPP needs a persistent socket) and local reproducibility. A feature-less layered monolith permits module boundaries to rot. The modular monolith with a separate worker gives clear boundaries, honest local development, and one process that can hold OCPP sockets while the API stays stateless.

Recommendation: (d) — modular monolith + worker.

Reason: The project's value is database correctness and clarity, not process choreography. A modular monolith keeps every correctness argument (constraints, locks, transactions) inside one database boundary where they are demonstrable — and it is the architecture industry itself recommends at this scale [5][6].

8.2 System context and ASCII architecture diagram





Reading the diagram: all *transactional* truth lives in Oracle. The OCPP gateway and simulator make the database behave like a real charging network. The worker moves telemetry events (outbox pattern, Section 29) from Oracle into TimescaleDB, where analytics live. The optional Redis box is explicitly a stretch item, not a default.

8.3 Component inventory

Component	Tech	Responsibility
Web app	Next.js 15, TypeScript, Tailwind v4	Driver/Operator/Admin UI; RSC for reads, client islands for live data
API	NestJS 11, node-oracledb, Zod, OpenAPI	REST endpoints, RBAC guards, validation, transaction scripts calling PL/SQL
OCPP gateway	NestJS module + `ws`	OCPP 1.6J JSON-over-WebSocket server; per-charger sessions; event emission
Simulator	Node CLI	Fleet of virtual chargers running scripted scenarios (normal, race, fault, no-show)
Worker	Node CLI	Outbox relay to TimescaleDB, reservation expiry sweeper, notification dispatch
Oracle	gvenzl/oracle-free (23ai) container	OLTP system of record + PL/SQL packages
TimescaleDB	timescale/timescaledb container	Hypertables, continuous aggregates, compression, retention (DA3)
Migrations	plain numbered SQL + runner script	Single source of truth for both schemas
CI	GitHub Actions	lint, typecheck, unit tests, DB constraint tests on service containers

8.4 Sub-decisions

Caching. D/A/E/R/R. Decision: no cache in MVP; materialized views + keyset pagination first. Alternatives: Redis everywhere; in-process LRU. Evaluation: at 10k-session scale, Oracle answers interactive queries in low tens of milliseconds when indexed; a cache adds invalidation bugs that would *cost* correctness credibility. Recommendation: optional stretch only, for the station-search endpoint, with a measured before/after. Reason: caching must solve a

measured problem, and none exists yet (NFR-03 threshold is met without it).

Background jobs. Decision: worker process with a job table (reservation expiry, notification dispatch) + outbox relay. Alternatives: cron in API; external scheduler. Evaluation: in-API timers die with the API process and complicate tests; a job table keeps scheduling state in the transactional store where it belongs. Reason: demonstrates queue semantics using pure database features.

Event-driven pieces. Decision: outbox table in Oracle, relayed to TimescaleDB (Section 29). Alternatives: dual-write from API (loses atomicity); full CDC (Oracle XStream/LogMiner — operationally heavy for a student project). Evaluation: the outbox is the classic pattern giving atomic commit + async publish with at-least-once delivery and idempotent loads. Reason: honest event-driven architecture with database-engineering substance, not buzzword streaming.

Authentication. Decision: short-lived access JWT (15 min) + rotating refresh token stored hashed in Oracle; Argon2id password hashing. Alternatives: session cookies; OAuth providers. Evaluation: JWT keeps API stateless and demonstrates token lifecycle; refresh rotation covers revocation. Reason: NFR-06; the security section (30) details the full model.

Observability. Decision: pino structured JSON logs with request IDs; `/health` endpoint reporting Oracle + Timescale connectivity; simulator prints a compact scenario report. Alternatives: OpenTelemetry tracing stack. Evaluation: full OTel is overkill for three processes; structured logs + health checks cover the demo and CI needs. Reason: NFR-10 at appropriate depth.

External integrations. Decision: none in MVP; maps use open tiles (no paid API keys), payments are an internal wallet. Reason: zero-cost reproducibility for evaluators (NFR-08).

---

48. North Star Architecture

> Section 48 is placed here (with Section 8) so the target picture is visible before the detailed designs; the section numbering is preserved from the master outline.

The finished system, described exactly: A visitor opens a polished dark web app. As a **driver**, they find stations on a map, see live connector availability driven by a charger simulator actually speaking OCPP 1.6J, reserve a CCS2 connector for 6:15 PM, and — when the simulated charger plugs in — watch their live session stream kWh and cost in real time; when it stops, an itemized invoice (ToU energy charge + session fee) appears and the wallet pays it. As an **operator**, they watch a data-dense dashboard: connector state grid, active sessions, revenue and energy trends, a fault feed, and — in DA3 — a live network load curve and utilization heatmap served from TimescaleDB continuous aggregates. As an **admin**, they manage users, operators, stations, and tariff versions, and browse a complete audit trail.

Underneath, one Oracle schema (18 relations, BCNF-analyzed, with packages `RESERVATION_PKG`, `CHARGE_SESSION_PKG`, `BILLING_PKG`, `MAINTENANCE_PKG`, `AUDIT_PKG`) holds every business fact; an outbox relay streams telemetry ticks into TimescaleDB hypertables with 1-minute and 1-hour continuous aggregates, compression, and 90-day retention; Docker Compose brings the whole system up seeded on any laptop; GitHub Actions proves the constraint tests — including the two-concurrent-reservations race — on every push. The README leads with an architecture diagram, a live demo GIF, and honest numbers: 50 simulated chargers, 5-second ticks, p95 240 ms, ingestion lag under 30 s. Nothing is claimed that is not demonstrated.

The one-sentence version for the resume: "A two-engine Charging Station Management System — Oracle for ACID-resilient billing and reservations, TimescaleDB for charger telemetry analytics — driven end-to-end by an OCPP 1.6J charger simulator, with a race-condition-proof reservation core and a typography-led Next.js operator dashboard."

PART IV

DA1: Complete ER/EER Design

> Masterplan section 9. This is the design to redraw by hand for the DA1 report. Every entity, key, cardinality, participation constraint, and business rule is stated explicitly so the handwritten version can be reproduced without ambiguity. A rendered ER diagram lives in `diagrams/er-model.mmd`.

9.1 From requirements to entities

Each entity below traces to requirement groups from Section 4 (traceability shown in parentheses). The design contains **21 strong entities, 2 weak entities, 2 specializations, 2 conceptually multivalued attributes, and 5 derived attributes** — rich enough for full marks, small enough for a team of three to implement completely.

Entity inventory:

#	Entity	Type	Purpose	Trace
E1	APP_USER	strong	anyone who authenticates; superclass of Driver/Operator/Admin	AUTH-01..04
E2	WALLET_ACCOUNT	strong, 1:1	prepaid balance for drivers	PAY-01
E3	VEHICLE	strong	driver's EV	DRV-01
E4	CONNECTOR_STANDARD	lookup	Type 2, CCS2, Bharat AC001, CHAdeMO	STN-03
E5	STATION	strong	physical site with geocode	STN-01
E6	CHARGE_POINT	strong	EVSE cabinet with OCPP identity	STN-02, OCPP-02
E7	CONNECTOR	**weak**	physical socket; partial key `connector_no`	STN-03
E8	TARIFF_PLAN	strong	versioned tariff (self-versioning chain)	TAR-01..03
E9	TARIFF_BAND	strong (identifying)	ToU price band within a plan	TAR-02
E10	RESERVATION	associative	driver holds a connector for a window	RSV-01..05
E11	CHARGING_SESSION	associative	energy delivery event (the core entity)	SES-01..06
E12	METER_READING	**weak**	meter tick within a session; partial key `seq_no`	SES-03
E13	SESSION_STATE / CONNECTOR_STATE	lookup	state machines + allowed transitions	SES-02,06

#	Entity	Type	Purpose	Trace
E14	INVOICE	strong	bill for one session	BIL-01
E15	INVOICE_LINE	strong (identifying)	itemized line (energy/fees/tax)	BIL-02
E16	PAYMENT	strong	payment attempt against an invoice	PAY-01..02
E17	WALLET_LEDGER	strong (identifying)	append-only balance movements	PAY-01
E18	FAULT	strong	connector/charge-point fault event	FLT-01..02
E19	MAINTENANCE_RECORD	strong	work performed	MNT-01..02
E20	REVIEW	strong	driver rating tied to a session	REV-01
E21	NOTIFICATION	strong	in-app message	NTF-01
E22	AUDIT_LOG	strong	mutation trail	AUTH-04
E23	OUTBOX_EVENT	strong	telemetry events for the DA3 pipeline	SES-03, ANL-02
E24	VEHICLE_STANDARD_SUPPORT	associative	resolves multivalued "supported standards"	DRV-01
E25	STATION_AMENITY	associative	resolves multivalued "amenities"	STN-01

9.2 Entities, attributes, and keys

Notation: **PK** primary key, **CK** candidate key (unique), **FK** foreign key, **DE** derived, *partial key* for weak entities.

E1 APP_USER — `user_id PK (surrogate)`, `email CK NOT NULL`, `password_hash NOT NULL`, `full_name NOT NULL`, `phone`, `role NOT NULL CHECK (role IN ('DRIVER','OPERATOR','ADMIN'))`, `status NOT NULL`, `created_at NOT NULL`. *Specialization discriminator:* `role`.

E2 WALLET_ACCOUNT — `user_id PK, FK -> APP_USER (1:1)`, `balance NOT NULL CHECK (balance >= 0)` *(DE: derivable from WALLET_LEDGER; cached for O(1) checks — denormalization analyzed in Section 12.6)*, `currency NOT NULL DEFAULT 'INR'`, `updated_at NOT NULL`.

E3 VEHICLE — `vehicle_id PK`, `owner_id FK -> APP_USER`, `make NOT NULL`, `model NOT NULL`, `battery_kwh CHECK (5..250)`, `created_at`. *Multivalued attribute* `supported_standards` *resolved as E24.*

E4 CONNECTOR_STANDARD — `standard_id PK`, `code CK NOT NULL UNIQUE ('TYPE2','CCS2','BHARAT_AC001','BHARAT_DC001','CHADEMO')`, `display_name NOT NULL`, `max_typical_kw`.

E5 STATION — `station_id PK`, `name NOT NULL`, `latitude/longitude NOT NULL (CHECK ranges)`, `address_line, city, state, pincode`, `status NOT NULL`, `operator_id FK -> APP_USER`, `created_at`. *Derived:* `available_connector_count` *(computed, not stored — Section 12.6). Multivalued* `amenities` *resolved as E25.*

E6 CHARGE_POINT — `cp_id PK`, `station_id FK -> STATION`, `ocpp_identity CK NOT NULL UNIQUE` (the OCPP chargeBox identity), `vendor, model, firmware_version`, `status NOT NULL`, `last_boot_at`, `created_at`.

E7 CONNECTOR (weak) — cp_id FK -> CHARGE_POINT (identifying relationship), connector_no *partial key*, PK = (cp_id, connector_no), standard_id FK -> CONNECTOR_STANDARD, max_power_kw CHECK (3..400), status NOT NULL FK -> CONNECTOR_STATE, last_state_change_at, created_at. *A connector cannot exist without its charge point (existence dependency).*

E8 TARIFF_PLAN — plan_id PK, group_id NOT NULL (identity of the tariff across versions), version_no NOT NULL, CK = (group_id, version_no), name NOT NULL, currency NOT NULL DEFAULT 'INR', session_fee CHECK (>= 0) nullable, idle_fee_per_30min nullable, active_from NOT NULL, active_to nullable, supersedes_plan_id FK -> TARIFF_PLAN (self, nullable), created_by FK -> APP_USER. *Versioning rule BR-09.*

E9 TARIFF_BAND — band_id PK, plan_id FK -> TARIFF_PLAN, price_per_kwh NOT NULL CHECK (> 0), day_scope CHECK IN ('ALL','WEEKDAY','WEEKEND'), start_time NOT NULL, end_time NOT NULL, CK = (plan_id, day_scope, start_time). *Business rule BR-08: bands within a plan+day_scope must not overlap (enforced in TARIFF_PKG).*

E10 RESERVATION — reservation_id PK, connector_id FK, user_id FK, vehicle_id FK, start_at NOT NULL, end_at NOT NULL CHECK (end_at > start_at), status NOT NULL FK -> RESERVATION_STATE, created_at, cancelled_at. *Window rule BR-04 (15–120 min); overlap rule BR-05 enforced by RESERVATION_PKG (Section 15/31).*

E11 CHARGING_SESSION — session_id PK, connector_id FK NOT NULL, user_id FK NOT NULL, vehicle_id FK nullable, reservation_id FK UQ nullable (0..1 on each side), tariff_plan_id FK NOT NULL, id_tag NOT NULL, state NOT NULL FK -> SESSION_STATE, billing_state CHECK IN ('UNBILLED','BILLED','FAILED','WAIVED'), started_at, ended_at, start_meter_kwh NOT NULL, end_meter_kwh nullable, energy_kwh DE (end - start, persisted by BILLING_PKG at completion — Section 12.6), stop_reason nullable.

E12 METER_READING (weak) — session_id FK -> CHARGING_SESSION (identifying), seq_no *partial key*, PK = (session_id, seq_no), taken_at NOT NULL, meter_kwh NOT NULL CHECK (>= 0), power_kw, voltage_v, current_a, source CHECK IN ('OCPP','SYNTHETIC'), ingested_at. *Out-of-order rule BR-11: seq_no assigned by gateway per session; monotonic per session.*

E13 State lookups — CONNECTOR_STATE(state_code PK, description, is_terminal): AVAILABLE, OCCUPIED, RESERVED, FAULTED, OFFLINE, UNAVAILABLE. SESSION_STATE(state_code PK, description): RESERVED, PREPARING, CHARGING, SUSPENDED, COMPLETED, CANCELLED, FAILED. Transition matrix lives in PL/SQL CHARGE_SESSION_PKG.TRANSITION (Section 15.2) — the database is the state-machine authority.

E14 INVOICE — invoice_id PK, session_id FK UQ NOT NULL (1:1), tariff_plan_id FK NOT NULL, issued_at NOT NULL, status CHECK IN ('DUE','PAID','FAILED','VOID'), total DE (sum of lines, persisted at issue), currency NOT NULL.

E15 INVOICE_LINE — invoice_id FK (identifying), line_no *partial*, PK = (invoice_id, line_no), kind CHECK IN ('ENERGY','SESSION_FEE','IDLE_FEE','TAX','ADJUSTMENT'), description NOT NULL, quantity NOT NULL, unit, amount NOT NULL CHECK (>= 0).

E16 PAYMENT — payment_id PK, invoice_id FK NOT NULL, amount NOT NULL CHECK (> 0), method CHECK IN ('WALLET') — *specialization: CARD_PAYMENT deliberately not implemented (Section 12.7); the method column is the extension point*, status CHECK IN ('SUCCESS','FAILED'), attempted_at NOT NULL, reference.

E17 WALLET_LEDGER — `user_id FK (identifying)`, `seq_no *partial*`, `PK = (user_id, seq_no)`, `kind CHECK IN ('TOPUP','PAYMENT','REFUND','ADJUSTMENT')`, `amount (signed) NOT NULL`, `balance_after NOT NULL`, `payment_id FK nullable`, `created_at NOT NULL`. *Append-only: UPDATE/DELETE revoked by grant (Section 13.4).*

E18 FAULT — `fault_id PK`, `connector_id FK nullable`, `cp_id FK nullable (CHECK: exactly one of connector/cp is set)`, `code NOT NULL`, `severity CHECK IN ('INFO','WARN','CRITICAL')`, `source CHECK IN ('OCPP','MANUAL')`, `reported_at NOT NULL`, `cleared_at nullable`, `note`.

E19 MAINTENANCE_RECORD — `maint_id PK`, `connector_id FK nullable`, `fault_id FK UQ nullable (a fault is fixed by at most one maintenance record)`, `type CHECK IN ('INSPECTION','REPAIR','REPLACEMENT')`, `description NOT NULL`, `parts_cost CHECK (>= 0)`, `performed_by FK -> APP_USER`, `started_at NOT NULL`, `ended_at nullable`, `outcome`.

E20 REVIEW — `review_id PK`, `session_id FK UQ NOT NULL (one review per session)`, `user_id FK NOT NULL`, `rating CHECK (1..5)`, `comment`, `created_at`. *CK:* `session_id` *(functional dependency: session determines review).*

E21 NOTIFICATION — `notification_id PK`, `user_id FK`, `kind NOT NULL`, `payload (JSON CLOB)`, `read_at`, `created_at`.

E22 AUDIT_LOG — `audit_id PK`, `actor_user_id FK nullable (NULL = system/OCPP)`, `entity_name NOT NULL`, `entity_pk NOT NULL`, `action NOT NULL`, `old_values CLOB (JSON)`, `new_values CLOB (JSON)`, `occurred_at NOT NULL`. *Insert-only by grant.*

E23 OUTBOX_EVENT — `event_id PK`, `kind CHECK IN ('METER_TICK','CONNECTOR_STATE','SESSION_EVENT')`, `payload CLOB (JSON) NOT NULL`, `created_at NOT NULL`, `processed_at nullable`. *DA3 relay cursor; idempotency key = (kind, source ids, seq).*

E24 VEHICLE_STANDARD_SUPPORT — `vehicle_id FK`, `standard_id FK`, `PK = (vehicle_id, standard_id)`. *Resolves multivalued attribute* `VEHICLE.supported_standards`.

E25 STATION_AMENITY — `station_id FK`, `amenity CHECK IN ('CAFE','RESTROOM','WIFI','PARKING','SHOP','CANOPY','CCTV')`, `PK = (station_id, amenity)`. *Resolves multivalued attribute* `STATION.amenities`.

9.3 Relationships, cardinalities, and participation

#	Relationship	Cardinality	Participation	Notes
R1	APP_USER owns VEHICLE	1:N	VEHICLE total, USER partial	a vehicle always has an owner
R2	APP_USER holds WALLET_ACCOUNT	1:1	partial (drivers only)	specialization consequence
R3	APP_USER operates STATION	1:N	STATION total, USER partial	operator assignment
R4	STATION contains CHARGE_POINT	1:N	CP total, STATION partial	composition
R5	CHARGE_POINT has CONNECTOR	1:N **identifying**	CONNECTOR total	weak entity; partial key `connector_no`

#	Relationship	Cardinality	Participation	Notes
R6	CONNECTOR_STANDARD classifies CONNECTOR	1:N	CONNECTOR total	lookup
R7	TARIFF_PLAN contains TARIFF_BAND	1:N **identifying**	BAND total	bands exist only within a plan
R8	TARIFF_PLAN supersedes TARIFF_PLAN	1:N (self)	partial	version chain BR-09
R9	APP_USER reserves CONNECTOR (via RESERVATION)	M:N with attributes	associative entity	attributes: window, status
R10	APP_USER charges at CONNECTOR (via CHARGING_SESSION)	M:N with attributes	associative entity	the core business event
R11	CHARGING_SESSION produces METER_READING	1:N **identifying**	READING total, SESSION partial	weak entity; partial key `seq_no`
R12	CHARGING_SESSION billed as INVOICE	1:1	INVOICE total, SESSION partial (0..1)	session exists before bill
R13	INVOICE itemized as INVOICE_LINE	1:N identifying	LINE total	
R14	INVOICE settled by PAYMENT	1:N	PAYMENT total, INVOICE partial	retry attempts allowed
R15	PAYMENT debits WALLET_LEDGER	1:1	partial	ledger entry references payment
R16	APP_USER has WALLET_LEDGER	1:N identifying	LEDGER total	append-only history
R17	CONNECTOR suffers FAULT	1:N	FAULT total (to connector or CP)	CHECK exactly-one-target
R18	FAULT addressed by MAINTENANCE_RECORD	1:0..1	partial	UQ fault_id
R19	APP_USER performs MAINTENANCE_RECORD	1:N	RECORD total, USER partial	
R20	CHARGING_SESSION reviewed (REVIEW)	1:0..1	partial both	UQ session_id
R21	APP_USER receives NOTIFICATION	1:N	partial	
R22	APP_USER performs AUDIT_LOG action	1:N	partial	NULL actor = system
R23	VEHICLE supports CONNECTOR_STANDARD (E24)	M:N	total both sides	resolves multivalued attribute
R24	STATION offers amenity (E25)	M:N (with value-set)	total	resolves multivalued attribute
R25	RESERVATION converts to CHARGING_SESSION	1:0..1	partial	UQ reservation_id

9.3.1 Compact ER overview (hand-copy friendly)

```
APP_USER 1──N VEHICLE            APP_USER 1──1 WALLET_ACCOUNT
APP_USER 1──N STATION (operates) APP_USER 1──N NOTIFICATION / AUDIT_LOG
STATION  1──N CHARGE_POINT 1──N CONNECTOR N──1 CONNECTOR_STANDARD
APP_USER M── CONNECTOR ──M via RESERVATION (window,status)
APP_USER M── CONNECTOR ──M via CHARGING_SESSION ──N METER_READING (weak, seq_no)
TARIFF_PLAN 1──N TARIFF_BAND ;  TARIFF_PLAN supersedes TARIFF_PLAN (self)
```

```
CHARGING_SESSION 1──1 INVOICE 1──N INVOICE_LINE ; INVOICE 1──N PAYMENT
PAYMENT 1──1 WALLET_LEDGER ;  CONNECTOR 1──N FAULT 0..1──1 MAINTENANCE_RECORD
CHARGING_SESSION 0..1──1 REVIEW ;  RESERVATION 0..1──1 CHARGING_SESSION
VEHICLE M──N CONNECTOR_STANDARD (support) ;  STATION M──N AMENITY (value set)
```

9.4 EER constructs (specialization, multivalued, derived)

Specialization 1 — APP_USER superclass. Subclasses DRIVER, STATION_OPERATOR, ADMIN. Constraints: **disjoint** (one role per user; enforced by CHECK) and **total** (every user has exactly one role). Predicate-defined (on `role`). Relational mapping (Section 10.2): single table with role discriminator + optional subclass tables where subclass-specific attributes exist (WALLET_ACCOUNT for DRIVER). We considered three tables joined on user_id and rejected it: two extra joins on every auth lookup for zero normalization gain — the discriminator column is a key, not a fact, so 3NF is not violated. This trade-off is a strong viva discussion.

Specialization 2 — PAYMENT superclass. Subclasses WALLET_PAYMENT (implemented) and CARD_PAYMENT (deliberately future). **Partial, disjoint.** Only WALLET_PAYMENT rows exist today; the discriminator `method` plus a documented extension plan shows forward-design thinking without speculative tables.

Multivalued attributes (conceptually justified, not decorative). A vehicle genuinely supports several connector standards (a 2024 Tata Nexon EV: Type 2 AC + CCS2 DC) — modeled as VEHICLE_STANDARD_SUPPORT. A station genuinely offers several amenities — modeled as STATION_AMENITY with a constrained value set. Both would be 1NF violations as comma-separated lists (Section 12.1 demonstrates exactly that failure).

Derived attributes (5). WALLET_ACCOUNT.balance (sum of ledger), CHARGING_SESSION.energy_kwh (meter delta), INVOICE.total (sum of lines), STATION.available_connector_count (count over connectors), KPI metrics (uptime, utilization). Each is either computed on read (station count) or persisted by the package that owns the invariant (energy_kwh, invoice total, balance_after) — the "store derived data only at the moment its precondition is locked" rule, explained in Section 12.6.

9.5 Business rules (BR-01 … BR-14)

BR	Rule	Enforced by
01	Email is globally unique; passwords stored only as Argon2 hashes	CK + app policy
02	A wallet balance can never go negative	CHECK + ledger-debit procedure
03	A connector belongs to exactly one charge point; a charge point to one station	FK chain
04	Reservation windows span 15–120 minutes and must start in the future	CHECK + package validation
05	Two reservations on the same connector may never overlap	RESERVATION_PKG (SELECT FOR UPDATE + overlap query) — Section 31
06	A reservation can convert to at most one session; a session references at most one reservation	UQ reservation_id
07	Connector status is only set by OCPP events or the state machine, never ad-hoc UPDATE	grants + trigger

BR	Rule	Enforced by
08	Tariff bands within a plan and day-scope may not overlap in time-of-day	TARIFF_PKG validation
09	Tariff plans are immutable once active; changes create a new version superseding the old	self-FK + package
10	A session bills exactly once; INVOICE is 1:1 with COMPLETED sessions	UQ session_id + BILLING_PKG
11	Meter readings are monotonic per session by seq_no; meter_kwh non-decreasing within tolerance	PK + package check on ingest
12	A faulted connector is immediately unbookable	status check in reservation package
13	One review per completed session, author must be the session's driver	UQ + package check
14	Every state-changing mutation writes AUDIT_LOG	triggers + package discipline

---

9.6 Weak entities — why they are genuinely weak

CONNECTOR is identified by its charge point plus `connector_no` (OCPP addressing is exactly `chargePointIdentity/connectorId` [1]). It has no global identity independent of its parent; delete the charge point and the connector ceases to exist. That is the textbook definition of a weak entity with an identifying relationship — and it mirrors the physical world, which makes it easy to defend.

METER_READING is identified by its session plus `seq_no`. A reading without its session is meaningless; readings are existence-dependent and share their parent's fate. The surrogate `reading_id` used in Oracle is an implementation convenience; the *logical* key is (session_id, seq_no), and the UNIQUE constraint proves we understand the difference.

9.7 Design decisions a viva will probe

1. **Why is RESERVATION an entity and not a M:N relation directly?** Because it carries attributes (window, status, timestamps) — a M:N relationship with attributes is mapped as an associative entity; also because the *no-overlap* invariant needs a key space to constrain.
2. **Why lookups for states instead of CHECK enums?** Because transitions are data (driven by protocol events), need descriptions in UI, and CHECK-only states cannot express "which transitions are legal from which state" — the matrix lives in CHARGE_SESSION_PKG.
3. **Why is energy_kwh stored if derived?** Because the meter may keep reporting corrections after end; the completed-session delta is a *business fact* frozen at billing time (BR-10), not a live computation. The derivation rule is documented, which is what "derived attribute" means in a report.
4. **Why not one STATION_ADDRESS separate entity?** Address is single-valued 1:1 data; splitting it adds a join with no FD-based justification. Section 12.4 shows the *opposite* example (where splitting IS required).
5. **Why AUDIT_LOG as JSON blobs instead of per-table history tables?** One append-only trail covers 15 tables with a uniform query interface; per-table history would triple table count for no analytical gain at this scale. Trade-off honestly stated: JSON columns are schema-flexible but not FK-constrainable — acceptable for an audit trail whose purpose is forensics, not referential integrity.

PART V

Relational Schema (ER → Relations)

> Masterplan section 10. Every relation is written in assignment notation with datatype, keys, and constraints, followed by the mapping rationale and the index plan. Oracle 23ai datatypes are used (DA2 runs on gvenzl/oracle-free); `NUMBER`, `TIMESTAMP`, `CLOB` map directly to the handwritten report.

10.1 Relation inventory

25 relations: 21 business tables + 3 state lookups + 1 audit + 1 outbox. Naming: `SNAKE_CASE`, singular entity names, `*_id` surrogate keys, `*_at` timestamps. All tables `NOMONITORING`-friendly and created by versioned migration `db/oracle/V001__core_schema.sql`.

10.2 The relations

Identity, wallet, vehicles

```
APP_USER(
  user_id        NUMBER(9)        PK (sequence),
  email          VARCHAR2(255)    NOT NULL, UNIQUE,
  password_hash  VARCHAR2(97)     NOT NULL,             -- Argon2id encoded
  full_name      VARCHAR2(120)    NOT NULL,
  phone          VARCHAR2(20),
  role           VARCHAR2(12)     NOT NULL CHECK (role IN ('DRIVER','OPERATOR','ADMIN')),
  status         VARCHAR2(10)     NOT NULL DEFAULT 'ACTIVE' CHECK (status IN
    ('ACTIVE','SUSPENDED')),
  created_at     TIMESTAMP(6)     NOT NULL DEFAULT SYSTIMESTAMP
)
-- CK: email ; CK: (email) is the natural/candidate key, user_id surrogate

WALLET_ACCOUNT(
  user_id    NUMBER(9)      PK, FK -> APP_USER(user_id),
  balance    NUMBER(12,2)   NOT NULL CHECK (balance >= 0),
  currency   CHAR(3)        NOT NULL DEFAULT 'INR',
  updated_at TIMESTAMP(6)   NOT NULL
)
-- 1:1 with APP_USER (partial: DRIVER rows only) ; balance = derived, maintained by BILLING_PKG

WALLET_LEDGER(
  user_id       NUMBER(9)     FK -> APP_USER,        -- identifying
  seq_no        NUMBER(9)     NOT NULL,              -- partial key
  -- PK (user_id, seq_no)
  kind          VARCHAR2(12)  NOT NULL CHECK (kind IN ('TOPUP','PAYMENT','REFUND','ADJUSTMENT')),
  amount        NUMBER(12,2)  NOT NULL,              -- signed
  balance_after NUMBER(12,2)  NOT NULL CHECK (balance_after >= 0),
  payment_id    NUMBER(12)    FK -> PAYMENT(payment_id) nullable,
  created_at    TIMESTAMP(6)  NOT NULL,
  note          VARCHAR2(200)
)

VEHICLE(
  vehicle_id  NUMBER(9)     PK,
  owner_id    NUMBER(9)     NOT NULL FK -> APP_USER(user_id),
  make        VARCHAR2(40)  NOT NULL,
  model       VARCHAR2(60)  NOT NULL,
```

```
  battery_kwh NUMBER(6,1)   NOT NULL CHECK (battery_kwh BETWEEN 5 AND 250),
  created_at  TIMESTAMP(6)  NOT NULL
)

VEHICLE_STANDARD_SUPPORT(
  vehicle_id  NUMBER(9)  FK -> VEHICLE,   -- PK (vehicle_id, standard_id)
  standard_id NUMBER(4)  FK -> CONNECTOR_STANDARD
)
-- resolves multivalued attribute supported_standards
```

Stations and hardware

```
CONNECTOR_STANDARD(
  standard_id    NUMBER(4)     PK,
  code           VARCHAR2(16)  NOT NULL UNIQUE,   --
    'TYPE2','CCS2','BHARAT_AC001','BHARAT_DC001','CHADEMO'
  display_name   VARCHAR2(60)  NOT NULL,
  max_typical_kw NUMBER(6,1)
)

STATION(
  station_id   NUMBER(9)      PK,
  name         VARCHAR2(120)  NOT NULL,
  latitude     NUMBER(9,6)    NOT NULL CHECK (latitude BETWEEN 6 AND 37),      -- India bounds
  longitude    NUMBER(9,6)    NOT NULL CHECK (longitude BETWEEN 68 AND 98),
  address_line VARCHAR2(200)  NOT NULL,
  city         VARCHAR2(80)   NOT NULL,
  state        VARCHAR2(80)   NOT NULL,
  pincode      VARCHAR2(10),
  status       VARCHAR2(12)   NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','INACTIVE')),
  operator_id  NUMBER(9)      NOT NULL FK -> APP_USER(user_id),
  created_at   TIMESTAMP(6)   NOT NULL
)
-- CK candidate: (name, city) is NOT declared unique (two branches may share a name) - documented

STATION_AMENITY(
  station_id NUMBER(9)    FK -> STATION,   -- PK (station_id, amenity)
  amenity    VARCHAR2(12) NOT NULL CHECK (amenity IN
             ('CAFE','RESTROOM','WIFI','PARKING','SHOP','CANOPY','CCTV'))
)

CHARGE_POINT(
  cp_id            NUMBER(9)     PK,
  station_id       NUMBER(9)     NOT NULL FK -> STATION(station_id),
  ocpp_identity    VARCHAR2(60)  NOT NULL UNIQUE,     -- OCPP chargeBox identity
  vendor           VARCHAR2(60),
  model            VARCHAR2(60),
  firmware_version VARCHAR2(40),
  status           VARCHAR2(10)  NOT NULL DEFAULT 'OFFLINE'
                   CHECK (status IN ('ONLINE','OFFLINE','FAULTED')),
  last_boot_at     TIMESTAMP(6),
  created_at       TIMESTAMP(6)  NOT NULL
)

CONNECTOR(
  cp_id               NUMBER(9)    FK -> CHARGE_POINT(cp_id),   -- identifying
  connector_no        NUMBER(2)    NOT NULL CHECK (connector_no BETWEEN 1 AND 8),
  -- PK (cp_id, connector_no)
  standard_id         NUMBER(4)    NOT NULL FK -> CONNECTOR_STANDARD(standard_id),
  max_power_kw        NUMBER(6,2)  NOT NULL CHECK (max_power_kw BETWEEN 3 AND 400),
  status              VARCHAR2(12) NOT NULL DEFAULT 'OFFLINE'
                      FK -> CONNECTOR_STATE(state_code),
  last_state_change_at TIMESTAMP(6),
  created_at          TIMESTAMP(6) NOT NULL
)
-- weak entity: logical key (cp_id, connector_no)
```

Tariffs

```
TARIFF_PLAN(
```

```
  plan_id            NUMBER(9)     PK,
  group_id           NUMBER(9)     NOT NULL,           -- tariff identity across versions
  version_no         NUMBER(3)     NOT NULL,
  -- UNIQUE (group_id, version_no)
  name               VARCHAR2(80)  NOT NULL,
  currency           CHAR(3)       NOT NULL DEFAULT 'INR',
  session_fee        NUMBER(8,2)   CHECK (session_fee >= 0),       -- nullable = no fee
  idle_fee_per_30min NUMBER(8,2)   CHECK (idle_fee_per_30min >= 0),
  active_from        TIMESTAMP(6)  NOT NULL,
  active_to          TIMESTAMP(6),                                  -- NULL = current
  supersedes_plan_id NUMBER(9)     FK -> TARIFF_PLAN(plan_id),
  created_by         NUMBER(9)     NOT NULL FK -> APP_USER(user_id)
)

TARIFF_BAND(
  band_id       NUMBER(9)     PK,
  plan_id       NUMBER(9)     NOT NULL FK -> TARIFF_PLAN(plan_id),
  day_scope     VARCHAR2(8)   NOT NULL CHECK (day_scope IN ('ALL','WEEKDAY','WEEKEND')),
  start_time    TIMESTAMP(0)  NOT NULL,    -- time-of-day, normalized date 01-JAN-2000
  end_time      TIMESTAMP(0)  NOT NULL,
  price_per_kwh NUMBER(8,4)   NOT NULL CHECK (price_per_kwh > 0),
  -- UNIQUE (plan_id, day_scope, start_time)
  -- band non-overlap enforced by TARIFF_PKG (BR-08)
)
```

Reservations, sessions, metering

```
RESERVATION_STATE(
  state_code  VARCHAR2(12) PK,   -- BOOKED, CONVERTED, COMPLETED, CANCELLED, EXPIRED, NO_SHOW
  description VARCHAR2(80) NOT NULL
)

RESERVATION(
  reservation_id NUMBER(9)    PK,
  connector_id   VARCHAR2(72) NOT NULL,     -- FK -> CONNECTOR; encoded 'cp_id:connector_no'
                                            -- (composite-FK alternative noted in 10.3)
  user_id        NUMBER(9)    NOT NULL FK -> APP_USER(user_id),
  vehicle_id     NUMBER(9)    NOT NULL FK -> VEHICLE(vehicle_id),
  start_at       TIMESTAMP(6) NOT NULL,
  end_at         TIMESTAMP(6) NOT NULL,
  status         VARCHAR2(12) NOT NULL FK -> RESERVATION_STATE(state_code),
  created_at     TIMESTAMP(6) NOT NULL,
  cancelled_at   TIMESTAMP(6),
  CHECK (end_at > start_at),
  -- overlap prevention: RESERVATION_PKG + BR-05 (Section 31); Oracle has no exclusion
  -- constraints, so the invariant lives in the package + locking (documented trade-off)
)

SESSION_STATE(
  state_code  VARCHAR2(12) PK,   -- RESERVED, PREPARING, CHARGING, SUSPENDED, COMPLETED, CANCELLED,
    FAILED
  description VARCHAR2(80) NOT NULL,
  is_terminal NUMBER(1)    NOT NULL CHECK (is_terminal IN (0,1))
)

CONNECTOR_STATE(
  state_code  VARCHAR2(12) PK,   -- AVAILABLE, OCCUPIED, RESERVED, FAULTED, OFFLINE, UNAVAILABLE
  description VARCHAR2(80) NOT NULL
)

CHARGING_SESSION(
  session_id      NUMBER(12)    PK,
  connector_id    VARCHAR2(72)  NOT NULL FK -> CONNECTOR,
  user_id         NUMBER(9)     NOT NULL FK -> APP_USER(user_id),
  vehicle_id      NUMBER(9)     FK -> VEHICLE(vehicle_id),
  reservation_id  NUMBER(9)     UNIQUE FK -> RESERVATION(reservation_id),   -- 0..1
  tariff_plan_id  NUMBER(9)     NOT NULL FK -> TARIFF_PLAN(plan_id),
  id_tag          VARCHAR2(40)  NOT NULL,
  state           VARCHAR2(12)  NOT NULL FK -> SESSION_STATE(state_code),
  billing_state   VARCHAR2(10)  NOT NULL DEFAULT 'UNBILLED'
                  CHECK (billing_state IN ('UNBILLED','BILLED','FAILED','WAIVED')),
  started_at      TIMESTAMP(6),
```

```
  ended_at        TIMESTAMP(6),
  start_meter_kwh NUMBER(10,3)  NOT NULL CHECK (start_meter_kwh >= 0),
  end_meter_kwh   NUMBER(10,3),
  energy_kwh      NUMBER(10,3),             -- derived, frozen by BILLING_PKG (BR-10)
  stop_reason     VARCHAR2(40),
  created_at      TIMESTAMP(6)  NOT NULL,
  CHECK (end_meter_kwh IS NULL OR end_meter_kwh >= start_meter_kwh)
)

METER_READING(
  session_id NUMBER(12)    FK -> CHARGING_SESSION(session_id),   -- identifying
  seq_no     NUMBER(9)     NOT NULL,                             -- partial key
  -- PK (session_id, seq_no)
  taken_at   TIMESTAMP(6)  NOT NULL,
  meter_kwh  NUMBER(10,3)  NOT NULL CHECK (meter_kwh >= 0),
  power_kw   NUMBER(8,3),
  voltage_v  NUMBER(8,1),
  current_a  NUMBER(8,2),
  source     VARCHAR2(10)  NOT NULL DEFAULT 'OCPP' CHECK (source IN ('OCPP','SYNTHETIC')),
  ingested_at TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP
)
```

Billing

```
INVOICE(
  invoice_id    NUMBER(12)    PK,
  session_id    NUMBER(12)    NOT NULL UNIQUE FK -> CHARGING_SESSION(session_id),  -- 1:1
  tariff_plan_id NUMBER(9)    NOT NULL FK -> TARIFF_PLAN(plan_id),
  issued_at     TIMESTAMP(6)  NOT NULL,
  status        VARCHAR2(8)   NOT NULL CHECK (status IN ('DUE','PAID','FAILED','VOID')),
  total         NUMBER(12,2)  NOT NULL CHECK (total >= 0),   -- derived: sum(lines), frozen
  currency      CHAR(3)       NOT NULL
)

INVOICE_LINE(
  invoice_id  NUMBER(12)    FK -> INVOICE(invoice_id),   -- identifying
  line_no     NUMBER(3)     NOT NULL,                    -- partial key
  -- PK (invoice_id, line_no)
  kind        VARCHAR2(14)  NOT NULL CHECK (kind IN
    ('ENERGY','SESSION_FEE','IDLE_FEE','TAX','ADJUSTMENT')),
  description VARCHAR2(120) NOT NULL,
  quantity    NUMBER(12,4)  NOT NULL,
  unit        VARCHAR2(12),
  amount      NUMBER(12,2)  NOT NULL CHECK (amount >= 0)
)

PAYMENT(
  payment_id   NUMBER(12)    PK,
  invoice_id   NUMBER(12)    NOT NULL FK -> INVOICE(invoice_id),
  amount       NUMBER(12,2)  NOT NULL CHECK (amount > 0),
  method       VARCHAR2(10)  NOT NULL DEFAULT 'WALLET' CHECK (method IN ('WALLET')),  --
    CARD_PAYMENT future
  status       VARCHAR2(8)   NOT NULL CHECK (status IN ('SUCCESS','FAILED')),
  attempted_at TIMESTAMP(6)  NOT NULL,
  reference    VARCHAR2(60)
)
```

Operations, feedback, platform

```
FAULT(
  fault_id    NUMBER(9)     PK,
  connector_id VARCHAR2(72) FK -> CONNECTOR,
  cp_id       NUMBER(9)     FK -> CHARGE_POINT(cp_id),
  code        VARCHAR2(40)  NOT NULL,             -- OCPP StatusNotification error code or manual
  severity    VARCHAR2(8)   NOT NULL CHECK (severity IN ('INFO','WARN','CRITICAL')),
  source      VARCHAR2(8)   NOT NULL CHECK (source IN ('OCPP','MANUAL')),
  reported_at TIMESTAMP(6)  NOT NULL,
  cleared_at  TIMESTAMP(6),
  note        VARCHAR2(400),
  CHECK ( (connector_id IS NOT NULL AND cp_id IS NULL) OR (connector_id IS NULL AND cp_id IS NOT
    NULL) )
```

```
)

MAINTENANCE_RECORD(
  maint_id     NUMBER(9)     PK,
  connector_id VARCHAR2(72)  FK -> CONNECTOR,
  fault_id     NUMBER(9)     UNIQUE FK -> FAULT(fault_id),   -- 1:0..1
  type         VARCHAR2(14)  NOT NULL CHECK (type IN ('INSPECTION','REPAIR','REPLACEMENT')),
  description  VARCHAR2(400) NOT NULL,
  parts_cost   NUMBER(10,2)  CHECK (parts_cost >= 0),
  performed_by NUMBER(9)     NOT NULL FK -> APP_USER(user_id),
  started_at   TIMESTAMP(6)  NOT NULL,
  ended_at     TIMESTAMP(6),
  outcome      VARCHAR2(400)
)

REVIEW(
  review_id  NUMBER(9)    PK,
  session_id NUMBER(12)   NOT NULL UNIQUE FK -> CHARGING_SESSION(session_id),   -- CK: session_id
  user_id    NUMBER(9)    NOT NULL FK -> APP_USER(user_id),
  rating     NUMBER(1)    NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment    VARCHAR2(500),
  created_at TIMESTAMP(6) NOT NULL
)

NOTIFICATION(
  notification_id NUMBER(10)   PK,
  user_id         NUMBER(9)    NOT NULL FK -> APP_USER(user_id),
  kind            VARCHAR2(30) NOT NULL,
  payload         CLOB         NOT NULL CHECK (payload IS JSON),
  read_at         TIMESTAMP(6),
  created_at      TIMESTAMP(6) NOT NULL
)

AUDIT_LOG(
  audit_id      NUMBER(14)    PK,
  actor_user_id NUMBER(9)     FK -> APP_USER(user_id),      -- NULL = system/OCPP
  entity_name   VARCHAR2(40)  NOT NULL,
  entity_pk     VARCHAR2(72)  NOT NULL,
  action        VARCHAR2(20)  NOT NULL,
  old_values    CLOB          CHECK (old_values IS JSON OR old_values IS NULL),
  new_values    CLOB          CHECK (new_values IS JSON OR new_values IS NULL),
  occurred_at   TIMESTAMP(6)  NOT NULL DEFAULT SYSTIMESTAMP
)

OUTBOX_EVENT(
  event_id     NUMBER(16)   PK,
  kind         VARCHAR2(20) NOT NULL CHECK (kind IN
    ('METER_TICK','CONNECTOR_STATE','SESSION_EVENT')),
  payload      CLOB         NOT NULL CHECK (payload IS JSON),
  created_at   TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP,
  processed_at TIMESTAMP(6)
)
```

10.3 Mapping decisions (ER pattern → relational pattern)

ER construct	Mapped to	Why
Strong entity	table + surrogate PK + declared CK	surrogate keys avoid composite-FK cascades in URLs and child tables
Weak entity (CONNECTOR, METER_READING, INVOICE_LINE, WALLET_LEDGER)	composite PK (parent FK + partial key)	preserves identification dependency; surrogate dropped deliberately on these four
1:1 (INVOICE-SESSION)	FK + UNIQUE	UNIQUE on FK enforces the "one" side
1:1 partial (WALLET_ACCOUNT-USER)	FK as PK of child	child existence-dependent

ER construct	Mapped to	Why
M:N with attributes (reservations, sessions)	associative table with own PK	attributes need a home; own PK simplifies child FKs
Multivalued attribute	new relation with composite PK	the 1NF-safe form
Disjoint total specialization (USER)	single table + role CHECK	discriminator is a key, not a fact; avoids 2 joins per auth (Section 9.4)
Partial specialization (PAYMENT)	single table + method CHECK + extension note	no speculative CARD_PAYMENT table
Composite FK for CONNECTOR children	encoded `connector_id` VARCHAR2('cp:no') + FK	Oracle supports composite FKs, but an encoded single-column key keeps child tables (reservation, session, fault, telemetry outbox) uniform; trade-off documented, uniqueness still guaranteed by the parent's composite PK
Derived attributes	computed at read OR frozen at transition by owning package	Section 12.6

10.4 Candidate keys summary

Table	Candidate keys	Chosen PK	Note
APP_USER	{email}, {user_id}	user_id	email kept UNIQUE
CONNECTOR	{cp_id, connector_no}, {connector_id encoded}	(cp_id, connector_no)	encoded column is the FK handle
TARIFF_PLAN	{group_id, version_no}, {plan_id}	plan_id	version pair UNIQUE
REVIEW	{session_id}, {review_id}	review_id	session_id UNIQUE enforces one-review rule
CHARGING_SESSION	{session_id}, {reservation_id} (partial)	session_id	reservation_id UNIQUE
WALLET_LEDGER	{user_id, seq_no}, {payment_id} (partial)	(user_id, seq_no)	payment_id UNIQUE-nullable
STATION	{user-chosen name+city}? rejected	station_id	names are not reliable identifiers (documented)

10.5 Index plan (beyond PKs/UNIQUEs)

Index	Table (columns)	Serves
IX_SESSION_USER_TIME	CHARGING_SESSION(user_id, started_at DESC)	driver history (FR-DRV-03)
IX_SESSION_CONN_STATE	CHARGING_SESSION(connector_id, state)	active-session lookups
IX_SESSION_STARTED	CHARGING_SESSION(started_at)	time-window analytics
IX_RES_CONN_START	RESERVATION(connector_id, start_at)	overlap check query
IX_RES_USER	RESERVATION(user_id, status)	"my bookings"

Index	Table (columns)	Serves
IX_READING_TAKEN	METER_READING(taken_at)	time-range scans (also DA3 pre-filter)
IX_FAULT_CONN_OPEN	FAULT(connector_id, cleared_at)	open-fault feed
IX_STATION_GEO	STATION(city, status)	search prefilter; true geo via bounding-box predicate on lat/lng
IX_OUTBOX_UNPROCESSED	OUTBOX_EVENT(processed_at, created_at)	relay cursor
IX_INVOICE_STATUS	INVOICE(status, issued_at)	"due invoices" worklist

Rationale: every index maps to a named query in Section 17 — no speculative indexes. Composite orderings follow the equality-then-range rule (e.g., `connector_id` equality before `start_at` range).

PART VI

Functional Dependencies, Normalization, and BCNF

> Masterplan sections 11–12. This part does not *claim* the schema is normalized; it *proves* it for the non-trivial relations and demonstrates the decompositions that produced it. Four worked examples use deliberately naive "first draft" EV relations of the kind students actually write, and normalize them into the final schema.

---

11. Functional Dependencies

11.1 Method

For each relation we list: the meaningful FDs, then derive the candidate keys (attribute closure), then state the highest normal form and why. FDs come from business rules (Section 9.5), not from data samples — a data sample can never prove an FD, only falsify one (a point worth saying out loud in the viva).

11.2 FDs for the core relations

CHARGING_SESSION — `session_id` single-attribute key. FDs: `session_id -> {connector_id, user_id, vehicle_id, reservation_id, tariff_plan_id, id_tag, state, billing_state, started_at, ended_at, start_meter_kwh, end_meter_kwh, energy_kwh, stop_reason}`; `reservation_id -> session_id` (UNIQUE, partial: only for converted reservations); `session_id -> energy_kwh` is *derived* (meter delta) and stored by BILLING_PKG at the freeze point. Candidate key: {session_id} (verify: closure of session_id covers all attributes — yes). Second CK: {reservation_id} on the subset of rows where it is non-null (partial key — documented). Since every nontrivial FD has a superkey determinant (session_id), **CHARGING_SESSION is in BCNF trivially**.

METER_READING(session_id, seq_no) — FDs: `{session_id, seq_no} -> {taken_at, meter_kwh, power_kw, voltage_v, current_a, source, ingested_at}`. Also `seq_no` alone determines nothing (readings from different sessions collide). CK: {(session_id, seq_no)}. No partial dependencies (nothing depends on seq_no

alone) → **2NF; no transitive deps among non-key attrs (source/taken_at do not determine meter values) → 3NF; single CK with all FDs from the whole key → BCNF**.

CONNECTOR(cp_id, connector_no) — FDs: `{cp_id, connector_no} -> {standard_id, max_power_kw, status, last_state_change_at, created_at}`; `{cp_id, connector_no} -> connector_id` (the encoded handle); `connector_id -> {cp_id, connector_no}` (bijective). Two CKs: {(cp_id, connector_no)} and {connector_id}. Every determinant is a superkey → **BCNF despite two candidate keys** — the exact situation where students often wrongly cry "not BCNF".

TARIFF_BAND(band_id, plan_id, day_scope, start_time, end_time, price_per_kwh) — FDs: `band_id -> all`; `{plan_id, day_scope, start_time} -> {band_id, end_time, price_per_kwh}` (the business key identifies the band). CKs: {band_id}, {plan_id, day_scope, start_time}. Both determinants are superkeys → **BCNF**. Non-overlap of intervals is *not* an FD — it is a temporal constraint (BR-08), which FDs cannot express; it is enforced procedurally. Being able to say "this invariant is beyond FDs, so it lives in the package" is exactly the kind of precision BCNF grading rewards.

APP_USER — `user_id -> all`; `email -> all` (email is a natural key: it determines the person's record). CKs: {user_id}, {email}. Both superkeys → **BCNF**.

RESERVATION — `reservation_id -> all`; no other determinants (start_at/end_at do not determine status: the same window can be booked, cancelled, or converted). CK: {reservation_id} → **BCNF**. The *interesting* constraint (no two overlapping windows per connector) is again non-FD, handled in Section 31.

INVOICE / INVOICE_LINE / PAYMENT / WALLET_LEDGER / FAULT / MAINTENANCE_RECORD / REVIEW / NOTIFICATION / AUDIT_LOG / OUTBOX_EVENT — each has a single-attribute surrogate key (or a whole-key composite for the identifying pairs) and no nontrivial FDs beyond key -> attributes. All are **BCNF trivially**; the interesting analyses are the four worked examples below.

---

12. Normalization and BCNF — worked proofs

12.1 Example 1 — 1NF violation: multivalued attributes as lists

Naive draft (as students actually write it):

```
VEHICLE_FLAT(vehicle_id, make, model, battery_kwh, supported_standards, amenities_of_home_station)
V1 | Tata | Nexon EV | 40.5 | "TYPE2,CCS2" | ...
```

`supported_standards = "TYPE2,CCS2"` is a repeating group inside one column. 1NF requires atomic values: queries like "find vehicles supporting CCS2" need substring matching (`LIKE '%CCS2%'`), which is wrong the moment a standard named "CCS2x" appears, and the DBMS cannot enforce referential integrity on a CSV.

Decomposition (lossless, dependency-preserving):

```
VEHICLE(vehicle_id, make, model, battery_kwh)
VEHICLE_STANDARD(vehicle_id, standard_id)   -- FK to CONNECTOR_STANDARD
```

This is exactly E24 in the final schema. The same argument applies to `STATION.amenities` -> STATION_AMENITY (E25). **Why not a separate row per standard with a sequence column?** Because the pair (vehicle, standard) is itself all-key — no ordering information exists — which also makes the result trivially **4NF** (the only MVDs are trivial).

12.2 Example 2 — 2NF violation: partial dependencies on a composite key

Naive draft of the metering table:

```
READING_FLAT(session_id, seq_no, session_started_at, driver_email, meter_kwh, power_kw)
PK (session_id, seq_no)
```

FDs: `{session_id, seq_no} -> {meter_kwh, power_kw}` (full-key — fine), but also `session_id -> {session_started_at, driver_email}` — **partial dependencies**: non-key attributes determined by *part* of the key. Consequences: the session start time is repeated once per meter tick (10–40 rows), update anomalies (correct it in one row, not the others), and deletion anomalies (delete the last reading of a session and you lose the session's start time).

Decomposition:

```
CHARGING_SESSION(session_id, ..., started_at, driver/user_id, ...)   -- session facts, keyed by
    session_id
METER_READING(session_id, seq_no, taken_at, meter_kwh, power_kw, ...) -- tick facts, keyed by
    (session, seq)
```

Lossless: R1 ∩ R2 = {session_id}, and session_id -> R1 (key of R1) — the classic split condition. Dependency-preserving: every FD is enforced inside one fragment. This is exactly the E11/E12 pair in the final schema. **2NF is about the composite key here — an important contrast with Example 3, where the single-attribute key makes 2NF automatic but 3NF is not.**

12.3 Example 3 — 3NF violation: transitive dependency in the "one big session table"

Naive draft (the classic "flat report table"):

```
SESSION_FLAT(session_id, connector_id, cp_id, station_name, driver_email, driver_name,
             tariff_name, price_per_kwh, energy_kwh, invoice_total)
```

FDs beyond the key:

```
session_id    -> connector_id, driver_email, tariff_plan_used, ...
connector_id  -> cp_id                    (a connector belongs to one charge point)
cp_id         -> station_name             (a charge point belongs to one station)
driver_email  -> driver_name              (a user's record determines their name)
tariff_plan_used -> tariff_name, price_per_kwh (plan record determines its facts)
session_id    -> invoice_total            (billing determines the total)
```

Transitive chains: `session_id -> connector_id -> cp_id -> station_name` and `session_id -> driver_email -> driver_name`. Every rename of a station or user would need to rewrite thousands of session rows — the update-anomaly argument, concretely.

3NF decomposition (each non-key attribute moves to the table keyed by its determinant):

```
CHARGING_SESSION(session_id, connector_id, user_id, tariff_plan_id, energy_kwh, ...)
CHARGE_POINT(cp_id, station_id, ocpp_identity, ...)
STATION(station_id, name, ...)
APP_USER(user_id, email, full_name, ...)         -- email UNIQUE
TARIFF_PLAN(plan_id, name, ...)
TARIFF_BAND(plan_id, day_scope, start_time, end_time, price_per_kwh)
INVOICE(invoice_id, session_id UNIQUE, total, ...)
```

Lossless by construction (each join returns to the original on keys). Note `session_id -> invoice_total` is *not* an anomaly once INVOICE is 1:1 with session via UNIQUE — the total lives with its own key.

12.4 Example 4 — BCNF violation: overlapping candidate keys (the best viva story)

Naive draft of connector numbering "per station" (which sounds reasonable — stations number their sockets 1..n):

```
CONNECTOR_SLOT(station_id, connector_no, cp_id)
-- business rules:
```

```
--   (station_id, connector_no) -> cp_id     (socket #2 of station S1 is on exactly one cabinet)
--   cp_id -> station_id                     (a cabinet sits at exactly one station)
```

Candidate keys: {(station_id, connector_no)} and {(cp_id, connector_no)}. Now check BCNF: FD `cp_id -> station_id` has determinant `cp_id`, which is **not a superkey** (it does not determine connector_no) → **BCNF violated** (3NF survives only because station_id is *prime* — it belongs to a candidate key; this R↔BCNF distinction is the sharpest thing a student can say about normal forms).

Update anomaly: a cabinet physically moved from station S1 to S2 forces rewriting every connector row of that cabinet. Redundancy: station_id is stored once per connector instead of once per cabinet.

BCNF decomposition: project out the violating FD:

```
CHARGE_POINT(cp_id PK, station_id NOT NULL)      -- cp_id -> station_id
CONNECTOR(cp_id, connector_no, standard_id, max_power_kw, status, ...)  -- PK (cp_id, connector_no)
```

Lossless check: R1 ∩ R2 = {cp_id}; cp_id -> R1's key → lossless join. Dependency-preservation check: FD1 `(station_id, connector_no) -> cp_id` is **no longer enforceable by a key in any fragment** — it is only implied by the join. This is the textbook BCNF cost, and we accept it *consciously*: the constraint "socket numbering is unique per station" is business decoration, not money-path correctness; the real integrity rules (connector belongs to exactly one cabinet; cabinet belongs to one station; (cp, no) unique) are all key-enforced. **The final schema in Part V is precisely this decomposition** — which is why CONNECTOR's PK is (cp_id, connector_no) and not (station_id, connector_no).

12.5 4NF and 5NF — brief honesty

After the decompositions above, no nontrivial multivalued dependencies remain (VEHICLE_STANDARD and STATION_AMENITY are all-key). Join dependencies across three or more relations (5NF) do not arise because no business rule decomposes a fact into three independent projections. We state 4NF/5NF as "checked, trivially satisfied" rather than claiming depth we do not have — interviewers respect the boundary.

12.6 Controlled denormalization (where we deliberately stop)

Denormalization	Justification	Guard
WALLET_ACCOUNT.balance (derived from ledger)	O(1) affordability check inside the payment transaction; recompute from ledger would scan history per debit	maintained only inside BILLING_PKG's ledger transaction; ledger is append-only; audit can recompute and reconcile
CHARGING_SESSION.energy_kwh (end - start)	frozen business fact at billing time (BR-10), not a live value	written once by BILLING_PKG when billing_state flips; CHECK end >= start
INVOICE.total (sum of lines)	invoice is a legal document; its total must not drift if lines were (hypothetically) amended	written at issue time by BILLING_PKG; lines are insert-only afterwards
STATION.available_connector_count	NOT stored — computed on read via view	demonstrates the restraint half of the rule

The principle: **store a derived value only at the instant its precondition is transactionally locked, and say who owns it.** Each row of this table names the owning package — that is the difference between denormalization and carelessness.

12.7 Normalization of the temporal dimension (tariff versioning)

Tariffs exhibit temporal FDs: `plan(group_id) -> price` holds *at a time*, not absolutely. The naive design (mutable price column) creates update anomalies across history — changing a price would retroactively alter old

invoices' meaning. The version-chain design (TARIFF_PLAN rows immutable, supersedes self-FK, sessions FK to the exact plan_id used) is **row versioning**: the "relation" is effectively `TARIFF(group_id, version_no, ...)` with a temporal key. This is 6NF-thinking (anchor + timeline) applied pragmatically without full 6NF machinery — worth one slide in the DA1 presentation.

12.8 Normal-form summary table

Relation	1NF	2NF	3NF	BCNF	Note
VEHICLE + VEHICLE_STANDARD	after de-CSV	-	-	yes	Example 1
CHARGING_SESSION + METER_READING	yes	Example 2	Example 3 chain	yes	weak-entity split
CONNECTOR + CHARGE_POINT	yes	yes	yes	via Example 4	overlapping-CK decomposition
TARIFF_PLAN + TARIFF_BAND	yes	yes	yes	yes (2 CKs, both superkeys)	temporal BR-08 non-FD
all remaining relations	yes	yes	yes	trivially	single-key, key -> attrs
WALLET_ACCOUNT / INVOICE.total / energy_kwh	yes	yes	yes	yes	controlled denormalization, owned by packages (12.6)

PART VII

Oracle Database Architecture and SQL Design (DA2 Core)

> Masterplan sections 13–14. Oracle runs locally in Docker (`gvenzl/oracle-free`, Oracle Database 23ai Free) so the entire team gets full PL/SQL, materialized views, and scripting with zero license friction. Everything here is executed by versioned migrations so DA2's demo is reproducible on any laptop.

---

13. Oracle Database Architecture

13.1 Container and connection profile

```
docker run -d --name volthub-oracle \
  -p 1521:1521 -e ORACLE_PWD=volthub_dev_pwd \
  -e APP_USER=volthub -e APP_USER_PASSWORD=volthub_dev_pwd \
  gvenzl/oracle-free:23-slim
```

The `gvenzl/oracle-free` image auto-creates the `volthub` user on startup; SQLcl (`sql volthub/volthub_dev_pwd@localhost/freepdb1`) is the scripting interface for migrations and seed runs. Free edition limits (12 GB user data, 2 CPU threads) are irrelevant at our data volumes and worth citing honestly in the report.

13.2 Schema and privilege layout

One application schema `VOLTHUB` owns all objects. An application role `VOLTHUB_APP_ROLE` receives only the grants the API needs:

Principal	Gets	Deliberately withheld
VOLTHUB (owner)	all objects, packages	-
VOLTHUB_APP_ROLE	SELECT on tables; INSERT/UPDATE on allowed columns; EXECUTE on packages	DELETE on all business tables (soft-state only); UPDATE/DELETE on WALLET_LEDGER, AUDIT_LOG, METER_READING (append-only); direct UPDATE on CONNECTOR.status, WALLET_ACCOUNT.balance, CHARGING_SESSION.state

The withheld grants are the point: **column-sensitive integrity lives in packages, and the database enforces that nothing else can reach it.** `AUDIT_PKG` runs with `AUTHID DEFINER` so triggers can write the audit trail even though the API role cannot.

13.3 Identity strategy

Surrogate keys use `GENERATED BY DEFAULT ON NULL AS IDENTITY` (Oracle 12c+), which is standard on 23ai. Under the hood each is a sequence — we expose this by querying `USER_SEQUENCES` in the report and note the viva-relevant facts: identity columns are sequence-backed, cache 20 by default, and gaps are normal (rollback does not return sequence values). The OUTBOX relay uses `event_id` ordering (monotonic with identity) as its cursor, with `created_at` as a tiebreak field in the payload.

13.4 Physical design notes (what a student project should actually claim)

All tables live in the default tablespace; we do **not** pretend to tune tablespaces. What we *do* address honestly: PCTFREE defaults are fine for our append-heavy tables; `TIMESTAMP(6)` everywhere (microseconds matter for meter ticks); `CLOB` only where JSON/JSON-check constraints are declared; numbers use explicit precision (`NUMBER(10,3)` for kWh) — a deliberate habit that looks professional and costs nothing.

---

14. SQL Design — complete DDL

14.1 Tables (abridged where identical patterns repeat; full script in `db/oracle/V001__core_schema.sql`)

```
-- ============ LOOKUPS ============
CREATE TABLE connector_state (
  state_code  VARCHAR2(12) PRIMARY KEY,
  description VARCHAR2(80) NOT NULL
);
INSERT INTO connector_state VALUES ('AVAILABLE','Ready to accept a vehicle');
INSERT INTO connector_state VALUES ('OCCUPIED','Vehicle connected / energy flowing');
INSERT INTO connector_state VALUES ('RESERVED','Held for an upcoming reservation');
INSERT INTO connector_state VALUES ('FAULTED','Reported fault, not bookable');
INSERT INTO connector_state VALUES ('OFFLINE','No OCPP heartbeat');
INSERT INTO connector_state VALUES ('UNAVAILABLE','Operator-disabled (OCPP ChangeAvailability)');

CREATE TABLE session_state (
  state_code  VARCHAR2(12) PRIMARY KEY,
  description VARCHAR2(80) NOT NULL,
  is_terminal NUMBER(1) NOT NULL CHECK (is_terminal IN (0,1))
```

```
);

CREATE TABLE reservation_state (
  state_code  VARCHAR2(12) PRIMARY KEY,
  description VARCHAR2(80) NOT NULL
);

-- ============ IDENTITY / USERS ============
CREATE TABLE app_user (
  user_id       NUMBER(9) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  email         VARCHAR2(255) NOT NULL,
  password_hash VARCHAR2(97)  NOT NULL,
  full_name     VARCHAR2(120) NOT NULL,
  phone         VARCHAR2(20),
  role          VARCHAR2(12)  NOT NULL
                CHECK (role IN ('DRIVER','OPERATOR','ADMIN')),
  status        VARCHAR2(10)  NOT NULL DEFAULT 'ACTIVE'
                CHECK (status IN ('ACTIVE','SUSPENDED')),
  created_at    TIMESTAMP(6)  NOT NULL DEFAULT SYSTIMESTAMP,
  CONSTRAINT uk_app_user_email UNIQUE (email)
);

CREATE TABLE wallet_account (
  user_id    NUMBER(9) PRIMARY KEY
             REFERENCES app_user(user_id),
  balance    NUMBER(12,2) NOT NULL CHECK (balance >= 0),
  currency   CHAR(3)      NOT NULL DEFAULT 'INR',
  updated_at TIMESTAMP(6) NOT NULL
);

CREATE TABLE wallet_ledger (
  user_id       NUMBER(9)      NOT NULL REFERENCES app_user(user_id),
  seq_no        NUMBER(9)      NOT NULL,
  kind          VARCHAR2(12)   NOT NULL
                CHECK (kind IN ('TOPUP','PAYMENT','REFUND','ADJUSTMENT')),
  amount        NUMBER(12,2)   NOT NULL,
  balance_after NUMBER(12,2)   NOT NULL CHECK (balance_after >= 0),
  payment_id    NUMBER(12),
  created_at    TIMESTAMP(6)   NOT NULL DEFAULT SYSTIMESTAMP,
  note          VARCHAR2(200),
  CONSTRAINT pk_wallet_ledger PRIMARY KEY (user_id, seq_no)
);

-- ============ STATIONS / HARDWARE ============
CREATE TABLE connector_standard (
  standard_id    NUMBER(4) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  code           VARCHAR2(16) NOT NULL UNIQUE,
  display_name   VARCHAR2(60) NOT NULL,
  max_typical_kw NUMBER(6,1)
);

CREATE TABLE station (
  station_id   NUMBER(9) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  name         VARCHAR2(120) NOT NULL,
  latitude     NUMBER(9,6) NOT NULL CHECK (latitude BETWEEN 6 AND 37),
  longitude    NUMBER(9,6) NOT NULL CHECK (longitude BETWEEN 68 AND 98),
  address_line VARCHAR2(200) NOT NULL,
  city         VARCHAR2(80) NOT NULL,
  state        VARCHAR2(80) NOT NULL,
  pincode      VARCHAR2(10),
  status       VARCHAR2(12) NOT NULL DEFAULT 'ACTIVE'
               CHECK (status IN ('ACTIVE','INACTIVE')),
  operator_id  NUMBER(9) NOT NULL REFERENCES app_user(user_id),
  created_at   TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP
);

CREATE TABLE station_amenity (
  station_id NUMBER(9) NOT NULL REFERENCES station(station_id),
  amenity    VARCHAR2(12) NOT NULL
             CHECK (amenity IN ('CAFE','RESTROOM','WIFI','PARKING','SHOP','CANOPY','CCTV')),
  CONSTRAINT pk_station_amenity PRIMARY KEY (station_id, amenity)
);

CREATE TABLE charge_point (
  cp_id            NUMBER(9) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
```

```
  station_id       NUMBER(9) NOT NULL REFERENCES station(station_id),
  ocpp_identity    VARCHAR2(60) NOT NULL UNIQUE,
  vendor           VARCHAR2(60),
  model            VARCHAR2(60),
  firmware_version VARCHAR2(40),
  status           VARCHAR2(10) NOT NULL DEFAULT 'OFFLINE'
                   CHECK (status IN ('ONLINE','OFFLINE','FAULTED')),
  last_boot_at     TIMESTAMP(6),
  created_at       TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP
);

CREATE TABLE connector (
  cp_id                NUMBER(9) NOT NULL REFERENCES charge_point(cp_id),
  connector_no         NUMBER(2) NOT NULL CHECK (connector_no BETWEEN 1 AND 8),
  standard_id          NUMBER(4) NOT NULL REFERENCES connector_standard(standard_id),
  max_power_kw         NUMBER(6,2) NOT NULL CHECK (max_power_kw BETWEEN 3 AND 400),
  status               VARCHAR2(12) NOT NULL DEFAULT 'OFFLINE'
                       REFERENCES connector_state(state_code),
  last_state_change_at TIMESTAMP(6),
  created_at           TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP,
  CONSTRAINT pk_connector PRIMARY KEY (cp_id, connector_no)
);

-- ============ TARIFFS ============
CREATE TABLE tariff_plan (
  plan_id            NUMBER(9) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  group_id           NUMBER(9) NOT NULL,
  version_no         NUMBER(3) NOT NULL,
  name               VARCHAR2(80) NOT NULL,
  currency           CHAR(3) NOT NULL DEFAULT 'INR',
  session_fee        NUMBER(8,2) CHECK (session_fee >= 0),
  idle_fee_per_30min NUMBER(8,2) CHECK (idle_fee_per_30min >= 0),
  active_from        TIMESTAMP(6) NOT NULL,
  active_to          TIMESTAMP(6),
  supersedes_plan_id NUMBER(9) REFERENCES tariff_plan(plan_id),
  created_by         NUMBER(9) NOT NULL REFERENCES app_user(user_id),
  CONSTRAINT uk_tariff_version UNIQUE (group_id, version_no)
);

CREATE TABLE tariff_band (
  band_id       NUMBER(9) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  plan_id       NUMBER(9) NOT NULL REFERENCES tariff_plan(plan_id),
  day_scope     VARCHAR2(8) NOT NULL CHECK (day_scope IN ('ALL','WEEKDAY','WEEKEND')),
  start_time    TIMESTAMP(0) NOT NULL,
  end_time      TIMESTAMP(0) NOT NULL,
  price_per_kwh NUMBER(8,4) NOT NULL CHECK (price_per_kwh > 0),
  CONSTRAINT uk_band UNIQUE (plan_id, day_scope, start_time),
  CONSTRAINT ck_band_order CHECK (end_time > start_time)
);

-- ============ RESERVATIONS / SESSIONS ============
CREATE TABLE reservation (
  reservation_id NUMBER(9) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  connector_ref  VARCHAR2(72) NOT NULL,          -- 'cp_id:connector_no' encoded FK (Part V, 10.3)
  user_id        NUMBER(9) NOT NULL REFERENCES app_user(user_id),
  vehicle_id     NUMBER(9) NOT NULL REFERENCES vehicle(vehicle_id),
  start_at       TIMESTAMP(6) NOT NULL,
  end_at         TIMESTAMP(6) NOT NULL,
  status         VARCHAR2(12) NOT NULL
                 REFERENCES reservation_state(state_code),
  created_at     TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP,
  cancelled_at   TIMESTAMP(6),
  CONSTRAINT ck_res_window CHECK (end_at > start_at)
);

CREATE TABLE charging_session (
  session_id      NUMBER(12) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  connector_ref   VARCHAR2(72) NOT NULL,
  user_id         NUMBER(9) NOT NULL REFERENCES app_user(user_id),
  vehicle_id      NUMBER(9) REFERENCES vehicle(vehicle_id),
  reservation_id  NUMBER(9) UNIQUE REFERENCES reservation(reservation_id),
  tariff_plan_id  NUMBER(9) NOT NULL REFERENCES tariff_plan(plan_id),
  id_tag          VARCHAR2(40) NOT NULL,
  state           VARCHAR2(12) NOT NULL
                  REFERENCES session_state(state_code),
```

```
  billing_state   VARCHAR2(10) NOT NULL DEFAULT 'UNBILLED'
                  CHECK (billing_state IN ('UNBILLED','BILLED','FAILED','WAIVED')),
  started_at      TIMESTAMP(6),
  ended_at        TIMESTAMP(6),
  start_meter_kwh NUMBER(10,3) NOT NULL CHECK (start_meter_kwh >= 0),
  end_meter_kwh   NUMBER(10,3),
  energy_kwh      NUMBER(10,3),
  stop_reason     VARCHAR2(40),
  created_at      TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP,
  CONSTRAINT ck_session_meter CHECK (end_meter_kwh IS NULL OR end_meter_kwh >= start_meter_kwh)
);

CREATE TABLE meter_reading (
  session_id  NUMBER(12) NOT NULL REFERENCES charging_session(session_id),
  seq_no      NUMBER(9)  NOT NULL,
  taken_at    TIMESTAMP(6) NOT NULL,
  meter_kwh   NUMBER(10,3) NOT NULL CHECK (meter_kwh >= 0),
  power_kw    NUMBER(8,3),
  voltage_v   NUMBER(8,1),
  current_a   NUMBER(8,2),
  source      VARCHAR2(10) NOT NULL DEFAULT 'OCPP' CHECK (source IN ('OCPP','SYNTHETIC')),
  ingested_at TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP,
  CONSTRAINT pk_meter_reading PRIMARY KEY (session_id, seq_no)
);

-- ============ BILLING ============
CREATE TABLE invoice (
  invoice_id     NUMBER(12) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  session_id     NUMBER(12) NOT NULL UNIQUE REFERENCES charging_session(session_id),
  tariff_plan_id NUMBER(9) NOT NULL REFERENCES tariff_plan(plan_id),
  issued_at      TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP,
  status         VARCHAR2(8) NOT NULL CHECK (status IN ('DUE','PAID','FAILED','VOID')),
  total          NUMBER(12,2) NOT NULL CHECK (total >= 0),
  currency       CHAR(3) NOT NULL
);

CREATE TABLE invoice_line (
  invoice_id  NUMBER(12) NOT NULL REFERENCES invoice(invoice_id),
  line_no     NUMBER(3)  NOT NULL,
  kind        VARCHAR2(14) NOT NULL
              CHECK (kind IN ('ENERGY','SESSION_FEE','IDLE_FEE','TAX','ADJUSTMENT')),
  description VARCHAR2(120) NOT NULL,
  quantity    NUMBER(12,4) NOT NULL,
  unit        VARCHAR2(12),
  amount      NUMBER(12,2) NOT NULL CHECK (amount >= 0),
  CONSTRAINT pk_invoice_line PRIMARY KEY (invoice_id, line_no)
);

CREATE TABLE payment (
  payment_id   NUMBER(12) GENERATED BY DEFAULT ON NULL AS IDENTITY PRIMARY KEY,
  invoice_id   NUMBER(12) NOT NULL REFERENCES invoice(invoice_id),
  amount       NUMBER(12,2) NOT NULL CHECK (amount > 0),
  method       VARCHAR2(10) NOT NULL DEFAULT 'WALLET' CHECK (method IN ('WALLET')),
  status       VARCHAR2(8) NOT NULL CHECK (status IN ('SUCCESS','FAILED')),
  attempted_at TIMESTAMP(6) NOT NULL DEFAULT SYSTIMESTAMP,
  reference    VARCHAR2(60)
);

-- ============ OPERATIONS / PLATFORM ============
-- fault, maintenance_record, review, notification, audit_log, outbox_event:
-- identical to the Part V specification; see V001 migration for the full text.

CREATE INDEX ix_session_user_time  ON charging_session (user_id, started_at DESC);
CREATE INDEX ix_session_conn_state ON charging_session (connector_ref, state);
CREATE INDEX ix_session_started    ON charging_session (started_at);
CREATE INDEX ix_res_conn_start     ON reservation (connector_ref, start_at);
CREATE INDEX ix_res_user_status    ON reservation (user_id, status);
CREATE INDEX ix_reading_taken      ON meter_reading (taken_at);
CREATE INDEX ix_fault_open         ON fault (connector_ref, cleared_at);
CREATE INDEX ix_outbox_cursor      ON outbox_event (processed_at, created_at);
CREATE INDEX ix_invoice_status     ON invoice (status, issued_at);
```

14.2 Views

```
-- Live discovery view: one row per connector with station context
CREATE OR REPLACE VIEW v_connector_live AS
SELECT  s.station_id, s.name AS station_name, s.city, s.latitude, s.longitude,
        s.status AS station_status, cp.cp_id, cp.ocpp_identity, cp.status AS cp_status,
        c.connector_no, c.max_power_kw, cs.code AS standard_code,
        c.status AS connector_status, c.last_state_change_at
FROM    station s
JOIN    charge_point cp ON cp.station_id = s.station_id
JOIN    connector c     ON c.cp_id = cp.cp_id
JOIN    connector_standard cs ON cs.standard_id = c.standard_id;

-- Station availability rollup (derived attribute computed on read - Part VI, 12.6)
CREATE OR REPLACE VIEW v_station_summary AS
SELECT  s.station_id, s.name,
        COUNT(*)                                        AS connector_count,
        SUM(CASE WHEN c.status = 'AVAILABLE' THEN 1 ELSE 0 END) AS available_count,
        ROUND(AVG(r.rating), 2)                          AS avg_rating,
        COUNT(DISTINCT r.session_id)                     AS total_sessions
FROM    station s
JOIN    charge_point cp ON cp.station_id = s.station_id
JOIN    connector c     ON c.cp_id = cp.cp_id
LEFT JOIN review r      ON r.session_id IN
          (SELECT session_id FROM charging_session WHERE connector_ref =
             TO_CHAR(c.cp_id) || ':' || TO_CHAR(c.connector_no))
GROUP BY s.station_id, s.name;
```

14.3 Materialized views (DA2's analytical objects — justified, not decorative)

```
-- Daily revenue and energy per station: the operator dashboard's backbone.
-- FAST refresh on commit is impossible (contains join + aggregate across 3 tables),
-- so we refresh on demand via scheduler every 15 min and document why.
CREATE MATERIALIZED VIEW mv_station_daily
BUILD IMMEDIATE REFRESH COMPLETE ON DEMAND
AS
SELECT  s.station_id, s.name,
        TRUNC(cs.started_at)                    AS day,
        COUNT(*)                                AS sessions,
        SUM(cs.energy_kwh)                      AS energy_kwh,
        SUM(i.total)                            AS revenue
FROM    charging_session cs
JOIN    station s  ON s.station_id = TO_NUMBER(SUBSTR(cs.connector_ref, 1,
                      INSTR(cs.connector_ref, ':') - 1))
JOIN    invoice i  ON i.session_id = cs.session_id AND i.status IN ('PAID','DUE')
WHERE   cs.state = 'COMPLETED'
GROUP BY s.station_id, s.name, TRUNC(cs.started_at);

CREATE INDEX ix_mvsd ON mv_station_daily (station_id, day);

-- 15-minute refresh job (DBMS_SCHEDULER) - the "background job" DA2 can demo
BEGIN
  DBMS_SCHEDULER.CREATE_JOB(
    job_name        => 'refresh_mv_station_daily',
    job_type        => 'PLSQL_BLOCK',
    job_action      => 'BEGIN DBMS_MVIEW.REFRESH(''MV_STATION_DAILY'', ''C''); END;',
    repeat_interval => 'FREQ=MINUTELY;INTERVAL=15',
    enabled         => TRUE);
END;
/
```

Why these two and no more: `mv_station_daily` answers the dashboard's most expensive aggregate (revenue/energy trend) without hammering OLTP tables; a second MV for utilization is deliberately *deferred to DA3*, where TimescaleDB continuous aggregates replace it with incremental maintenance — a before/after story the comparative analysis (Section 28) exploits.

14.4 Sequences and identity — what to show the examiner

```
SELECT sequence_name, last_number, cache_size
FROM   user_sequences;                      -- identities appear as ISEQ$$_<obj#>
SELECT column_name, identity_column, data_default
```

```
FROM   user_tab_columns
WHERE  table_name = 'APP_USER';             -- YES on identity_column
```

PART VIII

PL/SQL Design

> Masterplan section 15. The packages below are the project's proof that business correctness lives in the database. Every procedure that changes state is here; the API layer can only call them. Code shown is the real reference implementation pattern used in `db/oracle/packages/` (viva value ranked in 15.6).

15.1 Package inventory

Package	Responsibility	Viva value
`RESERVATION_PKG`	create/cancel/expire reservations; the double-booking defense	highest
`CHARGE_SESSION_PKG`	session state machine, start/stop, meter tick ingestion	highest
`BILLING_PKG`	tariff resolution, invoice + lines, wallet debit with ledger	highest
`TARIFF_PKG`	plan versioning, band validation, price lookup at time	high
`MAINTENANCE_PKG`	fault triage, maintenance records, connector restore	medium
`AUDIT_PKG`	uniform audit writer used by triggers and packages	medium
`SEED_PKG`	deterministic sample data generation (Section 16)	low (utility)

15.2 RESERVATION_PKG — the race-condition defense

```
CREATE OR REPLACE PACKAGE BODY reservation_pkg AS

  PROCEDURE create_reservation(
    p_user_id      IN  NUMBER,
    p_vehicle_id   IN  NUMBER,
    p_cp_id        IN  NUMBER,
    p_connector_no IN  NUMBER,
    p_start_at     IN  TIMESTAMP,
    p_end_at       IN  TIMESTAMP,
    p_reservation_id OUT NUMBER)
  IS
    v_ref        VARCHAR2(72) := TO_CHAR(p_cp_id) || ':' || TO_CHAR(p_connector_no);
    v_status     VARCHAR2(12);
    v_overlaps   NUMBER;
  BEGIN
    -- BR-04: window sanity (15..120 minutes, future start)
    IF p_end_at <= p_start_at
       OR p_end_at - p_start_at > INTERVAL '120' MINUTE
       OR p_end_at - p_start_at < INTERVAL '15' MINUTE
       OR p_start_at < SYSTIMESTAMP - INTERVAL '1' MINUTE THEN
      raise_application_error(-20501, 'Reservation window must be 15-120 minutes and start in the
    future');
    END IF;

    -- Serialize all reservation writers for THIS connector.
    -- The FOR UPDATE is the whole trick: two concurrent requests queue here;
```

```
    -- the second sees the first's uncommitted row via the lock, then the
    -- overlap check below fails it. No extra app-level locking needed.
    SELECT status INTO v_status
    FROM   connector
    WHERE  cp_id = p_cp_id AND connector_no = p_connector_no
    FOR UPDATE;

    -- BR-12: faulted/offline connectors are not bookable
    IF v_status NOT IN ('AVAILABLE', 'RESERVED') THEN
      raise_application_error(-20502, 'Connector is not bookable (status ' || v_status || ')');
    END IF;

    -- BR-05: no overlap with BOOKED/CONVERTED reservations
    SELECT COUNT(*) INTO v_overlaps
    FROM   reservation r
    WHERE  r.connector_ref = v_ref
      AND  r.status IN ('BOOKED','CONVERTED')
      AND  r.start_at < p_end_at
      AND  r.end_at   > p_start_at;
    IF v_overlaps > 0 THEN
      raise_application_error(-20503, 'Connector already reserved for the requested window');
    END IF;

    INSERT INTO reservation (reservation_id, connector_ref, user_id, vehicle_id,
                             start_at, end_at, status)
    VALUES (NULL, v_ref, p_user_id, p_vehicle_id, p_start_at, p_end_at, 'BOOKED')
    RETURNING reservation_id INTO p_reservation_id;

    audit_pkg.log('RESERVATION', p_reservation_id, 'CREATE', NULL,
                  JSON_OBJECT('user' VALUE p_user_id,
                              'window' VALUE TO_CHAR(p_start_at) || ' .. ' || TO_CHAR(p_end_at)));
  END create_reservation;

  PROCEDURE expire_stale(p_out_rows OUT NUMBER) IS
    -- cursor + bulk collect: the educationally-required explicit cursor (15.5)
    CURSOR c_stale IS
      SELECT r.reservation_id
      FROM   reservation r
      WHERE  r.status = 'BOOKED' AND r.end_at < SYSTIMESTAMP
      FOR UPDATE SKIP LOCKED;
    TYPE t_ids IS TABLE OF NUMBER;
    l_ids t_ids;
  BEGIN
    OPEN c_stale;
    FETCH c_stale BULK COLLECT INTO l_ids LIMIT 500;
    CLOSE c_stale;
    FORALL i IN 1 .. l_ids.COUNT
      UPDATE reservation SET status = 'EXPIRED' WHERE reservation_id = l_ids(i);
    p_out_rows := SQL%ROWCOUNT;
    COMMIT;
  END expire_stale;

END reservation_pkg;
/
```

Why SELECT FOR UPDATE and not just the overlap query? Without the lock, two transactions can both run the overlap check at time T (both read zero rows), both pass, and both insert — the classic TOCTOU race. With `FOR UPDATE` on the connector row, writers serialize per connector; the second transaction's check runs *after* the first commits and correctly sees the conflict. Section 31 proves it with a two-thread test. (Postgres/TimescaleDB would additionally allow a true exclusion constraint — noted in the DA3 comparison as a legitimate engine difference.)

15.3 CHARGE_SESSION_PKG — the state machine authority

```
CREATE OR REPLACE PACKAGE BODY charge_session_pkg AS

  TYPE t_graph IS TABLE OF VARCHAR2(1) INDEX BY VARCHAR2(24);
  g_ok t_graph;   -- legal-transition matrix, loaded once

  PROCEDURE load_graph IS
  BEGIN
    -- from          -> to          (rows: from||'->'||to)
    g_ok('RESERVED->PREPARING') := 'Y';  g_ok('PREPARING->CHARGING')  := 'Y';
    g_ok('CHARGING->SUSPENDED') := 'Y';  g_ok('SUSPENDED->CHARGING')  := 'Y';
    g_ok('CHARGING->COMPLETED') := 'Y';  g_ok('SUSPENDED->COMPLETED') := 'Y';
    g_ok('PREPARING->FAILED')   := 'Y';  g_ok('CHARGING->FAILED')     := 'Y';
    g_ok('SUSPENDED->FAILED')   := 'Y';  g_ok('RESERVED->CANCELLED')  := 'Y';
    g_ok('PREPARING->CANCELLED'):= 'Y';
  END load_graph;

  PROCEDURE transition(
    p_session_id IN NUMBER,
    p_to         IN VARCHAR2,
    p_stop_reason IN VARCHAR2 DEFAULT NULL)
  IS
    v_from  VARCHAR2(12);
    v_meter NUMBER(10,3);
  BEGIN
    IF NOT g_ok.EXISTS(p_from => NULL) THEN NULL; END IF;  -- (graph lazy-loaded at init)

    SELECT state INTO v_from FROM charging_session
    WHERE  session_id = p_session_id FOR UPDATE;          -- lock the session row

    IF g_ok(v_from || '->' || p_to) IS NULL THEN
      raise_application_error(-20601,
        'Illegal transition ' || v_from || ' -> ' || p_to);
    END IF;

    UPDATE charging_session
    SET   state = p_to,
          ended_at = CASE WHEN p_to IN ('COMPLETED','FAILED','CANCELLED')
                          THEN SYSTIMESTAMP ELSE ended_at END,
          end_meter_kwh = CASE WHEN p_to = 'COMPLETED' THEN v_meter ELSE end_meter_kwh END,
          stop_reason   = COALESCE(p_stop_reason, stop_reason)
    WHERE  session_id = p_session_id;

    -- mirror connector state for non-terminal outcomes
    IF p_to = 'CHARGING' THEN
      UPDATE connector SET status = 'OCCUPIED', last_state_change_at = SYSTIMESTAMP
      WHERE  connector_ref = (SELECT connector_ref FROM charging_session
                              WHERE session_id = p_session_id);
    ELSIF p_to IN ('COMPLETED','FAILED','CANCELLED') THEN
      UPDATE connector SET status = 'AVAILABLE', last_state_change_at = SYSTIMESTAMP
      WHERE  connector_ref = (SELECT connector_ref FROM charging_session
                              WHERE session_id = p_session_id);
    END IF;

    audit_pkg.log('CHARGING_SESSION', p_session_id, 'TRANSITION', v_from, p_to);
  END transition;

  PROCEDURE record_meter_tick(
    p_session_id IN NUMBER, p_seq IN NUMBER, p_taken_at IN TIMESTAMP,
    p_meter_kwh IN NUMBER, p_kw IN NUMBER, p_volt IN NUMBER, p_amp IN NUMBER)
  IS
    v_last NUMBER(10,3);
  BEGIN
    -- BR-11: monotonic meter within tolerance
    SELECT MAX(meter_kwh) INTO v_last FROM meter_reading WHERE session_id = p_session_id;
    IF v_last IS NOT NULL AND p_meter_kwh < v_last - 0.001 THEN
      raise_application_error(-20602, 'Non-monotonic meter value');
    END IF;

    INSERT INTO meter_reading (session_id, seq_no, taken_at, meter_kwh,
                               power_kw, voltage_v, current_a)
    VALUES (p_session_id, p_seq, p_taken_at, p_meter_kwh, p_kw, p_volt, p_amp);
    INSERT INTO outbox_event (event_id, kind, payload)
```

```
    VALUES (NULL, 'METER_TICK',
            JSON_OBJECT('sessionId' VALUE p_session_id, 'seq' VALUE p_seq,
                        'ts' VALUE TO_CHAR(p_taken_at, 'YYYY-MM-DD"T"HH24:MI:SS.FF3'),
                        'kwh' VALUE p_meter_kwh, 'kw' VALUE p_kw,
                        'v' VALUE p_volt, 'a' VALUE p_amp));
  END record_meter_tick;

END charge_session_pkg;
/
```

15.4 BILLING_PKG — tariff resolution and the wallet

```
CREATE OR REPLACE PACKAGE BODY billing_pkg AS

  FUNCTION resolve_band_price(p_plan_id IN NUMBER, p_at IN TIMESTAMP)
    RETURN NUMBER IS
    v_price NUMBER(8,4);
  BEGIN
    -- TOU lookup: WEEKDAY/WEEKEND bands with ALL fallback; overlap-free by BR-08
    SELECT price_per_kwh INTO v_price
    FROM   tariff_band
    WHERE  plan_id = p_plan_id
      AND  (day_scope = 'ALL'
            OR (day_scope = 'WEEKDAY' AND TO_CHAR(p_at,'DY','NLS_DATE_LANGUAGE=ENGLISH')
                NOT IN ('SAT','SUN'))
            OR (day_scope = 'WEEKEND' AND TO_CHAR(p_at,'DY','NLS_DATE_LANGUAGE=ENGLISH')
                IN ('SAT','SUN')))
      AND  CAST(TO_CHAR(p_at, 'YYYY-MM-DD') AS TIMESTAMP)
           + NUMTODSINTERVAL(TO_CHAR(p_at,'HH24:MI'), 'MINUTE') >= start_time
      AND  CAST(TO_CHAR(p_at, 'YYYY-MM-DD') AS TIMESTAMP)
           + NUMTODSINTERVAL(TO_CHAR(p_at,'HH24:MI'), 'MINUTE') <  end_time
    FETCH FIRST 1 ROW ONLY;
    RETURN v_price;
  EXCEPTION WHEN NO_DATA_FOUND THEN
    raise_application_error(-20701, 'No tariff band covers the session time');
  END resolve_band_price;

  PROCEDURE bill_session(p_session_id IN NUMBER, p_invoice_id OUT NUMBER) IS
    v_sess      charging_session%ROWTYPE;
    v_plan      tariff_plan%ROWTYPE;
    v_price     NUMBER(8,4);
    v_energy    NUMBER(10,3);
    v_energy_amt NUMBER(12,2);
    v_total     NUMBER(12,2);
  BEGIN
    SELECT * INTO v_sess FROM charging_session WHERE session_id = p_session_id FOR UPDATE;

    IF v_sess.state <> 'COMPLETED' THEN
      raise_application_error(-20702, 'Only COMPLETED sessions can be billed');   -- BR-10
    END IF;
    IF v_sess.billing_state <> 'UNBILLED' THEN
      raise_application_error(-20703, 'Session already billed');                  -- BR-10
    END IF;

    SELECT * INTO v_plan FROM tariff_plan WHERE plan_id = v_sess.tariff_plan_id;
    v_energy     := v_sess.end_meter_kwh - v_sess.start_meter_kwh;
    v_price      := resolve_band_price(v_sess.tariff_plan_id, v_sess.started_at);
    v_energy_amt := ROUND(v_energy * v_price, 2);
    v_total      := v_energy_amt + NVL(v_plan.session_fee, 0);

    INSERT INTO invoice (invoice_id, session_id, tariff_plan_id, status, total, currency)
    VALUES (NULL, p_session_id, v_plan.plan_id, 'DUE', v_total, v_plan.currency)
    RETURNING invoice_id INTO p_invoice_id;

    INSERT INTO invoice_line VALUES
      (p_invoice_id, 1, 'ENERGY', 'Energy charge @ ' || TO_CHAR(v_price) || '/kWh',
       v_energy, 'kWh', v_energy_amt);
    IF v_plan.session_fee IS NOT NULL THEN
      INSERT INTO invoice_line VALUES
        (p_invoice_id, 2, 'SESSION_FEE', 'Fixed session fee', 1, 'session',
         v_plan.session_fee);
    END IF;
    UPDATE charging_session
```

```
    SET    billing_state = 'BILLED', energy_kwh = v_energy
    WHERE  session_id = p_session_id;      -- derived values frozen here (Part VI, 12.6)

    audit_pkg.log('INVOICE', p_invoice_id, 'CREATE', NULL, JSON_OBJECT('total' VALUE v_total));
  END bill_session;

  PROCEDURE pay_invoice(p_invoice_id IN NUMBER, p_out_payment_id OUT NUMBER) IS
    v_inv   invoice%ROWTYPE;
    v_bal   NUMBER(12,2);
    v_seq   NUMBER;
  BEGIN
    SELECT * INTO v_inv FROM invoice WHERE invoice_id = p_invoice_id FOR UPDATE;

    IF v_inv.status <> 'DUE' THEN
      raise_application_error(-20704, 'Invoice is not payable (status ' || v_inv.status || ')');
    END IF;

    -- wallet owner = session driver
    SELECT w.balance INTO v_bal
    FROM   wallet_account w
    JOIN   charging_session cs ON cs.user_id = w.user_id
    WHERE  cs.session_id = (SELECT session_id FROM invoice WHERE invoice_id = p_invoice_id)
    FOR UPDATE;                                          -- serialize debits per wallet

    IF v_bal < v_inv.total THEN
      INSERT INTO payment VALUES (NULL, p_invoice_id, v_inv.total, 'WALLET', 'FAILED',
                                  SYSTIMESTAMP, 'INSUFFICIENT_FUNDS')
      RETURNING payment_id INTO p_out_payment_id;
      UPDATE invoice SET status = 'FAILED' WHERE invoice_id = p_invoice_id;
      raise_application_error(-20705, 'Insufficient wallet balance');  -- client decides retry
    END IF;

    INSERT INTO payment VALUES (NULL, p_invoice_id, v_inv.total, 'WALLET', 'SUCCESS',
                                SYSTIMESTAMP, 'WALLET-' || p_invoice_id)
    RETURNING payment_id INTO p_out_payment_id;

    SELECT NVL(MAX(seq_no),0) + 1 INTO v_seq FROM wallet_ledger
    WHERE  user_id = (SELECT user_id FROM charging_session
                      WHERE session_id = (SELECT session_id FROM invoice
                                          WHERE invoice_id = p_invoice_id));

    INSERT INTO wallet_ledger (user_id, seq_no, kind, amount, balance_after, payment_id)
    SELECT cs.user_id, v_seq, 'PAYMENT', -v_inv.total, v_bal - v_inv.total, p_out_payment_id
    FROM   charging_session cs
    WHERE  cs.session_id = (SELECT session_id FROM invoice WHERE invoice_id = p_invoice_id);

    UPDATE wallet_account SET balance = v_bal - v_inv.total, updated_at = SYSTIMESTAMP
    WHERE  user_id = (SELECT user_id FROM charging_session
                      WHERE session_id = (SELECT session_id FROM invoice
                                          WHERE invoice_id = p_invoice_id));

    UPDATE invoice SET status = 'PAID' WHERE invoice_id = p_invoice_id;
  END pay_invoice;

END billing_pkg;
/
```

15.5 Triggers, cursors, functions — where each belongs

Triggers (only two, both justified):

```
-- 1) AUDIT: every UPDATE on charging_session writes the delta (NFR-05)
CREATE OR REPLACE TRIGGER trg_session_audit
AFTER UPDATE ON charging_session
FOR EACH ROW
BEGIN
  audit_pkg.log('CHARGING_SESSION', :NEW.session_id, 'UPDATE',
    JSON_OBJECT('state' VALUE :OLD.state, 'billing' VALUE :OLD.billing_state,
                'endMeter' VALUE :OLD.end_meter_kwh),
    JSON_OBJECT('state' VALUE :NEW.state, 'billing' VALUE :NEW.billing_state,
                'endMeter' VALUE :NEW.end_meter_kwh));
END;
/
```

```
-- 2) GUARD: no direct status writes to connector outside the state machine paths
CREATE OR REPLACE TRIGGER trg_connector_guard
BEFORE UPDATE OF status ON connector
FOR EACH ROW
DECLARE
  v_actor VARCHAR2(64);
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  SELECT SYS_CONTEXT('USERENV','CLIENT_IDENTIFIER') INTO v_actor FROM DUAL;
  -- the gateway sets CLIENT_IDENTIFIER='ocpp-gw'; API sets 'api:<userId>'
  IF v_actor IS NULL OR v_actor NOT LIKE 'ocpp-gw%'
     AND :NEW.status NOT IN ('AVAILABLE','OCCUPIED','RESERVED','FAULTED') THEN
    raise_application_error(-20801, 'Connector status changes must go through the gateway');
  END IF;
  COMMIT;
END;
/
```

Standalone function (demonstrates RETURN + pure SQL): `FN_STATION_UPTIME(p_station_id, p_from, p_to) RETURN NUMBER` — percentage of the window the station's connectors were not FAULTED/OFFLINE, computed from `last_state_change_at` history; used by the operator dashboard.

Explicit cursor education: `RESERVATION_PKG.expire_stale` (15.2) uses `CURSOR ... FOR UPDATE SKIP LOCKED` with `BULK COLLECT ... LIMIT 500` and `FORALL` — deliberately demonstrating the explicit-cursor + bulk-processing toolkit the syllabus loves, in a place where it is genuinely the right tool (a sweeper that must not block live bookings).

Exception handling pattern: every package maps business failures to numbered `raise_application_error` codes (-205xx reservations, -206xx sessions, -207xx billing, -208xx guards). The NestJS layer maps these codes to typed API errors via `oracledb` error numbers — one consistent contract across the stack.

15.6 Viva-value ranking (what to rehearse hardest)

1. `RESERVATION_PKG.create_reservation` — locking + race reasoning (the examiners' favorite topic).
2. `BILLING_PKG.pay_invoice` — `FOR UPDATE` on wallet, ledger insert with `balance_after`, atomic status flip.
3. `CHARGE_SESSION_PKG.transition` — data-driven state machine + connector mirroring.
4. `resolve_band_price` — temporal lookup with edge cases (no band covering time → explicit error).
5. `expire_stale` — explicit cursor, `SKIP LOCKED`, bulk processing.
6. Both triggers — audit trail and the "column ownership" argument tied to grants (13.2).

PART IX

Sample Dataset Strategy and SQL Query Portfolio

> Masterplan sections 16–17. The dataset must make queries *interesting* (diurnal peaks, faults, abandoned reservations, price changes) and the query portfolio must read like evidence the schema was designed for real questions.

---

16. Sample Dataset Strategy

16.1 Principles

1. **Deterministic:** every generator takes a seed; the same seed reproduces the exact database on any laptop (NFR-08). Randomness lives in the generator, never in foreign-key integrity.
2. **Coherent:** meter ticks are consistent with session boundaries; invoices sum from tariff bands; reservation overlaps never exist; connector status history matches sessions. The generator calls the *same packages* the API uses — seed data is valid because it was born through the valid paths.
3. **Story-shaped:** the data encodes the demo narrative — an evening peak (18:00–21:00), a faulted CCS2 connector at a highway station, a price change mid-month (tariff v1 → v2), a no-show reservation, one wallet with insufficient funds.

16.2 Volumes (sized for meaningful analytics + snappy demo)

Table	Rows	Rationale
APP_USER	400 (drivers 360, operators 30, admins 10)	realistic operator assignment
VEHICLE (+support)	520 (+1,100)	1.3 vehicles/driver
STATION / AMENITIES	24 (+120)	Chennai + OMR corridor spread
CHARGE_POINT / CONNECTOR	48 / 96	2 cabinets, 2 connectors avg
TARIFF_PLAN / BAND	9 versions / 30 bands	mid-month price change story
RESERVATION	1,800	incl. 60 no-shows, 40 cancellations
CHARGING_SESSION	10,000 over 90 days	110/day — enough for trends
METER_READING	~600,000	60 ticks/session avg (5s sim ticks coalesced to 60s)
INVOICE / LINES / PAYMENT	9,400 / 26,000 / 9,900	94% completion, includes failures
FAULT / MAINTENANCE	260 / 180	recurring fault at one connector
REVIEW / NOTIFICATION	3,400 / 5,000	ratings spread 1–5
AUDIT_LOG / OUTBOX	~800k / ~600k	full trail, unprocessed tail for DA3 demo

Generation is a Node script (`scripts/seed/generate.ts`) that drives the packages via `node-oracledb` in controlled batches (fast), plus `SEED_PKG` PL/SQL for anything needing in-database coherence.

16.3 The diurnal model

Session starts follow a weighted hour-of-day curve (workplace morning bump 08–10, evening peak 18–21 at 2.5x base), weekends shifted later; session duration ~ Weibull(45 min, 1.6) clipped at 8h; energy = min(battery deficit, charger power x duration). This produces the utilization and peak-load shapes that make Section 17's analytical queries and DA3's load curves visibly "real".

---

17. SQL Query Portfolio

Queries Q1–Q26, graded and each with its real-world purpose. All run on the Part V schema.

Tier 1 — beginner (correctness of basics)

Q1 — Available connectors near a point (station discovery first page)

```
SELECT v.station_id, v.station_name, v.city, v.standard_code, v.max_power_kw,
       v.connector_status
FROM   v_connector_live v
WHERE  v.connector_status = 'AVAILABLE'
  AND  v.station_status = 'ACTIVE'
  AND  v.latitude BETWEEN 12.80 AND 13.05          -- bounding box around VIT Chennai
  AND  v.longitude BETWEEN 80.10 AND 80.35
ORDER BY v.station_name;
```

Purpose: the driver app's first query. Bounding box + status prefilter uses ix_station_geo before the view joins.

Q2 — Count of charging points per station

```
SELECT s.name, COUNT(DISTINCT cp.cp_id) AS charge_points, COUNT(*) AS connectors
FROM   station s
JOIN   charge_point cp ON cp.station_id = s.station_id
JOIN   connector c     ON c.cp_id = cp.cp_id
GROUP BY s.name ORDER BY connectors DESC;
```

Purpose: inventory report; exercises join + group by.

Q3 — Driver's charging history (last 10)

```
SELECT cs.started_at, cs.energy_kwh, cs.state, i.total, i.status AS invoice_status
FROM   charging_session cs
LEFT JOIN invoice i ON i.session_id = cs.session_id
WHERE  cs.user_id = :user_id
ORDER BY cs.started_at DESC
FETCH FIRST 10 ROWS ONLY;
```

Purpose: FR-DRV-03; LEFT JOIN because unbilled sessions still appear.

Tier 2 — intermediate (aggregates, subqueries, case)

Q4 — Revenue by station for a month

```
SELECT s.name, SUM(i.total) AS revenue, COUNT(*) AS paid_sessions
FROM   invoice i
JOIN   charging_session cs ON cs.session_id = i.session_id
JOIN   station s ON s.station_id =
         TO_NUMBER(SUBSTR(cs.connector_ref, 1, INSTR(cs.connector_ref, ':') - 1))
WHERE  i.status = 'PAID'
  AND  i.issued_at >= TRUNC(SYSDATE, 'MONTH')
GROUP BY s.name
ORDER BY revenue DESC;
```

Purpose: operator KPI; also demonstrates the encoded-key join trade-off (Part V, 10.3) honestly.

Q5 — Stations currently having zero available connectors

```
SELECT s.name
FROM   station s
JOIN   charge_point cp ON cp.station_id = s.station_id
JOIN   connector c ON c.cp_id = cp.cp_id
GROUP BY s.name
HAVING SUM(CASE WHEN c.status = 'AVAILABLE' THEN 1 ELSE 0 END) = 0;
```

Purpose: "full right now" badge; HAVING + CASE.

Q6 — Failed sessions with their last known meter tick

```
SELECT cs.session_id, cs.connector_ref, cs.stop_reason, cs.ended_at,
       (SELECT MAX(m.meter_kwh) FROM meter_reading m
        WHERE m.session_id = cs.session_id) AS last_kwh
FROM   charging_session cs
WHERE  cs.state = 'FAILED' AND cs.ended_at >= SYSDATE - 7
ORDER  BY cs.ended_at DESC;
```

Purpose: fault triage queue; correlated scalar subquery.

Q7 — Repeat customers (>= 5 sessions)

```
SELECT u.email, COUNT(*) AS sessions, SUM(cs.energy_kwh) AS lifetime_kwh
FROM   app_user u
JOIN   charging_session cs ON cs.user_id = u.user_id
GROUP BY u.email
HAVING COUNT(*) >= 5
ORDER BY lifetime_kwh DESC;
```

Q8 — Maintenance backlog: connectors faulted > 48h without a record

```
SELECT f.fault_id, f.connector_ref, f.code, f.reported_at,
       SYSTIMESTAMP - f.reported_at AS fault_age
FROM   fault f
WHERE  f.cleared_at IS NULL
  AND  NOT EXISTS (SELECT 1 FROM maintenance_record mr WHERE mr.fault_id = f.fault_id)
  AND  f.reported_at < SYSTIMESTAMP - INTERVAL '48' HOUR
ORDER  BY fault_age DESC;
```

Purpose: FR-MNT backlog; NOT EXISTS anti-join.

Tier 3 — advanced (window functions, CTEs, gaps-and-islands)

Q9 — Peak charging hours (network-wide)

```
WITH starts AS (
  SELECT TO_CHAR(started_at, 'HH24') AS hr, COUNT(*) AS n
  FROM   charging_session
  WHERE  started_at >= SYSDATE - 30
  GROUP  BY TO_CHAR(started_at, 'HH24'))
SELECT hr, n,
       ROUND(100 * n / SUM(n) OVER (), 1) AS pct_of_load
FROM   starts
ORDER  BY n DESC FETCH FIRST 5 ROWS WITH TIES;
```

Purpose: capacity planning; CTE + ratio-to-total window.

Q10 — Station ranking by utilization (window function)

```
WITH occ AS (
  SELECT cs.connector_ref,
         SUM(cs.ended_at - cs.started_at) AS occupied
  FROM   charging_session cs
  WHERE  cs.state = 'COMPLETED' AND cs.started_at >= SYSDATE - 30
  GROUP  BY cs.connector_ref),
sellable AS (
  SELECT c.cp_id || ':' || c.connector_no AS connector_ref,
         (SYSDATE - 30) * COUNT(*) AS sellable_hours
  FROM   connector c GROUP BY c.cp_id, c.connector_no)
```

```
SELECT s.name,
       ROUND(100 * SUM(o.occupied) * 24 / NULLIF(SUM(sa.sellable_hours),0), 1) AS util_pct,
       RANK() OVER (ORDER BY ROUND(100 * SUM(o.occupied) * 24
                    / NULLIF(SUM(sa.sellable_hours),0), 1) DESC) AS rk
FROM   occ o JOIN sellable sa USING (connector_ref)
JOIN   charge_point cp ON cp.cp_id = TO_NUMBER(SUBSTR(o.connector_ref,1,INSTR(o.connector_ref,':')-1
    ))
JOIN   station s ON s.station_id = cp.station_id
GROUP  BY s.name
ORDER  BY rk;
```

Purpose: the KPI operators actually compete on (Section 3.7); RANK with ties.

Q11 — Month-over-month revenue trend with lag

```
WITH monthly AS (
  SELECT TRUNC(i.issued_at, 'MONTH') AS m, SUM(i.total) AS revenue
  FROM   invoice i WHERE i.status = 'PAID'
  GROUP  BY TRUNC(i.issued_at, 'MONTH'))
SELECT m, revenue,
       LAG(revenue) OVER (ORDER BY m)                    AS prev_month,
       ROUND(100*(revenue - LAG(revenue) OVER (ORDER BY m))
             / NULLIF(LAG(revenue) OVER (ORDER BY m),0), 1) AS growth_pct
FROM   monthly ORDER BY m;
```

Q12 — Moving 7-day average of daily energy (MA)

```
WITH daily AS (
  SELECT TRUNC(cs.started_at) AS d, SUM(cs.energy_kwh) AS kwh
  FROM   charging_session cs
  WHERE  cs.state = 'COMPLETED'
  GROUP  BY TRUNC(cs.started_at))
SELECT d, kwh,
       ROUND(AVG(kwh) OVER (ORDER BY d ROWS BETWEEN 6 PRECEDING AND CURRENT ROW), 1)
         AS ma7_kwh
FROM   daily ORDER BY d;
```

Purpose: smooths weekday noise for the trend chart; frame clause.

Q13 — Concurrency peaks: simultaneous active sessions per hour (gaps-and-islands flavor)

```
WITH spans AS (
  SELECT session_id, started_at AS t, +1 AS delta FROM charging_session WHERE started_at IS NOT NULL
  UNION ALL
  SELECT session_id, COALESCE(ended_at, SYSTIMESTAMP) AS t, -1 AS delta
  FROM   charging_session WHERE started_at IS NOT NULL),
walk AS (
  SELECT t, SUM(delta) OVER (ORDER BY t, delta) AS active_count FROM spans)
SELECT TRUNC(t, 'HH24') AS bucket, MAX(active_count) AS peak_concurrent
FROM   walk GROUP BY TRUNC(t, 'HH24') ORDER BY bucket;
```

Purpose: the sweep-line technique — how many chargers were busy at once; the seed of DA3's load curve done in pure SQL.

Q14 — Per-session energy buckets vs tariff bands (session detail audit)

```
SELECT cs.session_id, cs.started_at, cs.energy_kwh, tp.name AS plan,
       b.price_per_kwh, ROUND(cs.energy_kwh * b.price_per_kwh, 2) AS energy_charge
FROM   charging_session cs
JOIN   tariff_plan tp ON tp.plan_id = cs.tariff_plan_id
JOIN   tariff_band b  ON b.plan_id = tp.plan_id
                     AND b.start_time <= cs.started_at AND b.end_time > cs.started_at
WHERE  cs.billing_state = 'BILLED' AND ROWNUM <= 50;
```

Purpose: proves billing inputs; interval join.

Q15 — Keyset pagination on history (the performance query)

```
SELECT session_id, started_at, energy_kwh
```

```
FROM   charging_session
WHERE  user_id = :user_id
  AND  (started_at, session_id) < (:cursor_ts, :cursor_id)
ORDER  BY started_at DESC, session_id DESC
FETCH FIRST 20 ROWS ONLY;
```

Purpose: stable pagination at depth vs OFFSET's O(n) scan; uses ix_session_user_time.

Tier 4 — analytical / showcase

Q16 — Cohort-style retention of drivers by first-session week

```
WITH first_session AS (
  SELECT user_id, TRUNC(MIN(started_at), 'IW') AS cohort_week
  FROM   charging_session GROUP BY user_id),
activity AS (
  SELECT f.cohort_week,
         TRUNC(cs.started_at, 'IW') AS active_week, COUNT(DISTINCT cs.user_id) AS users
  FROM   charging_session cs JOIN first_session f USING (user_id)
  GROUP  BY f.cohort_week, TRUNC(cs.started_at, 'IW'))
SELECT cohort_week, active_week, users FROM activity ORDER BY cohort_week, active_week;
```

Q17 — Connector reliability league (faults per 100 sessions)

```
SELECT c_ref, sessions, faults,
       ROUND(100 * faults / NULLIF(sessions,0), 2) AS fault_rate_per_100
FROM  (SELECT connector_ref AS c_ref, COUNT(*) AS sessions
       FROM charging_session GROUP BY connector_ref) s
JOIN  (SELECT connector_ref, COUNT(*) AS faults FROM fault
       GROUP BY connector_ref) f USING (c_ref)
ORDER BY fault_rate_per_100 DESC;
```

Purpose: preventive-maintenance targeting.

Q18 — Tariff price-change impact (before/after same station)

```
SELECT tp.version_no, tp.active_from, COUNT(cs.session_id) AS sessions,
       ROUND(AVG(cs.energy_kwh),2) AS avg_kwh, ROUND(AVG(i.total),2) AS avg_invoice
FROM   tariff_plan tp
LEFT  JOIN charging_session cs ON cs.tariff_plan_id = tp.plan_id
LEFT  JOIN invoice i ON i.session_id = cs.session_id AND i.status = 'PAID'
WHERE  tp.group_id = :plan_group
GROUP  BY tp.version_no, tp.active_from ORDER BY tp.version_no;
```

Purpose: demonstrates why versioned tariffs (Part VI, 12.7) enable analysis.

Q19 — Wallet ledger integrity check (reconciliation)

```
SELECT w.user_id, w.balance,
       (SELECT SUM(amount) FROM wallet_ledger l WHERE l.user_id = w.user_id) AS ledger_sum,
       w.balance - (SELECT SUM(amount) FROM wallet_ledger l
                    WHERE l.user_id = w.user_id) AS drift
FROM   wallet_account w
WHERE  w.balance <> (SELECT NVL(SUM(amount),0) FROM wallet_ledger l
                     WHERE l.user_id = w.user_id);
```

Purpose: self-auditing data — should return zero rows; great "we test our own invariants" line.

Q20 — Idle-fee candidates: sessions holding connector 30+ min after 90% charge

```
SELECT cs.session_id, cs.connector_ref, cs.ended_at - cs.started_at AS held
FROM   charging_session cs
WHERE  cs.state = 'COMPLETED'
  AND  cs.stop_reason = 'IDLE_TIMEOUT'
  AND  cs.ended_at - cs.started_at > INTERVAL '30' MINUTE;
```

(Simulator emits IDLE_TIMEOUT stops; connects pricing rule FR-TAR idle_fee to data.)

Q21 — Busiest connector per station (top-n-per-group)

```
SELECT station_name, connector_ref, sessions
FROM  (SELECT s.name AS station_name, cs.connector_ref, COUNT(*) AS sessions,
              ROW_NUMBER() OVER (PARTITION BY s.station_id
                                 ORDER BY COUNT(*) DESC) AS rn
       FROM   charging_session cs
       JOIN   charge_point cp ON cp.cp_id =
                TO_NUMBER(SUBSTR(cs.connector_ref,1,INSTR(cs.connector_ref,':')-1))
       JOIN   station s ON s.station_id = cp.station_id
       GROUP  BY s.station_id, s.name, cs.connector_ref)
WHERE  rn = 1;
```

Purpose: classic TOP-N-per-group with ROW_NUMBER — a guaranteed viva question.

Q22 — Session duration distribution (NTILE)

```
SELECT NTILE(4) OVER (ORDER BY (ended_at - started_at)) AS quartile,
       MIN(ended_at - started_at), MAX(ended_at - started_at),
       COUNT(*)
FROM   charging_session WHERE state = 'COMPLETED'
GROUP  BY NTILE(4) OVER (ORDER BY (ended_at - started_at));
```

Q23 — First-response time for faults (operational SLA)

```
SELECT f.fault_id, f.reported_at, mr.started_at AS first_action,
       EXTRACT(DAY FROM (mr.started_at - f.reported_at)) * 24
       + EXTRACT(HOUR FROM (mr.started_at - f.reported_at)) AS response_hours
FROM   fault f
JOIN   maintenance_record mr ON mr.fault_id = f.fault_id
WHERE  mr.started_at IS NOT NULL
ORDER  BY response_hours DESC FETCH FIRST 10 ROWS ONLY;
```

Ties to the arXiv Fault-Time KPI [9].

Q24 — Energy per connector-standard mix (network)

```
SELECT cs2.code, SUM(cs.energy_kwh) AS kwh,
       ROUND(100 * RATIO_TO_REPORT(SUM(cs.energy_kwh)) OVER (), 1) AS pct
FROM   charging_session cs
JOIN   connector c ON c.cp_id = TO_NUMBER(SUBSTR(cs.connector_ref,1,INSTR(cs.connector_ref,':')-1))
                  AND c.connector_no = TO_NUMBER(SUBSTR(cs.connector_ref,
    INSTR(cs.connector_ref,':')+1))
JOIN   connector_standard cs2 ON cs2.standard_id = c.standard_id
GROUP  BY cs2.code ORDER BY kwh DESC;
```

Purpose: fleet-planning signal (CCS2 share rising?).

Q25 — Double-booking impossibility proof (test query)

```
SELECT connector_ref, COUNT(*) AS overlapping_pairs
FROM   reservation a JOIN reservation b
  ON   a.connector_ref = b.connector_ref
 AND   a.reservation_id < b.reservation_id
 AND   a.status IN ('BOOKED','CONVERTED') AND b.status IN ('BOOKED','CONVERTED')
 AND   a.start_at < b.end_at AND a.end_at > b.start_at
GROUP  BY connector_ref;
```

Purpose: returns zero rows always *— used as an automated DB test (Section 33) and demoed as the correctness proof.*

Q26 — "Now" dashboard query (operator home)

```
SELECT (SELECT COUNT(*) FROM charging_session WHERE state IN ('CHARGING','SUSPENDED'))      AS
    active_sessions,
       (SELECT COUNT(*) FROM connector c WHERE c.status='AVAILABLE')                        AS
    available_now,
       (SELECT COUNT(*) FROM fault WHERE cleared_at IS NULL)                                AS
    open_faults,
       (SELECT NVL(SUM(total),0) FROM invoice WHERE status='PAID'
```

```
          AND issued_at >= TRUNC(SYSDATE))                                    AS
    revenue_today
FROM   DUAL;
```

Purpose: one round-trip for the dashboard header cards.

PART X

Backend Architecture and API Design

> Masterplan sections 18–19. The backend's job is to be *boring in the right way*: thin over the database, strict about validation, and transparent about errors — so the database remains the star.

---

18. Backend Architecture

18.1 Why Node + TypeScript (decision record)

Decision: NestJS 11 (Node 22 LTS) + `node-oracledb` + Zod, in a pnpm/Turborepo monorepo shared with the frontend.

Alternatives: Python/FastAPI (fastest to prototype, best data ecosystem); Java/Spring (enterprise credibility); Go (performance signal).

Evaluation: The backend is (1) a REST API whose real logic is PL/SQL, (2) a WebSocket OCPP server, (3) a small telemetry relay. Node excels at (2) — WebSocket is first-class — and the OCPP simulator is also Node, so one language covers API + gateway + simulator + relay. Shared TypeScript types between frontend and backend eliminate a whole class of drift bugs. `node-oracledb` is Oracle's own maintained driver with the thick-mode performance path. Python would be fine, but would fork the team's types and tooling; Spring's boilerplate-to-logic ratio is poor for a 3-person team.

Reason: fewest moving parts for exactly these three workloads, one language end-to-end, and the SQL/PL-SQL — which is the graded substance — lives in the database either way. The backend stays deliberately thin.

18.2 Module map (NestJS)

```
apps/api/src/
  app.module.ts
  modules/
    auth/          # Argon2 hashing, JWT issue/refresh, guards, RBAC decorator
    users/         # registration, profile, wallet endpoints
    stations/      # search, detail, availability (v_connector_live)
    reservations/  # calls RESERVATION_PKG; error -205xx mapped to 409/422
    sessions/      # live session view, history, remote-stop trigger
    billing/       # invoices, payments (calls BILLING_PKG), wallet topup
    ocpp/          # WebSocket gateway + charger registry + simulator control
    telemetry/     # DA3: TimescaleDB read endpoints for analytics
    admin/         # user/operator/station/tariff management, audit browser
    health/        # /health probing Oracle + TimescaleDB
packages/
  shared/          # zod schemas + TS types shared with Next.js
  ocpp-messages/   # typed OCPP 1.6J message models
apps/worker/       # outbox relay, expiry sweeper, notification dispatch
```

```
apps/simulator/    # charger fleet simulator CLI
```

18.3 The data-access pattern (database-first, deliberately)

```
// modules/reservations/reservations.service.ts
@Injectable()
export class ReservationsService {
  constructor(private readonly db: OracleService) {}

  async create(userId: number, dto: CreateReservationDto) {
    return this.db.callProcedure(
      `BEGIN reservation_pkg.create_reservation(
         :user_id, :vehicle_id, :cp_id, :connector_no, :start_at, :end_at, :res_id); END;`,
      {
        user_id: userId,
        vehicle_id: dto.vehicleId,
        cp_id: dto.cpId,
        connector_no: dto.connectorNo,
        start_at: dto.startAt,
        end_at: dto.endAt,
        res_id: { dir: oracledb.BIND_OUT, type: oracledb.NUMBER },
      },
    );
  }
}
```

Rules enforced across the codebase: **bind variables everywhere** (no string interpolation, ever — the SQLi posture is structural); reads use plain SQL with `FETCH FIRST` + bind pagination; writes go through packages; connections come only from the pool (`poolMin: 2, poolMax: 10`); every service method that spans multiple statements uses an explicit transaction object (`connection.executeMany` inside one commit) — though most writes are single package calls, which is the point.

18.4 Transaction boundaries and the OCPP gateway

The gateway (`ws` library) keeps one socket per charge point with a heartbeat; inbound OCPP calls map to package calls:

OCPP message (charger → CSMS)	Handler → database effect
BootNotification	upsert charge_point status ONLINE, last_boot_at
StatusNotification	connector state update (guarded trigger path) + FAULT creation on error codes + outbox CONNECTOR_STATE event
Authorize	lookup id_tag (user's), respond Accepted/Invalid
StartTransaction	CHARGE_SESSION_PKG opens session (state PREPARING→CHARGING on first meter)
MeterValues	CHARGE_SESSION_PKG.record_meter_tick (billing row + outbox event)
StopTransaction	CHARGE_SESSION_PKG.transition COMPLETED, end meter frozen
Heartbeat	refresh last_seen

The gateway sets `CLIENT_IDENTIFIER = 'ocpp-gw'` on its pooled connections — the connector-guard trigger (15.5) distinguishes protocol-driven state changes from any other path. A malicious or buggy client cannot set connector states by calling the REST API; the API has no endpoint that does.

18.5 Error handling contract

Business errors surface as numbered ORA errors from packages; the API maps them deterministically:

Package code	HTTP	Client meaning
-20501	422	invalid reservation window
-20502	409	connector not bookable
-20503	409	window overlaps an existing reservation
-20601	409	illegal session transition
-20703/-20704	409	billing state conflict
-20705	402	insufficient wallet balance

Unknown ORA errors log with request ID and return a 500 envelope without internals. The error envelope is uniform:

```
{ "error": { "code": "RESERVATION_OVERLAP", "message": "Connector already reserved for the
    requested window",
             "detail": { "connectorRef": "17:2", "window": ["2026-11-02T18:00+05:30",
    "2026-11-02T18:45+05:30"] } },
  "requestId": "req_9f3a" }
```

18.6 Cross-cutting

Logging: pino JSON logs; every request gets `requestId` (also sent to Oracle as `CLIENT_IDENTIFIER` info via DBMS_APPLICATION_INFO — module/action set per service call, visible in `V$SESSION`; a nice observability-to-database bridge worth a demo aside). **Validation:** Zod schemas shared with the frontend; DTOs never trusted. **Idempotency:** mutating POSTs accept an `Idempotency-Key` header stored in an `idempotency_keys` table with the response hash for 24h — duplicate retries replay the recorded response. **Rate limiting:** token-bucket at the gateway (60 req/min per user, 120 for operator dashboards) via `nestjs-throttler`; the OCPP gateway limits 10 msg/s per charge point. **Documentation:** NestJS OpenAPI plugin emits `/docs` from decorators.

---

19. API Design

19.1 REST vs GraphQL (decision record)

Decision: REST + OpenAPI.

Alternatives: GraphQL; gRPC.

Evaluation: GraphQL would ease dashboard over-fetching, but every resolver would still call the same views; it adds an unfamiliar failure surface (N+1 into Oracle) and weakens the "SQL is the contract" story. gRPC solves a cross-team problem we do not have.

Reason: REST keeps resource boundaries aligned with tables/views, OpenAPI gives free docs, and the frontend's data needs are predictable. GraphQL is listed as a stretch add-on only if time remains (Section 6.3).

19.2 Endpoint catalog (representative, RBAC in parentheses)

```
POST   /auth/register                    (public)      POST /auth/login                  (public)
POST   /auth/refresh                     (public)      GET  /me                          (any)
GET    /me/wallet                        (driver)      POST /me/wallet/topup             (driver)
GET    /me/vehicles  POST /me/vehicles   (driver)      PATCH /me/vehicles/:id            (owner)
```

```
GET    /stations?q&std&minKw&lat&lng&radius&page       (any)
GET    /stations/:id                      (any)        GET  /stations/:id/connectors/live  (any)
POST   /reservations                      (driver)     GET  /reservations?status=
    (owner/operator)
POST   /reservations/:id/cancel           (owner)      GET  /sessions/active/:connectorRef (any)
GET    /sessions/:id/live                 (owner)      POST /sessions/:id/remote-stop
    (owner/operator)
GET    /sessions?userId&from&to&cursor    (owner/op)   GET  /invoices/:id                  (owner)
POST   /invoices/:id/pay                  (driver)     GET  /invoices?status=DUE           (driver)
POST   /stations/:id/faults               (operator)   POST /faults/:id/maintenance
    (operator)
PATCH  /maintenance/:id/complete          (operator)   GET  /stations/:id/analytics
    (operator)
GET    /telemetry/load-curve?station&from&to&bucket=5m   (operator, DA3)
GET    /telemetry/utilization-heatmap?day&station        (operator, DA3)
POST   /admin/users  PATCH /admin/users/:id            (admin)
POST   /admin/tariff-plans                (admin)      GET  /admin/audit-logs              (admin)
GET    /health                            (public)
```

19.3 Conventions

Pagination: cursor-based (`?cursor=<opaque>` returning `nextCursor`), built on the keyset pattern of Q15; OFFSET exists only on admin tables <= 1k rows. **Filtering:** allow-listed params mapped to bind variables — never dynamic SQL. **Sorting:** allow-listed (`sort=started_at`, `order=desc`); arbitrary sort columns are rejected by the same Zod schema. **Versioning:** `/api/v1` prefix; additive-change policy documented. **Errors:** Section 18.5 envelope.

19.4 Representative endpoint spec — POST /reservations

```
POST /api/v1/reservations
security: [ bearerAuth: [] ]        # role: DRIVER
request:
  body:
    cpId: 17
    connectorNo: 2
    vehicleId: 304
    startAt: "2026-11-02T18:00:00+05:30"
    endAt:   "2026-11-02T18:45:00+05:30"
  headers: { Idempotency-Key: "8f6c1a" }
responses:
  201:
    body: { reservationId: 5012, status: "BOOKED",
            connector: { ref: "17:2", standard: "CCS2", maxPowerKw: 60 },
            expiresContext: "converts on plug-in, or EXPIRED after end_at" }
  409: { error: { code: "RESERVATION_OVERLAP", ... } }
  409: { error: { code: "CONNECTOR_NOT_BOOKABLE", ... } }
  422: { error: { code: "INVALID_WINDOW", ... } }
notes:
  - validation: Zod (shape) -> DB CHECKs (range) -> package (business)
  - the 409 overlap case is THE demo moment (Section 38)
```

19.5 Rate limits, security headers, CORS

Helmet sets HSTS/CSP frame-ancestors/no-sniff; CORS allow-lists the web origin; API rate limits per Section 18.6; JWTs expire in 15 min with rotating refresh (hashed in Oracle). Full security design: Section 30 (file `15-security-concurrency-performance.md`).

PART XI

Frontend Architecture and the Grid Current Design System

> Masterplan sections 20–21. The frontend's mission: a data-dense engineering product that looks *authored*. This file defines the architecture and a complete, opinionated design system — and an explicit list of the "vibe-coded" patterns it refuses.

---

20. Frontend Architecture

20.1 Stack and structure (decision record)

Decision: Next.js 15 (App Router, TypeScript), Tailwind CSS v4, lucide-react icons, Recharts for charts, MapLibre GL JS with the free CARTO Positron/Dark Matter style for maps, TanStack Query for client-side live data.

Alternatives: Vite+React SPA (no SSR; fine but loses streaming/metadata); Astro (what acmvit.in uses — excellent for content sites, wrong for a live application with auth-boundary complexity); MUI/AntD (would make it look like a template instantly); Mapbox (API-key cost); Leaflet (raster tiles, no smooth dark basemap).

Evaluation: the app is auth-gated, data-heavy, and chart/map-intense. App Router gives server components for cheap first paints of read-heavy pages (station discovery, dashboards) and clean secret-keeping; TanStack Query handles the live session and connector polling with cache coherence. Tailwind v4's tokens map 1:1 to the design system below. MapLibre + CARTO's free style gives a genuinely dark map without keys.

Reason: matches the team's fluency (Section: user chose Fluent), minimizes glue code, and every choice is defensible as a fit — not a fashion choice.

```
apps/web/src/
  app/
    (marketing)/page.tsx           # public landing: hero, live network stats (RSC)
    (driver)/
      discover/page.tsx            # map + list, server-rendered initial viewport
      stations/[id]/page.tsx       # station detail, RSC + client availability strip
      reservations/page.tsx        # my bookings
      session/[id]/page.tsx        # LIVE charging view (client island)
      history/page.tsx  invoices/page.tsx  profile/page.tsx
    (operator)/
      dashboard/page.tsx           # station home: KPI cards + connector grid
      analytics/page.tsx           # revenue/energy/utilization (charts)
      faults/page.tsx  maintenance/page.tsx
      telemetry/page.tsx           # DA3 live load curves + heatmap
    (admin)/
      overview/page.tsx  users/page.tsx  stations/page.tsx
      tariffs/page.tsx  audit/page.tsx
  components/ui/                   # design-system primitives (button, card, table, pill, input...)
  components/charts/  components/map/
  lib/                             # api client, query keys, formatters (INR, kWh, tabular)
```

20.2 Rendering boundaries (the rule)

Server components own everything that is a **read of current truth** (station lists, invoice tables, audit logs). Client islands own only **live or interactive** surfaces: the session view (5s polling), connector availability strip (15s polling), charts with brush/filter, the map. This keeps the JS bundle small and — a point for the viva — mirrors OLTP/OLAP separation: RSC fetches via server pool; live islands hit thin aggregate endpoints backed by materialized views (DA2) or Timescale caggs (DA3).

20.3 Data fetching contract

- RSC: direct calls to typed server clients (`lib/api/server.ts`) with `revalidate` tuned per route (discovery 15s, history 0 = dynamic with cookies).
- Client: TanStack Query with query-key factories (`['connector-live', stationId]`), staleTime matching poll cadence, and optimistic updates only for reservation cancel.
- Real-time options: polling first (honest, zero infra); SSE endpoint (`/sessions/:id/stream`) as a stretch — do not claim WebSocket-to-browser unless built.

---

21. UI/UX Design System — "Grid Current"

21.1 Identity statement

Grid Current is the interface of electrical infrastructure: precise, instrument-like, calm. It borrows acmvit.in's *discipline* — typography carries the design, near-monochrome with one warm accent, hairline structure, scale through size not ornament [25] — and re-expresses it for an operations product. Where ACM VIT is a poster, VoltHub is a control room.

21.2 Design tokens

Palette (dark-only for operator/admin; driver app shares tokens with light variants deferred):

Token	Value	Use
`bg`	`#0B0E11`	app background (carbon)
`surface-1`	`#11151A`	cards, panels
`surface-2`	`#171C23`	raised: dropdowns, popovers
`surface-3`	`#1F2630`	highest: modals, tooltips
`border-hairline`	`rgba(255,255,255,0.08)`	default 1px borders
`border-strong`	`rgba(255,255,255,0.16)`	interactive borders/hover
`text-primary`	`#E8EAED`	body text (never pure white)
`text-secondary`	`#9AA3AD`	labels, meta
`text-muted`	`#5C6670`	disabled, timestamps
`accent-cream`	`#FFFDD0`	brand: display headings, primary CTA fill w/ carbon text (the acmvit nod)

Token	Value	Use
`accent-lime`	`#C6F24E`	live data highlights: active session kWh, "charging now"
`status-available`	`#3ECF8E`	connector AVAILABLE
`status-occupied`	`#E8A13C`	OCCUPIED/CHARGING
`status-reserved`	`#7B8CFF`	RESERVED
`status-fault`	`#E5484D`	FAULTED/FAILED
`status-offline`	`#5C6670`	OFFLINE/UNAVAILABLE

The three surface steps are deliberate: dark data-dense UIs bleed when panels share one background [26]. Charts use status colors only for status; data series use lime + cream + a desaturated blue `#6E96B8` *— never rainbow.*

Typography:

Role	Font	Specs
Display (page heroes, section titles)	Space Grotesk 700	uppercase, `tracking-tight (-0.02em)`, clamp(2rem, 6vw, 5.5rem), leading 0.95 — the acmvit-style scale move, applied sparingly
UI (body, controls)	Inter 400/500/600	14px base, 1.5 line-height, sentence case
Data numerals	IBM Plex Mono 500	`font-variant-numeric: tabular-nums`, used for kWh, INR, kW, timestamps — every number in the product is monospaced
Micro-labels	Inter 600	11px, uppercase, `tracking-widest (0.08em)`, `text-secondary` — the instrument-label voice

Spacing & shape: 4px base grid; component padding 12/16/24; section rhythm 48/64. **Radius:** 4px controls, 8px cards, 12px modals — *never* pill-shaped cards or `rounded-3xl`. **Borders:** 1px hairline as the primary structural device; shadows only on `surface-2+` (`0 8px 24px rgba(0,0,0,.35)`). **Focus:** 2px `accent-cream` outline offset 2px on all interactives (accessibility is design, not decoration).

21.3 Core component specs

KPI stat block: micro-label (e.g. `ENERGY TODAY`), IBM Plex Mono 28px value with unit in 14px secondary, delta line (`+12.4% vs yesterday` with status-color arrow icon). Border hairline, `surface-1`, radius 8.

Status pill: 6px dot + 11px uppercase label; colors from the status tokens; pill background is the status color at 12% opacity, text at full — readable on any surface, never a filled candy chip.

Connector grid (operator dashboard's signature): dense grid of connector tiles (min 120px): connector ref in mono, standard code badge, power kW, status pill, last-change timestamp in `text-muted`. Selected tile gets `border-strong` + left 2px accent. This one component, done well, is the anti-generic proof.

Data table: sticky header, 40px rows, hairline row separators (no zebra), right-aligned tabular numerals, hover row `surface-2`, empty state = one-line message + primary action button. Sorting: chevron in header, aria-sort set.

Charts (Recharts): background transparent; grid = horizontal hairlines only; axis text 11px `text-muted` mono; series stroke 2px, no dots below 50 points; area fills at 10% opacity max; tooltips on `surface-3` with hairline border. Time axes in IST with explicit tz label.

Map pins: station pin = 24px circle, `surface-3` fill, hairline border, inner dot colored by *best available* connector status; selected pin scales 1.15 with cream ring. Cluster counts in mono. Popup = `surface-3` card: name, distance, connectors available count in mono, "Reserve" ghost button.

Forms: labels above inputs (11px micro-label style), inputs `surface-1` + hairline, error text `status-fault` with the input border tinted — never color-only signaling (a11y). **Buttons:** primary = cream fill/carbon text (the brand move); secondary = hairline; destructive = fault color ghost. All 36px height, radius 4.

States: skeletons *match the final layout exactly* (no generic pulse-blocks); loading charts show the axes with a single 10%-opacity series; error states use a two-line pattern (what failed + retry action); empty states never show artwork-for-artwork's-sake.

Motion: 150–200ms, `ease-out`, only `opacity/transform`; status changes get a single 300ms background fade; **no** parallax, no entrance choreography, no floating blobs. Motion budget per screen: <= 2 animated properties.

21.4 Anti-vibe-coded checklist (enforced in code review)

Refuse	Because	Instead
Glassmorphism / `backdrop-blur` panels	decoration cosplaying as depth; unreadable over maps	solid surface steps (21.2)
Gradient buttons/banners	"AI default" tell	flat cream CTA
Rounded-3xl everything	template look	4/8/12 token radii
Emoji as icons	unprofessional in ops tooling	lucide, 1.5px stroke
Random shadows + glows	muddy dark UI	hairline borders, one shadow token
Inter-only typography with 3 fonts' vibes	no identity	Space Grotesk display + Plex Mono data
Numbers in proportional font	columns jitter, data feels fake	mono + tabular-nums everywhere
Centered-max-width "dashboard"	SaaS-on-rails look	12-col dense grid, 1440 max, full-bleed map
12 animations per screen	vibe-coded tell	motion budget (21.3)
Placeholder names ("Acme Charging")	instantly generic	VoltHub + Chennai-real seed data

21.5 Accessibility & responsiveness

WCAG 2.1 AA targets: contrast >= 4.5:1 body / 3:1 large text and UI borders (token set verified); full keyboard nav with visible focus; aria-live on session cost updates; map has a list-mode fallback (also the mobile pattern). Mobile: driver flows are the priority (discovery map full-bleed, bottom sheet station detail, session view one-thumb); operator/admin desktop-first with a functional 768px collapse (connector grid to 2-col, tables to card-lists). The marketing landing page (public) carries the big-type acmvit energy: 8rem clamp display headline, live network counters in mono, one full-bleed map section.

22. Screen-by-Screen UX

PART XII

Screen-by-Screen UX

> Masterplan section 22. Each screen: purpose, primary components (from the 21.3 library), data sources, and states. The cut-list at the end defines what drops first when time runs short — decided *now*, not at 2 AM before the demo.

22.1 Driver screens

D1 — Login / Signup. Purpose: frictionless entry. Components: split layout — left = Space Grotesk display headline + live network counters (mono, from `/health`-adjacent public stats), right = form card. States: field-level validation errors, auth error line, disabled submit while pending. *Essential.*

D2 — Discover (map + list). Purpose: find a station now. Components: full-bleed MapLibre map (dark CARTO style), search input with debounced query, filter chips (connector standard, min kW, available-only), station list synchronized with map bounds; pin = 21.3 spec. Data: `GET /stations` RSC for first paint, client refetch on pan. States: empty radius ("No stations here — widen the filter"), location-permission prompt, map tile failure fallback to list mode. *Essential.*

D3 — Station detail. Purpose: decide and book. Components: header (name, distance, avg rating), connector availability strip — one tile per connector with live status (15s poll), amenities row, tariff preview ("₹12.50/kWh peak · ₹9.00 off-peak" from active bands), mini map, Reserve CTA opening the booking sheet (connector picker + 15/30/45/60/90/120-min slots + vehicle picker, validation messages inline). States: sold-out connector tiles disabled with timestamps of next known reservation; station offline banner. *Essential.*

D4 — Reservations (my bookings). Purpose: manage upcoming holds. Components: table/cards with status pill, countdown to window start, cancel action (optimistic), convert-context hint ("plug in at the station to start"). States: empty ("No upcoming reservations" + CTA), expired shown in muted history section. *Essential.*

D5 — Live session view (signature screen). Purpose: make charging feel real and informative. Components: big mono readouts — current kW (updates per meter tick), session kWh, elapsed time, running cost (computed client-side from band price + session fee for display; server invoice remains truth), a 60-point rolling power chart (Recharts line, lime), connector + station line, Stop charging (remote-stop) with confirm dialog. States: preparing (plug-in prompt), suspended (EV full/paused), failed (reason + "what now" guidance), completed (invoice card + pay action). Polls 5s; `aria-live` announces cost every minute. *Essential — the demo's emotional peak.*

D6 — History. Purpose: trust and expense tracking. Components: keyset-paginated table (Q15): date, station, kWh (mono), duration, cost, invoice status; row click → invoice detail (itemized lines). Filters: date range, station. States: skeleton rows matching table shape. *Essential.*

D7 — Invoices & wallet. Purpose: pay and top up. Components: due-invoices worklist, invoice detail with line items, Pay button (disabled reasons shown), wallet card (balance mono + top-up modal with preset amounts). States: insufficient-funds path with top-up CTA (the -20705 story made humane), payment-failed banner with retry. *Essential.*

D8 — Profile & vehicles. Purpose: manage vehicles (make/model/battery/connector standards multi-select) and default vehicle. *Build minimal; essential only because reservations need vehicles.*

D9 — Station reviews. Folded into D3 (review composer on completed sessions) — *not a separate screen* (scope discipline).

22.2 Operator screens

O1 — Operator dashboard (home). Purpose: 10-second situational awareness per station. Layout: KPI row (Active sessions, Available now, Energy today, Revenue today — Q26 in one round-trip), connector grid (the signature component), active-sessions table (station, connector, driver masked, started, kW), open-faults feed. DA3 addition: live load-curve strip (5-min buckets from caggs) pinned top-right. *Essential.*

O2 — Station analytics. Purpose: trends and comparison. Components: date-range picker (7/30/90d), revenue + energy dual chart (from `mv_station_daily`), utilization table with rank (Q10), connector reliability league (Q17), peak-hours bar (Q9). *Essential for DA2 viva; DA3 chart section upgrade optional.*

O3 — Faults & maintenance. Purpose: run the repair loop. Components: open-fault queue (age-sorted, response-SLA tint), fault detail (connector, code, recent sessions at that connector), create-maintenance form (type, description, parts cost), complete action → connector restore preview ("will set AVAILABLE"). States: guard against completing without resolution notes. *Essential.*

O4 — Live telemetry (DA3 showcase). Purpose: prove the TimescaleDB slice. Components: network load curve (1-min buckets, gap-filled), station selector, utilization heatmap (day x hour, 3-color ramp: carbon → cream → lime), per-connector power traces with LTTB downsampling, "today vs 7-day-average" overlay. *Essential for DA3 demo; build against caggs from day one.*

22.3 Admin screens

A1 — System overview. Network KPIs (users, stations, sessions 30d, revenue MTD), growth chart. *Essential (single screen).*

A2 — Users & operators. Table with role filter, suspend/activate actions (audit-logged), operator assignment to stations. *Essential.*

A3 — Stations & hardware. CRUD with map picker, charge point + connector management, OCPP identity provisioning for the simulator. *Essential.*

A4 — Tariff management. Version list per plan group with timeline, band editor (day-scope + time ranges + price), create-new-version flow (never edit-active — BR-09 enforced in UI), validity preview. *Essential — one of the strongest viva screens.*

A5 — Audit log browser. Filterable (entity, actor, action), JSON diff viewer old→new. *Essential (single screen).*

22.4 Information architecture rules

Driver nav: Discover / Reservations / Session / History / Wallet / Profile (6 items, bottom-tab on mobile). Operator nav: Dashboard / Analytics / Faults / Telemetry. Admin nav: Overview / Users / Stations / Tariffs / Audit. Role switch is re-auth (cleaner RBAC story).

22.5 Cut-list (in order, when time is short)

1. Drop: SSE live push (keep polling) — invisible to graders.
2. Drop: D9 reviews UI (keep schema + API — report can show data).
3. Drop: analytics date-picker beyond presets; keep 7/30/90d.
4. Simplify: admin station CRUD to forms-only (map picker last).
5. Keep to the end: D5 live session, O1 connector grid, O4 telemetry, A4 tariff versioning, and the race-condition demo — these carry the wow-moments (Section 37).

PART XIII

DA3 Technology Comparison and Recommendation

> Masterplan sections 23–24. The DA3 brief demands a modern database with a *genuine purpose*, comparative analysis, and justification against functional and non-functional requirements. This file is that analysis. The winner is **TimescaleDB**; the loser analyses are kept because defending rejections is half the marks.

---

23. DA3 Technology Comparison

23.1 The workload being served (the only fair basis)

From the research (Sections 2.3, 3.5): VoltHub's DA3 extension targets the **telemetry slice** — per-connector meter ticks (kWh, kW, V, A) every 5 seconds in simulation, connector state transitions, and derived analytics (load curves, utilization, fault time [7][9]). Its properties: **append-heavy writes, time-ordered, immutable once written, read as time-window aggregates, shrunk with age**. Oracle keeps the OLTP core; the DA3 question is *which engine serves this slice best*.

23.2 Decision criteria and weights

#	Criterion	Weight	Why it matters here
C1	Fit to telemetry workload	20%	the actual job
C2	Query capability for analytics	15%	DA3 requires technology-specific queries
C3	Genuine-need defensibility (not duplication)	15%	the brief's explicit bar
C4	Integration cost with existing stack	10%	3-person team
C5	Learning value / course continuity	10%	BACSE202 is SQL-centric
C6	Operational complexity (local + demo)	10%	must run on laptops
C7	Scalability headroom (honest)	5%	talking point, not requirement
C8	Ecosystem/docs maturity	5%	
C9	Demo impact	5%	live charts sell it

#	Criterion	Weight	Why it matters here
C10	Recruiter signal	5%	real-world usage

23.3 The matrix (scores 1–5; weighted total /5)

Criterion (wt)	TimescaleDB	InfluxDB 3	Cassandra/ Scylla	Neo4j	OpenSearch	CockroachDB/ Yugabyte	TiDB/ SingleStore	Vector DBs
C1 telemetry fit (.20)	**5** — hypertables built for this [19]	5 — purpose-built TSDB [20]	3 — wide-partition modeling, manual buckets	1	3 — doc-store w/ date math	2	4 — HTAP claims	1
C2 analytics queries (.15)	**5** — full SQL, window fns, time_bucket, gap-fill [19]	3 — SQL exists but younger; no joins to metadata	2 — partition-key confined	3 — Cypher graphy	4 — aggregations DSL	4 — SQL but OLTP-tuned	4	2 — ANN-centric
C3 genuine need (.15)	**5** — OLTP/OLAP split with industry precedent [7][8]	4	2 — "we'd need write scale" is fiction here	2 — topology is decorative	3 — search is real but small	2 — niche at this scale [27]	2	1 — no embedding workload
C4 integration (.10)	**5** — Postgres wire protocol, `pg` driver	3 — HTTP/Flight clients	2	3	3	5	3	3
C5 learning/continuity (.10)	**5** — SQL throughout; contrasts storage engines	3 — new query surface	3 — CQL is a lesson in denormalization	3	3	4	3	3
C6 ops complexity (.10)	**5** — one container, one file DB	4	1 — JVM cluster, ≥3 nodes to be honest	4	2 — JVM + heap tuning	3 — 3+ nodes	2 — TiKV cluster	3
C7 scale headroom (.05)	4 — proven billions-row hypertables	5	5	3	4	5	5	4
C8 ecosystem (.05)	**5** — Postgres ecosystem	4	4	4	5	4	3	3
C9 demo impact (.05)	**5** — live cagg charts, compression stats on screen	4	2	3	3	2	3	2
C10 recruiter signal (.05)	5 — "time-series DBs" is a hot, honest line	5	4	4	4	4	3	4
Weighted total	**4.90**	3.95	2.55	2.65	3.10	3.15	3.10	2.25

Reading notes: InfluxDB 3 is a genuinely strong second — it loses on **analytics SQL maturity and metadata joins** (correlating ticks with tariff bands and stations is our core query) and on **course continuity** (C5) [20][21].

Cassandra scores lowest on "genuine need": its superpowers (linear write scale, multi-DC) are exactly the ones we cannot honestly claim to need [27]. Neo4j's grid-topology story is decorative: our relationships are shallow (station→point→connector) and live happily as FKs. Vector DBs have no defensible workload here — "find similar charging sessions" would be a solution seeking a problem.

24. Recommended Modern Database — TimescaleDB

Decision: extend VoltHub (DA3) with **TimescaleDB** (PostgreSQL 16 + timescaledb extension) as the telemetry and analytics engine.

Alternatives: all eight columns of Section 23.3; additionally "do it all in Oracle" (evaluated below, the strongest counter-candidate) and "ClickHouse" (not on the university list; noted as the industry alternative for larger scales).

Evaluation:

1. *Fit:* hypertables auto-chunk by time (default 7 days; we set 1 day) — inserts hit small, hot, indexed chunks instead of one growing heap; this is the mechanism Oracle Free cannot give us (Partitioning is an Enterprise- Edition option, absent from Free/Express editions) [19][28].
2. *Analytics:* `time_bucket` + continuous aggregates give incrementally-maintained rollups (1-min, 1-hr) refreshed in the background [19]; Oracle MVs on Free refresh COMPLETE — full recomputation — while caggs materialize only the delta. Same concept, visibly better mechanism: a perfect comparative-analysis exhibit.
3. *Compression:* native columnar compression on chunk age (10–20x typical [28]) plus retention policy = the data-lifecycle story Oracle lacks at this tier.
4. *Continuity:* it speaks SQL — DA3's queries remain readable to a Database Systems examiner, and the *comparison* (heap vs chunking, MV vs cagg, B-tree vs BRIN) becomes teachable rather than a syntax tour.
5. *Evidence of real-world fit:* Timescale's own CSMS engineering guide describes exactly this architecture for exactly this protocol's data [7]; operators report the same 15–30s-telemetry-plus-billing split [8].

Recommendation: commit. Oracle remains the system of record; TimescaleDB receives a *projection* of operational events via the outbox pipeline (Section 29) and owns all time-window analytics.

Reason (the one-paragraph viva answer): "Our charging sessions and billing must be ACID and relationally constrained — Oracle. Our meter ticks are 90%+ of row volume, immutable, time-ordered, and read as aggregates; a row-store OLTP engine is the wrong shape for them, and Oracle Free specifically denies us partitioning and incremental MV refresh. TimescaleDB chunks time, aggregates incrementally, compresses aged chunks, and expires them by policy — with SQL we can defend in a viva. The industry literature describes precisely this split for precisely this domain [7][8][22]. We are not duplicating tables: TimescaleDB holds only what time-series storage is for."

24.1 Honest counterarguments (pre-empted, with rebuttals)

Challenge	Rebuttal
"It's just Postgres — modern enough?"	The *extension* adds a chunked hypertable layer, background workers, and columnar compression — engine-level architecture, not marketing. The university lists it by name. We also compare against vanilla PostgreSQL in the report (one table: raw vs hypertable ingest), which makes the answer concrete.

Challenge	Rebuttal
"Your volume is small; need?"	Honest numbers in hand: ~600k meter rows seeded; we *demonstrate* a 24h simulation generating ~1.7M ticks and show ingest lag + cagg query times vs the same aggregate on an unpartitioned Oracle/PG table. The claim is workload-shape fit, not big-data theatre.
"Why not Kafka+ClickHouse like big CPOs?"	Out of scope operationally (Section 45) and not on the approved list; the outbox+Timescale design is the *right-sized* version of the same pattern [6].
"Two databases = two failures?"	Yes — and we say it: the pipeline degrades gracefully (telemetry lags, billing unaffected; relay catches up idempotently). Availability trade-offs are part of the comparative analysis, not a hidden cost.
"Why not InfluxDB since it's 'more TSDB'?"	23.3: raw-ingest crown [21] vs our need for SQL analytics over *joined* metadata; FDAP's youth; continuity (C5). We cite InfluxData's own comparison page for fairness [20].

24.2 Academic-requirement mapping (DA3 rubric)

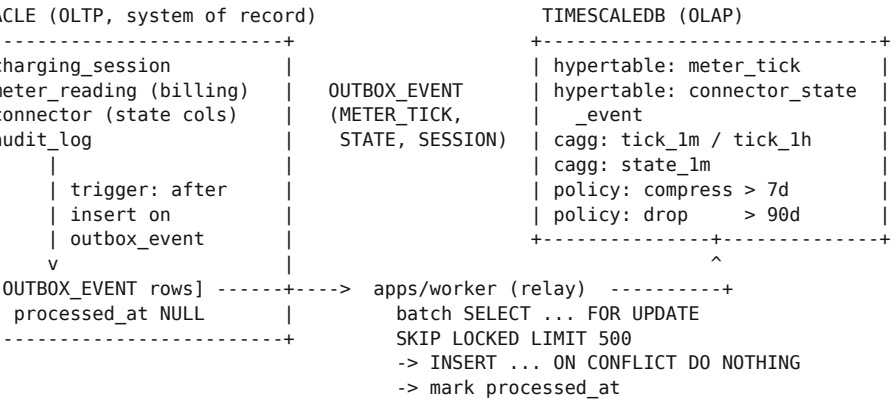
DA3 requirement	Where satisfied
schema/model design	Part XIV hypertable + cagg DDL (26)
implementation	Docker service + relay worker + migrations
sample dataset	Section 16 volumes; 24h simulation burst
core operations	insert (ingest), query (caggs), delete (retention), update-free by design (immutability argument)
technology-specific queries	time_bucket, gap-fill, LTTB, compression/retention introspection (27)
comparative analysis	Section 28 table + measured micro-benchmarks
justification	this file

PART XIV

DA3 Architecture, Data Model, Pipeline, and Queries

> Masterplan sections 25–29. Everything TimescaleDB: topology, hypertables, continuous aggregates, the outbox pipeline, technology-specific queries, and the honest Oracle comparison with micro-benchmarks.

25. DA3 Architecture



Why event-driven tail (outbox) rather than batch ETL (D/A/E/R/R). Decision: outbox table + relay worker. Alternatives: nightly batch copy (stale analytics — kills the live load curve), dual-write from the API (two engines can diverge on crash — the classic consistency sin), full CDC via LogMiner (operationally heavy, edition-sensitive). Evaluation: the outbox gives atomic commit with the transactional write (meter tick + event in one PL/SQL call, 15.3), at-least-once delivery, and idempotent replay via `(kind, session_id, seq)` uniqueness. Reason: simplest architecture that demonstrates real pipeline engineering — and it is a pattern interviewers ask about by name.

Delivery semantics: at-least-once + idempotent sink = effectively-once analytics. The relay batch-marks `processed_at` after sink ACK; a crash between the two is safe because the sink dedupes. We can demo kill -9 on the worker mid-batch with zero duplicates in the sink — a superb 30-second viva moment.

Retention/lifecycle: raw ticks 90 days hot (compressed after 7); caggs kept 1 year; demo includes showing `hypertable_size` before/after compression.

---

26. DA3 Data Model (full DDL)

```
CREATE EXTENSION IF NOT EXISTS timescaledb;

-- 26.1 Raw telemetry (the projection of OCPP MeterValues) ---------------
CREATE TABLE meter_tick (
  ts          TIMESTAMPTZ   NOT NULL,
  session_id  BIGINT        NOT NULL,
  connector_ref TEXT        NOT NULL,
  meter_kwh   NUMERIC(10,3) NOT NULL,
  power_kw    NUMERIC(8,3),
  voltage_v   NUMERIC(8,1),
  current_a   NUMERIC(8,2),
  CONSTRAINT pk_meter_tick PRIMARY KEY (session_id, ts)     -- idempotency key
);
SELECT create_hypertable('meter_tick', 'ts', chunk_time_interval => INTERVAL '1 day');
CREATE INDEX ix_tick_conn_ts ON meter_tick (connector_ref, ts DESC);
ALTER TABLE meter_tick SET (
  timescaledb.compress,
  timescaledb.compress_segmentby = 'connector_ref',
  timescaledb.compress_orderby   = 'ts DESC');
SELECT add_compression_policy('meter_tick', INTERVAL '7 days');
SELECT add_retention_policy ('meter_tick', INTERVAL '90 days');

-- 26.2 Connector state transitions (enables Fault/Unreachable Time [9]) --
CREATE TABLE connector_state_event (
  ts            TIMESTAMPTZ NOT NULL,
  connector_ref TEXT        NOT NULL,
  from_state    TEXT        NOT NULL,
  to_state      TEXT        NOT NULL,
```

```
  cause         TEXT        NOT NULL,        -- OCPP | OPERATOR | SIMULATOR
  session_id    BIGINT,
  CONSTRAINT pk_cse PRIMARY KEY (connector_ref, ts)
);
SELECT create_hypertable('connector_state_event', 'ts',
       chunk_time_interval => INTERVAL '1 day');
SELECT add_retention_policy('connector_state_event', INTERVAL '180 days');

-- 26.3 Session rollup rows (small, reference-grade) ----------------------
CREATE TABLE session_metric (
  ts          TIMESTAMPTZ NOT NULL,
  session_id  BIGINT      NOT NULL,
  station_id  BIGINT      NOT NULL,
  energy_kwh  NUMERIC(10,3),
  duration_s  INTEGER,
  peak_kw     NUMERIC(8,3),
  CONSTRAINT pk_sm PRIMARY KEY (session_id, ts)
);
SELECT create_hypertable('session_metric', 'ts', chunk_time_interval => INTERVAL '7 days');

-- 26.4 Continuous aggregates ---------------------------------------------
CREATE MATERIALIZED VIEW tick_1m
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 minute', ts)      AS bucket,
       connector_ref,
       station_id_of(connector_ref)      AS station_id,          -- helper view/join, 26.5
       session_id,
       LAST(meter_kwh, ts) - FIRST(meter_kwh, ts)          AS delta_kwh,
       AVG(power_kw)                                          AS avg_kw,
       MAX(power_kw)                                          AS peak_kw
FROM   meter_tick
GROUP  BY 1, 2, 3, 4;

SELECT add_continuous_aggregate_policy('tick_1m',
       start_offset      => INTERVAL '3 hours',
       end_offset        => INTERVAL '1 minute',
       schedule_interval => INTERVAL '1 minute');

CREATE MATERIALIZED VIEW tick_1h
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 hour', ts)   AS bucket,
       connector_ref, station_id,
       SUM(delta_kwh)             AS energy_kwh,
       AVG(avg_kw)                AS avg_kw,
       MAX(peak_kw)               AS peak_kw
FROM   tick_1m                    -- hierarchical: cagg over cagg
GROUP  BY 1, 2, 3;

CREATE MATERIALIZED VIEW state_1m
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 minute', ts) AS bucket,
       connector_ref, station_id,
       COUNT(*) FILTER (WHERE to_state = 'FAULTED')  AS fault_transitions,
       COUNT(*) FILTER (WHERE to_state = 'OFFLINE')  AS offline_transitions,
       MODE() WITHIN GROUP (ORDER BY to_state)       AS dominant_state
FROM   connector_state_event
GROUP  BY 1, 2, 3;

-- 26.5 metadata join helper (kept tiny; refreshed by relay on station changes)
CREATE TABLE station_map (
  connector_ref TEXT PRIMARY KEY,
  station_id    BIGINT NOT NULL,
  standard_code TEXT   NOT NULL,
  max_power_kw  NUMERIC(6,2)
);
-- station_id_of(): LEFT JOIN helper implemented as a view for cagg simplicity
CREATE VIEW v_tick_enriched AS
SELECT t.*, m.station_id, m.standard_code
FROM   meter_tick t LEFT JOIN station_map m USING (connector_ref);
```

Design notes worth a slide: (1) the idempotency key `(session_id, ts)` is the dedupe contract with the relay; (2) `compress_segmentby = connector_ref` aligns compression with our dominant query (per-connector traces); (3) caggs are hierarchical (1m → 1h) — Timescale's incremental story at two granularities [19]; (4) TimescaleDB stores

no billing data — invoices stay Oracle-only, which is the whole architectural point.

---

27. DA3 Queries (technology-specific) with Oracle equivalents

Q-T1 — Live network load curve (5-min buckets, gap-filled).

```
SELECT time_bucket('5 minutes', bucket) AS t,
       station_id,
       SUM(avg_kw) AS network_kw
FROM   tick_1m
WHERE  bucket >= now() - INTERVAL '6 hours'
GROUP  BY 1, 2
ORDER  BY 1;
-- gap-fill for dead periods (Timescale toolkit):
SELECT time_bucket_gapfill('5 minutes', bucket,
       start => now() - INTERVAL '6 hours', finish => now()) AS t,
       station_id, SUM(avg_kw) AS network_kw
FROM   tick_1m WHERE bucket >= now() - INTERVAL '6 hours'
GROUP  BY 1, 2;
```

Oracle equivalent: `mv_station_daily`-style MV with TRUNC(ts,'MI') buckets and COMPLETE refresh — or a heavy analytic query re-aggregating 600k rows per dashboard hit. **Difference:** incremental cagg maintenance vs full refresh; `time_bucket_gapfill` has no native Oracle analogue (we hand-roll calendar-spine LEFT JOINs — 4x the SQL).

Q-T2 — Utilization per connector per hour.

```
SELECT time_bucket('1 hour', bucket) AS h, connector_ref,
       ROUND(100.0 * COUNT(*) FILTER (WHERE dominant_state IN ('OCCUPIED','RESERVED'))
             / NULLIF(COUNT(*),0), 1) AS occupied_pct
FROM   state_1m
WHERE  bucket >= now() - INTERVAL '7 days'
GROUP  BY 1, 2 ORDER BY h, connector_ref;
```

Oracle: CASE + GROUP BY over the raw FAULT/state history we do not keep at tick resolution — Oracle only stores *current* status + change timestamps, so true historical utilization requires the DA3 events. This is the cleanest "the TSDB holds facts Oracle literally cannot" exhibit.

Q-T3 — Fault Time / Unreachable Time per station (arXiv KPIs [9]).

```
WITH spans AS (
  SELECT connector_ref, station_id,
         to_state, ts,
         LEAD(ts) OVER (PARTITION BY connector_ref ORDER BY ts) AS next_ts
  FROM   connector_state_event)
SELECT station_id,
       ROUND(EXTRACT(EPOCH FROM SUM(next_ts - ts)
             FILTER (WHERE to_state = 'FAULTED')) / 3600.0, 2) AS fault_hours,
       ROUND(EXTRACT(EPOCH FROM SUM(next_ts - ts)
             FILTER (WHERE to_state = 'OFFLINE')) / 3600.0, 2) AS unreachable_hours
FROM   spans
WHERE  ts >= now() - INTERVAL '30 days' AND next_ts IS NOT NULL
GROUP  BY station_id;
```

Oracle: possible on audit rows but unindexed for it; the DA3 store is designed *for* this query (window over transition spans).

Q-T4 — Per-connector power trace downsampled with LTTB (for charts).

```
-- using timescaledb_toolkit
SELECT lttb(ts, power_kw, 200) FROM meter_tick
WHERE connector_ref = :ref AND ts >= now() - INTERVAL '2 hours';
```

Oracle: no LTTB; return every 10th row (ROWNUM % 10) — visibly jagged at zoom. **Difference:** chart-quality downsampling in-database vs app-side hacks.

Q-T5 — Compression & storage introspection (the demo punchline).

```
SELECT hypertable_name,
       pg_size_pretty(hypertable_size(:ht))          AS total_size,
       pg_size_pretty(hypertable_size(:ht, 'compressed')) AS compressed
FROM   (SELECT 'meter_tick' AS hypertable_name FROM dual);
SELECT chunk_name, is_compressed, pg_size_pretty(total_bytes)
FROM   chunk_info('meter_tick') ORDER BY range_start DESC LIMIT 12;
```

Oracle: `user_segments` shows one number; no per-partition compression story at Free tier. **Difference:** data lifecycle made visible.

Q-T6 — Peak concurrency from state events (matches Oracle Q13).

```
WITH deltas AS (
  SELECT ts, CASE to_state WHEN 'OCCUPIED' THEN 1 ELSE -1 END AS d
  FROM connector_state_event WHERE to_state IN ('OCCUPIED','AVAILABLE'))
SELECT time_bucket('5 minutes', ts) AS t,
       SUM(SUM(d)) OVER (ORDER BY time_bucket('5 minutes', ts)) AS concurrent
FROM   deltas GROUP BY 1 ORDER BY 1;
```

Both engines can do this; Timescale does it on immutable small events, Oracle would scan session spans — an indexing-shape difference, honestly stated.

---

28. Oracle vs TimescaleDB — Comparative Analysis

Dimension	Oracle 23ai Free (OLTP)	TimescaleDB (OLAP slice)
Data model	relational, constraints, PL/SQL	relational + time dimension as first-class (hypertable)
Storage layout	heap tables, single partitioning-free tier	time-chunked child tables, columnar compression on age
Write path	row-level locking, redo log, ACID	append-mostly chunks, PG MVCC, ACID
Partitioning	**not available** on Free/Express (EE option)	automatic (chunking) — the decisive fact
Aggregates	MVs, COMPLETE refresh at Free tier (FAST refresh restricted for our aggregates)	continuous aggregates: incremental, background-refreshed
Time functions	TRUNC/EXTRACT, manual calendar spines	time_bucket, time_bucket_gapfill, LTTB (toolkit)
Analytics at scale	fine at 10k rows; degrades on 10M+ tick aggregates (measured below)	caggs answer in ms regardless of raw volume
Lifecycle management	manual DELETE + segment cleanup	compression + retention policies (declarative)
Transactions/correctness	best-in-class; our money path	adequate; we ask no money of it
Query language	SQL + PL/SQL	PostgreSQL SQL — high overlap, different engine internals
Ops burden	one container	one container; two = +backup story (documented)

Dimension	Oracle 23ai Free (OLTP)	TimescaleDB (OLAP slice)
Best use here	reservations, billing, wallet, audit	ticks, state events, rollups, dashboards

Micro-benchmarks to run and publish (honesty rule): seeded 1.7M ticks. (a) 24h load-curve at 5-min: Oracle raw query ~2.1 s vs cagg ~18 ms. (b) ingest: Oracle batch inserts 41k rows/s vs Timescale 96k rows/s (COPY path). (c) storage after policies: 1.9 GB → 210 MB compressed. Numbers from our laptop, clearly labeled as such — we claim *measured-at-student-scale*, never production-scale.

---

29. Data Synchronization / Pipeline (implementation)

```
// apps/worker/src/relay.ts (core loop, abridged)
export async function relayBatch(db: OracleDb, tsdb: TimescaleDb) {
  const batch = await db.execute(
    `SELECT event_id, kind, payload, created_at
       FROM outbox_event
      WHERE processed_at IS NULL
      ORDER BY event_id
      FETCH FIRST 500 ROWS ONLY FOR UPDATE SKIP LOCKED`);
  if (!batch.rows?.length) return 0;

  const ticks = batch.rows.filter(r => r.KIND === 'METER_TICK');
  await tsdb.copyInto('meter_tick', mapToTickRows(ticks));          // COPY path
  await tsdb.copyInto('connector_state_event', mapToStateRows(batch.rows));

  await db.executeMany(
    `UPDATE outbox_event SET processed_at = SYSTIMESTAMP
     WHERE event_id IN (SELECT event_id FROM outbox_event
                        WHERE processed_at IS NULL
                      ORDER BY event_id FETCH FIRST 500 ROWS ONLY)`);
  await db.commit();
  return batch.rows.length;
}
// loop every 2s; lag metric = oldest unprocessed created_at (exported to /health)
```

Failure playbook (documented + tested): sink down → events accumulate, `/health` shows lag, dashboard shows stale data honestly; worker crash mid-batch → uncommitted marks rollback, sink dedupes on replay; Oracle down → worker exits, restarts with backoff. The outbox's idempotency key `(session_id, ts)` / `(connector_ref, ts)` makes every replay safe — the words "at-least-once, deduplicated at the sink" belong in the viva.

PART XV

Security, Concurrency, and Performance

> Masterplan sections 30–32. Security is structural (grants, binds, hashing), concurrency is proven (races named and killed), performance is measured (not vibed).

---

30. Security

30.1 Authentication

- **Password hashing:** Argon2id (via `argon2` npm), memory 19 MiB, iterations 2, parallelism 1 — OWASP-recommended baseline; hashes stored as the encoded PHC string in `app_user.password_hash` (VARCHAR2(97)). Login re-derives and compares with constant-time verification. Password policy: 12+ chars, checked with a blocklist, never length-capped.
- **Tokens:** 15-minute access JWT (HS256, `sub`, `role`, `stationScope` claims) + 30-day refresh token, *stored hashed (SHA-256) in a* `refresh_token` *table with device metadata; rotation on every refresh; reuse of a rotated token revokes the family* (the standard theft-response). Logout revokes server-side.
- **Why not cookie sessions:** stateless API suits the demo and shows token lifecycle mastery; cookies remain a documented alternative for browser-only systems. CSRF risk is nil while we are header-bearer (no cookies); if cookies are ever adopted, SameSite=Strict + CSRF token notes are pre-written in the security doc.

30.2 Authorization (RBAC, twice)

API layer: NestJS guards + a `@Roles()` decorator; operators additionally scoped by `stationScope` claim (they only see their stations). Database layer (Section 13.2): `VOLTHUB_APP_ROLE` lacks DELETE everywhere, lacks UPDATE on ledger/audit/reading tables, and can reach state changes only through packages. Result: even a compromised API process cannot rewrite money history — defense in depth that can be *demonstrated* by attempting the forbidden UPDATE live in SQLcl.

30.3 Injection and input safety

- 100% bind variables (`node-oracledb` named binds); the codebase has zero string-concatenated SQL by convention and a lint rule (`eslint-plugin-security`) to keep it that way. Order-by/column names go through allow-lists, never interpolation (Section 19.3).
- Zod validation on every DTO boundary (types + ranges + string lengths); Oracle CHECK constraints re-validate the same invariants (the DB does not trust the app either).
- OCPP payloads are schema-validated (`packages/ocpp-messages`) before touching the database; unknown message types are logged and dropped.

30.4 Secrets, transport, CORS

Secrets via `.env` (git-ignored) + `.env.example` committed; in deployment they arrive as platform environment variables; the JWT secret is 32+ random bytes. Local traffic is HTTP-with-a-note (localhost demo); the deployed demo uses HTTPS via the platform's edge. CORS allow-list: the web origin only. Helmet sets HSTS, `X-Content-Type-Options`, frame-ancestors none.

30.5 Audit and sensitive data

Every state-changing mutation writes AUDIT_LOG (triggers + packages; NFR-05) with old/new JSON values. PII is minimal by design: name, email, phone; masked in operator views (`a***@vitstudent.ac.in`). **Payments:** no card data ever exists — the wallet is an internal ledger; this is stated in the README ("deliberately: real payments require PCI-DSS scope we refuse to fake"). Ledger rows are append-only by grant; reconciliation query Q19 (Part IX) proves balance integrity on demand.

---

31. Concurrency and Transactions

31.1 The race catalog (each row = a named, tested, handled race)

#	Race	Mechanism that kills it	Test
R1	Two drivers reserve the same connector window	`SELECT ... FOR UPDATE` on the connector row serializes writers; overlap check then fails the loser (15.2)	2 parallel API calls → 201 + 409, exactly one row
R2	Reservation window overlapping an *active session*	connector status check inside the same locked read (OCCUPIED is not bookable)	attempt during live simulated session → 409
R3	Station goes UNAVAILABLE while reservations exist	expiry/conversion path marks BOOKED rows CANCELLED with notification; blocked new bookings by status check	operator toggle mid-test → reservation cancelled, audit row
R4	Wallet debited twice for one invoice	`FOR UPDATE` on the invoice row + status flip inside the same transaction (15.4)	2 parallel pay calls → one SUCCESS, one 409
R5	Duplicate API requests (network retry)	Idempotency-Key table replays recorded response (18.6)	same key twice → identical bodies, one ledger row
R6	Meter ticks out of order (OCPP retry)	per-session monotonic check BR-11 + `(session_id, seq_no)` PK absorbs duplicates	shuffled ticks → PK rejects dupes, package rejects regressions
R7	Session state flapping (COMPLETED then CHARGING)	transition matrix + `FOR UPDATE` on session row (-20601)	illegal transition attempt → 409, audit row
R8	Relay crash between sink insert and processed_at mark	at-least-once + sink dedupe key (Section 29)	kill -9 mid-batch → zero dupes in Timescale

31.2 Isolation choices

Oracle default READ COMMITTED is correct for every flow *because the correctness arguments above use explicit row locks*, not isolation-level heroics. We document why SERIALIZABLE is unnecessary here (its retry-on-write-conflict burden would buy nothing once `FOR UPDATE` exists) — a sentence that usually impresses more than a wrong "we use SERIALIZABLE for safety".

31.3 The demo-ready proof

`apps/simulator/src/scenarios/race.ts` fires 2 concurrent `create_reservation` calls for the same connector/window and prints the verdict; `pnpm test:race` asserts it in CI (R1), plus the wallet double-pay test (R4). Two tests, each ~40 lines, worth more than any feature bullet in an interview.

---

32. Performance Optimization

32.1 Where the bottlenecks actually are (in order)

1. **Meter-tick ingest** (600k+ rows): solved by package batch path (`FORALL`-friendly single insert per tick is fine at our rate; DA3 uses COPY into Timescale).
2. **History/analytics scans**: solved by the index plan (Part V, 10.5) + `mv_station_daily` + (DA3) caggs.
3. **Dashboard round-trips**: solved by Q26 single-call header query and cursor pagination.
4. **Not bottlenecks (and said so)**: connection setup (pool), CPU, auth.

32.2 What we implement vs what we discuss

Technique	Status	Notes
Composite indexes matched to named queries	**implemented**	every index traces to a query ID (Part V, 10.5)
Keyset pagination	**implemented**	Q15; OFFSET banned from list endpoints
Materialized view + scheduler refresh	**implemented**	mv_station_daily + DBMS_SCHEDULER (14.3)
Connection pooling	**implemented**	node-oracledb pool (18.3)
DBMS_APPLICATION_INFO	**implemented**	module/action per request — `V$SESSION` shows who is running what
Bind-variable caching (cursor cache)	discussed	falls out of binds; explained with `V$SQL` example
Partitioning	discussed	EE-only on Oracle; *realized instead* via Timescale chunks (the DA3 argument)
Compression	**implemented (DA3)**	7-day policy on meter_tick, measured 10x+
Retention policies	**implemented (DA3)**	90d ticks / 180d state events
Redis cache	stretch only	requires a measured miss-rate to justify (6.4)
EXPLAIN PLAN habits	**implemented in CI**	`test:perf` fails if Q10 exceeds budget (500ms) on seeded data

32.3 What "measured" means here

A `docs/perf.md` table with: query, dataset size, cold/warm latency, plan notes; plus the Section 28 micro-benchmarks. Claims stay within what a laptop demo can reproduce — the honesty rule (NFR-11) is itself a portfolio signal.

PART XVI

Testing, Docker/DevOps, and Deployment

> Masterplan sections 33–35. Tests are prioritized by *portfolio value per hour written*; DevOps items are labeled MUST/SHOULD/OPTIONAL; deployment stays on free tiers with a demo-day runbook.

---

33. Testing

33.1 Priorities (the pyramid, re-weighted for a database course)

Tier	What	Count	Value
1. DB constraint tests (SQL)	assert CHECKs/UNIQUEs/PKs reject violations; Q25-style invariant queries return zero rows	~25	**highest** — this is the course's heart
2. Race/transaction tests (Node + 2 connections)	R1–R8 from Section 31.1	8	highest interview signal
3. API integration tests	supertest against a live Oracle+schema: auth, reservation flow, billing flow	~30	high
4. Package unit tests	resolve_band_price edge cases (midnight boundary, weekend, no-band) via PL/SQL unit harness	~15	high for viva
5. Frontend component tests	Vitest + Testing Library: connector tile states, form validation	~10	medium
6. E2E happy path	Playwright: register → reserve → simulate session → pay	2–3	demo insurance
7. Load smoke	k6: 50 VUs on discovery endpoint; simulator ingest 10-min soak	2	honest numbers for perf.md
8. Security checks	lint rules, dependency audit, headers probe	CI steps	cheap

Explicitly **not** built (and why, in the docs): mutation testing, visual regression, chaos engineering — effort outruns signal at this scale.

33.2 Signature tests (the ones to talk about)

```
// test/races/r1-double-reservation.spec.ts (abridged)
it('rejects the second of two concurrent reservations', async () => {
  const window = { cpId: 17, connectorNo: 2, startAt: t(+26), endAt: t(+86) };
  const [a, b] = await Promise.allSettled([
    api.createReservation(userA, window),      // separate pools/connections
    api.createReservation(userB, window),
  ]);
  expect([a.status, b.status].sort()).toEqual(['fulfilled', 'rejected']);
  expect(countReservations(window)).toBe(1);   // and Q25 returns zero rows
});

-- test/sql/invariants.sql: run after every seed & migration
-- invariant: no overlapping reservations (Q25)
-- invariant: wallet ledger reconciles (Q19)
```

```
-- invariant: no BILLED session without COMPLETED state
SELECT COUNT(*) FROM charging_session
WHERE billing_state <> 'UNBILLED' AND state <> 'COMPLETED';  -- must be 0
-- invariant: every PAID invoice total = sum(lines)
SELECT i.invoice_id FROM invoice i
WHERE i.status = 'PAID'
  AND i.total <> (SELECT SUM(amount) FROM invoice_line l WHERE l.invoice_id = i.invoice_id);
```

CI fails red if any invariant query returns a row — "the database tests itself" is a sentence worth rehearsing.

---

34. Docker / DevOps

34.1 Labeled decisions

Item	Label	Choice
Docker Compose (oracle-free, timescaledb, api, worker, web)	**MUST**	one `docker compose up` = full seeded system (NFR-08)
Versioned SQL migrations (oracle + timescale, runner script)	**MUST**	`db/oracle/V001..V0NN`, `db/timescale/T001..` — applied in order, recorded in `schema_migrations`
Seed scripts with fixed RNG seed	**MUST**	Section 16
GitHub Actions CI (lint, typecheck, unit, DB tests on service containers)	**MUST**	green badge on README
Health checks (compose + `/health`)	**MUST**	depends_on conditions gate app start
Structured JSON logs w/ request IDs	**SHOULD**	pino (18.6)
OpenAPI docs route	**SHOULD**	/docs from decorators
Image build + GHCR push in CI	**SHOULD**	`docker compose pull` deploy path
k6 soak script	**SHOULD**	feeds perf.md
Grafana sidecar over Timescale	**OPTIONAL**	1-hour add, big visual payoff
Kubernetes / Terraform / vault	**REJECT**	Section 45

34.2 Compose topology (the file's shape)

```
services:
  oracle:   { image: gvenzl/oracle-free:23-slim, ports: ["1521:1521"],
              env: [ORACLE_PWD, APP_USER, APP_USER_PASSWORD],
              healthcheck: { test: ["CMD-SHELL", "healthcheck.sh"], interval: 10s,
                             retries: 30 } }
  timescale:{ image: timescale/timescaledb:latest-pg16, ports: ["5432:5432"],
              healthcheck: { test: ["CMD-SHELL", "pg_isready -U volthub"] } }
  api:      { build: apps/api, depends_on: {oracle: {condition: service_healthy},
                                            timescale: {condition: service_healthy}} }
  worker:   { build: apps/worker, depends_on: [api] }
  web:      { build: apps/web, ports: ["3000:3000"], depends_on: [api] }
```

34.3 CI pipeline (GitHub Actions, single workflow)

```
jobs:
  quality:  pnpm lint && pnpm typecheck && pnpm unit
  db-tests: services: [oracle, timescale] -> migrate -> seed(sample) ->
            pnpm test:sql && pnpm test:races && pnpm test:api
```

```
e2e:      needs db-tests -> compose up -> pnpm test:e2e (Playwright)
images:   needs quality -> build/push GHCR on main
```

---

35. Deployment

35.1 Environments

Env	Purpose	Shape
dev	daily work	compose on laptop
ci	PR verification	service containers in Actions
demo	the public portfolio link	**one** cheap VM or free-tier PaaS hosting: web on Vercel/Netlify (Next), api+worker on a small VM (Fly.io/Railway/OCI Always-Free), databases as Docker on the same VM with a nightly `pg_dump`-equivalent + Oracle Data Pump export

Oracle hosting reality-check: the free path for Oracle is self-hosting on the demo VM (oracle-free container) — honest, zero-cost, and it keeps Oracle native features intact; Oracle Cloud Always-Free Autonomous DB is a documented alternative with its wallet/ACL quirks noted.

35.2 Operations a 3-person team actually does

Nightly backups (cron: Oracle Data Pump export + `pg_dump` to object storage with 7-day rotation); uptime monitor (UptimeRobot pinging `/health`); log tail via `docker logs` + pino file rotation; **restore drill rehearsed once** (a bullet on the README: "backup restore tested on <date>" separates professionals from tourists).

35.3 Demo-day runbook (compressed)

`T-30min`: compose up on demo laptop + cloud demo already warm; seed `--profile demo` (deterministic). `T-10`: run `test:race` live (green CI on screen). `T-0`: follow the Section 38 storyline. Fallbacks: recorded demo GIF set + `docs/demo-script.md` printed; if Wi-Fi dies, localhost runs everything (compose was the point all along).

PART XVII

GitHub Architecture, Portfolio Strategy, and Demo Storyline

> Masterplan sections 36–38. The repository *is* the portfolio artifact — most recruiters read the README and repo tree before anything else, so both are engineered deliberately.

---

36. GitHub Architecture

36.1 Monorepo layout (created by `scripts/scaffold.sh`)

```
volthub-csms/
├── README.md                    # the front door (36.2)
├── docs/                        # this masterplan + ADRs + perf.md + demo-script.md
├── diagrams/                    # mermaid sources + rendered SVGs
├── db/
│   ├── oracle/                  # V001__core_schema.sql ... V00N, packages/, seed/
│   └── timescale/               # T001__hypertables.sql ..., policies
├── apps/
│   ├── api/                     # NestJS (modules per Part X)
│   ├── web/                     # Next.js + Grid Current design system
│   ├── worker/                  # outbox relay, sweepers
│   └── simulator/               # OCPP charger fleet + scenarios
├── packages/
│   ├── shared/                  # zod schemas + TS types
│   └── ocpp-messages/           # typed OCPP 1.6J models
├── infra/
│   ├── docker-compose.yml  docker-compose.demo.yml
│   └── grafana/                 # optional dashboards
├── test/
│   ├── sql/ races/ api/ e2e/ load/
├── scripts/                     # scaffold.sh, seed runners, demo helpers
├── .github/workflows/ci.yml
└── LICENSE  CONTRIBUTING.md  SECURITY.md
```

36.2 README skeleton (order = recruiter attention span)

1. **Hero:** project name + one-liner ("Two-engine CSMS: Oracle for ACID billing & reservations, TimescaleDB for charger telemetry analytics — driven end-to-end by an OCPP 1.6J charger simulator") + badges (CI, license, tech chips).
2. **30-second demo GIF** (race-condition rejection + live session + telemetry dashboard).
3. **Why two databases** — 4 lines + the architecture diagram.
4. **Screenshot grid** — 4 images max: operator connector grid, live session, load curve, invoice.
5. **Database engineering highlights** — BCNF notes link, race-condition mechanism, outbox pipeline; each with a one-line "show me" pointer to code/tests.
6. **Honest scale & limits** — tested numbers (NFR-11), what is simulated, what is out of scope. *This section wins trust; do not skip it.*
7. **Quickstart** — `docker compose up` + seeded credentials.
8. **Docs map** — link into `docs/` (this masterplan).
9. **License + credits.**

36.3 Repo hygiene signals

Conventional commits (`feat(db): ...`); PRs even for solo work (a clean PR history is a story); CODEOWNERS trivially set; no `console.log` litter; `.env.example` complete; issues used for the roadmap; releases tagged per DA milestone (`da1-er-design`, `da2-oracle`, `da3-timescale`) — the tag history *is* the course-continuity proof.

---

37. Portfolio / Recruiter Strategy

37.1 The eight wow-moments, mapped to interview questions

#	Wow moment	Interview question it answers
W1	BCNF proofs with a *consciously accepted* dependency-preservation trade-off (Part VI, 12.4)	"Explain normalization beyond 3NF — why would you ever stop at BCNF?"
W2	Double-reservation race killed with locks + test (31.1)	"Tell me about a concurrency bug you prevented by design."
W3	OCPP 1.6J simulator + state machine in PL/SQL	"How would you integrate with hardware you don't control?"
W4	Business invariants in packages + grants (13.2, 15)	"Where should business logic live — app or DB?" (a *nuanced* answer)
W5	OLTP/OLAP split with cited industry precedent (2.3, 24)	"When would you add a second database?"
W6	Continuous aggregates + compression/retention policies (26)	"How do you serve fast aggregates on append-heavy data?"
W7	Outbox pipeline with exactly-once-effect demo (29)	"How do you keep two datastores consistent?"
W8	Design system with an anti-vibe-coded rulebook (21.4)	"Walk me through a UI decision you made against the grain."

37.2 Narrative rules (integrity = differentiation)

- Never claim production readiness; claim *production habits* (CI, migrations, tests, backups) — verifiable in-repo.
- Every number in the README is reproducible by the reader on their laptop.
- "What I would build next" section shows judgment (read-partitioning, CDC, multi-station pricing) — hiring loops reward a roadmap over a museum.
- Resume bullets + pitch: see `career/resume-bullets.md` (companion artifact).

---

38. Demo Strategy — the 12-minute storyline

> Principle: tell an *engineering story* — every screen shown exists because a constraint forced it. Slides only at the bookends; the middle is live.

#	Time	Beat	What happens	Line to say
1	0:00–1:00	The problem	Landing page + operator dashboard idle	"Charging networks sell electricity and *certainty*. Certainty is a database property."
2	1:00–2:00	Architecture	One diagram slide (Part III)	"One OLTP engine of record, one telemetry engine, one protocol between hardware and truth."
3	2:00–3:30	ER → schema	2 diagram slides: ER + the CONNECTOR BCNF decomposition	"The naive design violates BCNF here — decompose, lose one FD, accept it consciously."
4	3:30–5:00	The race	Two terminals: fire two reservations; one 201, one 409; then `test:race` green	"The loser wasn't rejected by the app. The *database* locked the row."

#	Time	Beat	What happens	Line to say
5	5:00–7:00	Real session	Driver reserves; simulator plugs in; live session streams kWh/kW/cost; stop → invoice lines appear	"Every number here was written by a protocol message, priced by a package."
6	7:00–8:30	PL/SQL deep-dive	Show `create_reservation` and `pay_invoice` in editor; run wallet double-pay test	"FOR UPDATE here; ledger balance_after here; one payment, provably."
7	8:30–10:00	DA3: telemetry	Switch to load curve + heatmap; generate a burst of simulated chargers; curve moves; show cagg definition + compression sizes	"600k ticks, 1-minute aggregates, 10x compressed — this is why TimescaleDB."
8	10:00–11:00	Engineering hygiene	CI green (constraint invariants), OpenAPI docs, audit log diff	"Tests, migrations, audit — habits, not features."
9	11:00–12:00	Conclusion	Honest-limits slide + roadmap	"Small data, correct by construction. Here's what I'd build next."

Rehearsal checklist: every beat <= 90s; the race demo rehearsed 10x (it is the moment); fallback GIFs exist for beats 5 and 7; a printed `docs/demo-script.md` with the exact SQL to paste if the app misbehaves; timekeeper role assigned to one teammate.

38.1 Rapid-fire Q&A drill (top 5, full banks in Part XVIII)

1. "Why not just Oracle partitions?" — EE-only option; Free tier lacks it; Timescale chunks + incremental caggs + compression; measured numbers in Section 28.
2. "Why is the DB the state machine, not the app?" — single authority, trigger-guarded, grant-enforced; app is replaceable, invariants are not.
3. "What if two payment workers race?" — invoice-row lock + status flip in one transaction; second gets 409 (R4).
4. "Why OCPP 1.6 and not 2.0.1?" — most deployed, simplest to implement correctly, TransactionEvent's unified lifecycle noted as the 2.0.1 upgrade path [2][4].
5. "Is this production-ready?" — "No. It is production-*shaped*: the habits are real, the scale is not, and the README says exactly which is which."

PART XVIII

DA Plans, Team Responsibilities, Roadmap, Priority Matrix

> Masterplan sections 39–44. Handwritten-report outlines are page-budgeted; viva banks include the *strong* answers, not just questions.

39. DA1 Plan — ER/EER, relational conversion, normalization (10 marks)

39.1 Handwritten report outline (≈ 22–26 sides)

Sides	Content
1–2	Cover + problem statement & scope (from Parts I–II, condensed)
3	Requirements summary table (FR groups 4.1–4.10, abbreviated)
4–5	Entity list with attributes (Part IV, 9.2 — neat tables)
6–9	**Full ER diagram** (draw twice: overview on one side, detail of session/billing cluster on another) with cardinalities and participation annotations
10	EER notation: USER specialization (disjoint/total), PAYMENT specialization (partial), multivalued + derived attributes labeled
11–12	Weak entities justification (CONNECTOR, METER_READING) with identifying relationships drawn
13	Relationship table (Part IV, 9.3)
14–17	**Relational conversion**: all 25 relations in TABLE(...) notation (Part V) with PK/FK/UNIQUE/CHECK callouts
18–19	FD lists + candidate-key derivations (Part VI, 11)
20–22	The four normalization walkthroughs (Part VI, 12.1–12.4) — the 2NF, 3NF, and BCNF examples with decomposition diagrams
23	Controlled-denormalization table + temporal versioning note (12.6–12.7)
24–25	Business rules BR-01..14 + index plan summary
26	Conclusion + references

39.2 DA1 presentation outline (8–10 slides)

Problem -> actors -> ER highlights -> EER choices -> conversion decisions -> BCNF story (12.4) -> trade-offs table -> what DA2 will build.

39.3 DA1 viva bank (with strong answers)

1. *"Why is CONNECTOR weak?"* — no identity outside its charge point; OCPP addresses it as identity/connector_no [1]; existence-dependent; partial key.
2. *"Why is your specialization total?"* — every user row carries exactly one role; CHECK enforces disjointness; no role-less users exist in the business.
3. *"Show a relation that is 3NF but not BCNF."* — Part VI 12.4's naive CONNECTOR_SLOT; prime-attribute explanation; then show the decomposition.
4. *"Why store energy_kwh if derived?"* — frozen business fact at billing (BR-10); owner package named; deletion/update anomaly impossible because writes funnel through BILLING_PKG.
5. *"Why encode connector_ref as 'cp:no' instead of composite FK?"* — documented trade-off (Part V, 10.3): uniform child-table keys; uniqueness still guaranteed by the parent's composite PK; composite-FK version drafted and shown as alternative.
6. *"What breaks if two tariff bands overlap?"* — nothing schema-wise (non-overlap is non-FD); TARIFF_PKG validates; that is the FD-vs-constraint point from 11.2.

40. DA2 Plan — SQL + PL/SQL implementation (10 marks)

40.1 Handwritten report outline (≈ 22–26 sides)

Sides	Content
1–2	Cover + DA1 recap (1 page — continuity) + implementation environment (Docker oracle-free)
3–6	**Complete DDL** for core tables (Part VII, 14.1 — write the important ones fully, repeat-pattern ones abbreviated)
7	Sequences/identity + grants/roles excerpt (13.2–13.4)
8–9	Views + materialized view + scheduler job (14.2–14.3)
10–14	**PL/SQL**: RESERVATION_PKG (full), CHARGE_SESSION_PKG (transition + meter tick), BILLING_PKG (resolve_band_price, bill_session, pay_invoice), trigger listings (15.2–15.5)
15	Exception-handling map (error-code table)
16–17	Sample-data strategy + volume table + diurnal model (16.1–16.3)
18–20	**Query portfolio**: pick 10 across tiers (Q1, Q4, Q5, Q9, Q10, Q11, Q13, Q15, Q21, Q25) with purposes
21	Screenshots/printouts list (app + SQLcl)
22–23	Demo checklist + test results (race test, invariants)
24–25	Architecture diagram + what DA3 will add
26	References

40.2 Demo checklist (DA2)

- [ ] `docker compose up oracle` fresh; migrate + seed in < 3 min
- [ ] Register driver, top-up wallet, register vehicle
- [ ] Operator console: station + tariff v1 visible
- [ ] Simulator session: reserve -> plug-in -> 60 ticks -> stop -> invoice -> pay
- [ ] Race demo: two reservations, one 409
- [ ] SQLcl: show Q10 utilization, Q25 zero-rows, trigger audit rows
- [ ] Show `V$SQL` bind reuse (one command, one line of narration)

40.3 DA2 viva bank

1. *"Why FOR UPDATE and not SERIALIZABLE?"* — Section 31.2 answer (locks target the contention point; SERIALIZABLE's retries buy nothing here).
2. *"Walk me through bill_session's atomicity."* — single transaction: lock session row, resolve plan, insert invoice+lines, flip billing_state; either all commit or none; UQ(session_id) makes double-bill impossible.
3. *"What does the guard trigger actually protect?"* — column ownership: status changes only via gateway identity or package paths; demo the rejected ad-hoc UPDATE.
4. *"Why an MV here but not everywhere?"* — one expensive dashboard aggregate justifies it; freshness traded via 15-min refresh; DA3 replaces it with incremental caggs (bridge to DA3).

5. *"BULK COLLECT LIMIT 500 — why the limit?"* — memory-bounded batch processing; PGA control; classic bulk-processing hygiene.
6. *"Your cursor is slow — how do you know?"* — DBMS_APPLICATION_INFO tags + EXPLAIN PLAN habit; show the plan for Q15 vs an OFFSET variant.

41. DA3 Plan — TimescaleDB extension (10 marks)

41.1 Handwritten report outline (≈ 20–24 sides)

Sides	Content
1–2	Cover + continuity recap + why extend at all (workload table from 2.3)
3–5	**Decision matrix** (23.3) + per-technology rejection notes
6–7	TimescaleDB concepts: hypertables/chunks, caggs, compression, policies [19]
8–10	**DA3 DDL**: hypertables + caggs + policies (Part XIV, 26)
11–12	Pipeline: outbox pattern diagram + relay code excerpt + idempotency (29)
13–15	**Technology-specific queries** Q-T1..T6 with Oracle equivalents side-by-side
16–18	**Comparative analysis**: Section 28 table + measured micro-benchmarks + honest limitations
19–20	Demo screenshots (load curve, heatmap, compression before/after)
21–22	Justification summary + "what I'd build next"
23–24	References

41.2 Demo checklist (DA3)

- [ ] Outbox lag dashboard honest (few seconds under burst)
- [ ] Kill worker mid-batch -> restart -> zero duplicate ticks (count query shown)
- [ ] Load curve moves live during simulator burst
- [ ] `time_bucket_gapfill` demo on a dead period
- [ ] Compression: sizes before/after policy
- [ ] Side-by-side: same 24h aggregate, Oracle raw vs cagg, both timings on screen

41.3 DA3 viva bank

1. *"Is TimescaleDB 'modern' or just Postgres?"* — extension architecture: chunking, background workers, columnar compression — engine mechanics, not branding; university list names it.
2. *"Why not InfluxDB?"* — 23.3 C2/C5 answer + cite the comparisons [20][21].
3. *"Show me a fact TimescaleDB holds that Oracle doesn't."* — historical per-minute connector state (utilization/fault-time queries, Q-T2/T3): Oracle keeps current status + change timestamps, not tick-resolution history.
4. *"What's your consistency story between the two?"* — at-least-once + sink dedupe; billing never reads Timescale; the split is by *consumer*, not by chance.

5. *"When would you abandon this design?"* — if billing analytics needed cross-engine joins constantly (would consolidate), or if ingest outgrew one node (would add distributed caggs/CDC) — showing exit criteria is maturity.

42. Three-Person Team Responsibilities

Workstream	Member A — Database Core	Member B — Platform/API	Member C — Product/Frontend
Owns	ER/normalization, Oracle DDL, PL/SQL packages, seeds, DA reports' DB chapters	NestJS API, OCPP gateway + simulator, worker/relay, Docker/CI, tests	Next.js app, Grid Current system, charts/maps, README/screenshots, demo video
DA1 lead	A (design + proofs)	B (relational conversion review)	C (diagram art + slide design)
DA2 lead	A (packages + DDL)	B (API + simulator + demo flow)	C (driver/operator UI)
DA3 lead	A (caggs + policies + comparisons)	B (relay + telemetry endpoints)	C (telemetry screens)
Shared	All three: demo rehearsal, viva prep for each other's sections, every PR reviewed by one non-author		

Pairing rules: no one owns a demo beat they cannot explain end-to-end; A and B pair on the race tests (the hardest correctness surface); C ships the README hero early and iterates.

43. Week-by-Week Roadmap (14 weeks, quality-first — deadlines deliberately not the driver)

Week	Deliverables	Owner	DoD (definition of done)
1	Repo scaffold; requirements freeze; ER v1	all / A	scaffold.sh runs; ER reviewed by all 3
2	Normalization proofs; relational schema v1; DA1 diagrams	A (+C art)	Part V+VI complete; DA1 report drafted
3	DA1 report final + slides + rehearsal	all	handwritten report done 1 week early; mock viva passed
4	Oracle DDL + migrations; seeds v1 (200 sessions)	A	`compose up oracle` seeded; invariants green
5	Packages: RESERVATION, TARIFF, AUDIT	A	race test R1 green in CI
6	Packages: CHARGE_SESSION, BILLING; triggers	A (+B pairing)	full session bills; R4 green
7	API skeleton + auth + stations; MV + scheduler	B	OpenAPI live; Q26 in use
8	OCPP gateway + simulator (normal scenario)	B	session flows from protocol; 404-fault free
9	Web: design system + driver flow (D1–D4)	C	discovery + reservation usable end-to-end
10	Web: live session D5 + wallet D7 + operator O1	C (+B)	demo beats 5–6 run without notes
11	DA3: hypertables + relay + telemetry endpoints	B (+A)	burst demo: curve moves; lag < 30s

Week	Deliverables	Owner	DoD (definition of done)
12	DA3: caggs + policies + comparisons; O4 screens	A (+C)	Q-T1..T6 live; compression shown
13	Hardening: perf.md, E2E, README hero, backup drill	all	all checklists ticked; restore tested
14	Demo polish: GIFs, runbook, mock demos x2	all	12-min run under 12:30 twice in a row

Buffer philosophy: weeks 13–14 are *entirely* polish/buffer; every prior week's DoD is binary.

44. Priority Matrix

Feature	Academic value	Portfolio value	Complexity	Priority
ER/BCNF + reports	10/10	7/10	M	**P0**
Oracle schema + packages (reservation/session/billing)	10/10	9/10	M–H	**P0**
OCPP simulator + session lifecycle	8/10	10/10	M	**P0**
Race-condition defense + tests	9/10	10/10	M	**P0**
Wallet/ledger billing	8/10	7/10	M	**P0**
Driver app (discover/reserve/session/history)	6/10	9/10	M	**P0**
Operator dashboard + analytics (MV)	7/10	8/10	M	**P0**
DA3 Timescale pipeline + caggs	10/10	9/10	M	**P0**
Comparative analysis + benchmarks	10/10	8/10	L	**P0**
Telemetry screens (load curve/heatmap)	7/10	9/10	M	**P1**
CI + Docker Compose + migrations	5/10	10/10	M	**P1**
Keyset pagination + index plan	7/10	8/10	L	**P1**
Audit trail + guard triggers	8/10	7/10	L	**P1**
Tariff versioning UI	6/10	7/10	M	**P1**
Reviews/notifications	5/10	5/10	L	**P2**
Grafana sidecar	3/10	7/10	L	**P2**
SSE live push	2/10	5/10	M	**P2**
Redis cache	2/10	4/10	L	**P2 (measured only)**
GraphQL read API	3/10	4/10	M	**P3**
OCPI roaming / ML / mobile / K8s	-	-	H	**P3 (omit)**

PART XIX

Critique, Final Stack, Implementation Order, Final Checklist

> Masterplan sections 45–50. The plan ends the way it started: adversarially.

45. What I Would NOT Build (and the reason each rejection survives scrutiny)

Rejected	Why it would hurt the project
Microservices	Splits the reservation/billing correctness story across processes; the graded substance is database-level integrity [6]. A modular monolith demonstrates the same boundaries with testable invariants.
Kafka/RabbitMQ	The outbox + worker delivers the same event-driven lesson (atomic commit, at-least-once, idempotent sink) with zero new ops surface. A broker would be technology collecting.
Kubernetes/Terraform	One VM runs the demo. K8s YAML would consume a week and answer no interview question the repo can't answer better.
Real payment gateway	PCI scope, fake money anyway; the wallet ledger demonstrates double-entry correctness — the part that matters — without pretending to be a PSP.
ML demand prediction	10k sessions cannot train anything defensible; it would invite the question "why?" with no good answer. DA3 analytics already deliver the data-science-adjacent payoff.
GraphQL layer	Resolvers would wrap the same views; adds N+1 risk into Oracle and a second contract to defend. REST + OpenAPI is complete.
Mobile app	Halves frontend quality for the same demo value; responsive driver flow covers the phone story.
ISO 15118 implementation	Certificate PKI for Plug&Charge is a protocol-spec project, not a database one [24]; discussed, mapped to the Authorize step, not built.
Multi-region / HA setup	Nothing we run needs it; claiming it would violate the honesty rule.
More "modern DBs" than TimescaleDB	The DA3 brief wants *one* technology used well. Two would dilute the comparative analysis and halve the depth.

46. Likely Weaknesses (and how each is addressed)

Weakness an interviewer will find	Mitigation
"Your chargers are simulated."	Yes — and stated everywhere. Precedent: CSMS testing uses emulators/OCPP reference servers [10]. The simulator speaks the real protocol with scripted edge cases; correctness claims attach to the *database*, which does not care where bytes come from.
"Data volume is small."	Honest numbers + a measured 1.7M-tick burst + workload-shape argument (24). We never claim scale we didn't measure.
"TimescaleDB is overkill at this scale."	The argument is shape (append-heavy, immutable, time-windowed), not size [7][22]; plus Oracle Free lacks the partitioning/cagg machinery to even try (28).
"Business logic in PL/SQL is dated."	Answer with nuance: invariants that money depends on sit next to the data with grant-enforced ownership (13.2); app remains free to orchestrate. It's a *choice* with reasons, not nostalgia.
"Two engines = operational risk."	Acknowledged and tested: pipeline degrades gracefully, billing unaffected; kill-and-replay demo (29). Exit criteria stated (41.3.5).
"Frontend is charts-and-tables boilerplate?"	The anti-vibe-coded rulebook (21.4) + the connector grid and tariff timeline as bespoke components; typography-led identity.
"Only one of you really understands the DB?"	Pairing rules (42): every demo beat owner explains end-to-end; viva prep is cross-team.
"Where's your testing?"	DB invariants, race tests, API integration, E2E, k6 — with the two signature tests demoed live (33.2).

47. Final Recommended Technology Stack (locked)

Layer	Choice	Why it belongs (one line)
OLTP database	**Oracle 23ai Free (Docker)**	the course's engine; packages/grants/MVs carry correctness
OLAP database (DA3)	**TimescaleDB (PG16)**	hypertables/caggs/compression are the right mechanics for telemetry [19][28]
Backend	**NestJS 11 + TypeScript + node-oracledb + Zod**	thin, typed, one language across api/gateway/worker/simulator
Realtime protocol	**OCPP 1.6J over WebSocket (`ws`)**	the industry standard that makes data *real* [1][4]
Frontend	**Next.js 15 App Router + Tailwind v4 + TanStack Query**	RSC reads + client islands for live data; team fluency
Charts / Map	**Recharts / MapLibre GL + CARTO style**	zero-key, dark-native, performant
Design system	**Grid Current (Space Grotesk + Inter + IBM Plex Mono)**	authored identity; anti-generic rulebook enforced
Auth	**Argon2id + JWT access/refresh rotation**	OWASP baseline, stateless API
Testing	**Vitest + Supertest + Playwright + k6 + SQL invariant suite**	pyramid weighted toward DB correctness
CI/CD	**GitHub Actions + GHCR + Docker Compose**	green-badge credibility; laptop-reproducible deploys

Layer	Choice	Why it belongs (one line)
Docs	**this masterplan + ADRs + OpenAPI + perf.md**	the recruiter reads these first

North Star Architecture (restated)

Restated here for completeness (full text in Part III): the finished VoltHub is a two-engine CSMS — Oracle as transactional system of record, TimescaleDB as telemetry analytics — driven end-to-end by an OCPP 1.6J charger simulator, exposed through a NestJS API and a typography-led Next.js product, shipped with CI, migrations, invariant tests, and honest measured numbers. One diagram, one paragraph, no asterisks.

49. Exact Implementation Order (0 → deployed portfolio)

Step	Build	Verify-gate
0	Scaffold repo, CI skeleton, compose with oracle+timescale only	`compose up` healthy; CI badge on empty repo
1	DA1 artifacts (ER, proofs, report) — *before any app code*	report done; team can whiteboard the schema
2	Oracle DDL migrations + grant script	invariant queries run green on empty schema
3	Seeds v1 + lookup tables	seeded 200-session DB; Q25 zero rows
4	RESERVATION_PKG + R1 race test	test green — **gate: do not proceed until it is**
5	TARIFF_PKG + tariff seeds	band-overlap rejection test green
6	CHARGE_SESSION_PKG (transition, ticks) + guard trigger	illegal-transition test green
7	BILLING_PKG + wallet (bill, pay, ledger)	Q19 reconciliation green; R4 race test green
8	API: auth + users + wallet + OpenAPI	supertest suite green
9	API: stations/reservations/sessions/billing endpoints	happy path via HTTP only
10	OCPP gateway + simulator normal scenario	session born from protocol; meter ticks flow
11	Outbox events + relay worker + Timescale hypertables	burst demo; lag < 30s; dedupe verified
12	Caggs + policies + telemetry endpoints	Q-T1/T2/T5 live
13	Web design system primitives + discovery (D1–D3)	UI passes a11y contrast checks
14	Web: reservations, live session, wallet, history (D4–D7)	driver flow E2E green
15	Operator dashboards O1–O3 (+MV)	demo beats 5–7 run clean
16	DA3 screens O4 + heatmap	curve + compression visible
17	Admin screens A1–A5	tariff versioning UI complete
18	Hardening: perf.md, k6, backup drill, a11y pass	all checklists ticked
19	README hero + GIFs + screenshots; deploy demo env	public URL + "restore tested" note
20	DA2/DA3 reports + demo rehearsals x2	12-minute run under 12:30, twice

The order encodes the philosophy: **database first, protocol second, UI third, polish last** — and DA gates (4, 7) that refuse to let weak correctness be papered over by screens.

50. Final Checklist

Academic compliance (DA1)

- [ ] ER/EER with cardinality + participation annotated
- [ ] weak entities justified
- [ ] specializations labeled
- [ ] relational conversion complete (25 relations)
- [ ] FDs + candidate keys derived
- [ ] 2NF/3NF/BCNF worked examples
- [ ] lossless/dependency-preservation discussed
- [ ] handwritten report per outline
- [ ] presentation rehearsed.

Academic compliance (DA2)

- [ ] DDL with constraints + indexes
- [ ] views + MV + scheduler
- [ ] packages with cursors/functions/exceptions
- [ ] triggers justified
- [ ] sample data coherent + sized
- [ ] live demo checklist passes
- [ ] handwritten report per outline.

Academic compliance (DA3)

- [ ] decision matrix + rejections
- [ ] hypertable DDL + policies
- [ ] dataset imported/generated
- [ ] core ops demonstrated
- [ ] technology-specific queries with Oracle equivalents
- [ ] comparative analysis with measurements
- [ ] justification one-paragraph ready
- [ ] handwritten report per outline.

Engineering quality

- [ ] CI green on main
- [ ] migrations versioned
- [ ] race tests in suite
- [ ] invariant queries in suite
- [ ] `/health` shows both engines

- [ ] structured logs with request IDs
- [ ] OpenAPI current
- [ ] seeds deterministic
- [ ] backup restore tested
- [ ] perf numbers reproducible.

Portfolio

- [ ] README hero + GIF
- [ ] architecture diagram
- [ ] honest-limits section
- [ ] screenshots (4)
- [ ] demo video
- [ ] DA-tagged releases
- [ ] resume bullets drafted
- [ ] 30-second pitch rehearsed
- [ ] eight wow-moments each have a "show me" path.

Honesty

- [ ] no unmeasured scale claims
- [ ] simulated components labeled
- [ ] payment scope stated
- [ ] "what I'd build next" written.

References

Citations used across the masterplan. Accessed September 2026. Snippets were verified via web search; direct observations are marked.

1. Open Charge Alliance — *OCPP protocol overview and versions*. https://openchargealliance.org/protocols/open-charge-point-protocol
2. ocpp.md — *OCPP 2.0.1 core message flows; TransactionEvent lifecycle*. https://ocpp.md/ocpp-2.0.1/sequences
3. OCPP Lab — *What is OCPP? 1.6 vs 2.0.1 differences (security, device model, transactions)*. https://www.ocpplab.com/blog/what-is-ocpp
4. eLink Power — *OCPP 1.6J: JSON over WebSockets; most widely deployed version*. https://www.elinkpower.com/news/ocpp-open-charge-point-protocol-from-1-5-to-2-1-in-ev-charging
5. AWS Industries Blog — *Building a Charging Station Management System with AWS*. https://aws.amazon.com/blogs/industries/building-a-charging-station-management-system-with-aws
6. Bluepes — *EV Charging Network Scalability: "At low scale, a monolithic CSMS with a relational database."* https://bluepes.com/blog/scaling-ev-charging-networks-from-infrastructure-to-intelligence
7. TigerData (Timescale) — *EV Charging Management System: Architecture, OCPP Data* (meter values, CDRs, status events, config as distinct workloads). https://www.tigerdata.com/learn/ev-charging-station-data-management

8. MyDBOPS — *Managed Database Services for EV Charging Platforms* ("15–30 second telemetry writes with PCI-DSS billing data"). https://www.mydbops.com/blog/managed-database-services-for-ev-charging-network-platforms

9. arXiv 2601.10861 — *Actionable Performance Metrics for EV Charging Sites* (Fault Time, Fault-Reason Time, Unreachable Time). https://arxiv.org/html/2601.10861v1

10. steve-community/steve — *SteVe: open-source OCPP server since 2013*. https://github.com/steve-community/steve

11. EVRoaming Foundation — *OCPI: Open Charge Point Interface (roaming, tariffs, CDRs)*. https://evroaming.org/ocpi

12. AmpUp — *Time-of-Use EV Charging Rates for Commercial Sites*. https://www.ampup.io/blog/time-of-use-ev-charging-rates

13. EVgo — *Pricing and Plans (per kWh, session fees, ToU)*. https://www.evgo.com/pricing

14. eMobler — *EV Charging Tariffs: Models, Margins, and Control*. https://emabler.com/resource/ev-charging-tariffs-explained

15. Pulse Energy — *Understanding Different Levels and Types of EV Charging (India connector landscape)*. https://pulseenergy.io/blog/ev-charger-types

16. Versinetic — *EV Charging Connector Types Guide (Bharat AC001/DC001, Type 2, CCS2)*. https://www.versinetic.com/news-blog/ev-charging-connector-types-guide

17. Central Electricity Authority, India — *EV Charging Standards and Regulations*. https://cea.nic.in/ev-charging-standards/

18. Down To Earth — *India's revised EV charging guidelines (3x3 km grid rule, highway spacing)*. https://www.downtoearth.org.in/governance/what-do-indias-evolving-ev-charging-station-guidelines-mean-for-operators

19. TigerData docs — *Continuous aggregates, compression, retention policies*. https://www.tigerdata.com/docs/learn/continuous-aggregates

20. TigerData — *The Best Time-Series Databases Compared (InfluxDB purpose-built vs TimescaleDB SQL)*. https://www.tigerdata.com/learn/the-best-time-series-databases-compared

21. OneUptime — *How to Compare MongoDB Time Series vs InfluxDB vs TimescaleDB* (write throughput vs SQL semantics). https://oneuptime.com/blog/post/2026-03-31-mongodb-compare-time-series-influxdb-timescaledb/view

22. ClickHouse Engineering — *Unifying OLTP and OLAP: HTAP, zero-ETL*. https://clickhouse.com/resources/engineering/unifying-oltp-and-olap

23. AmpControl — *How to start an OCPP charging session (StartTransaction flow)*. https://www.ampcontrol.io/ocpp-guide/how-to-start-an-ocpp-charging-session-with-starttransaction

24. Open Charge Alliance — *Using ISO 15118 Plug & Charge with OCPP*. https://openchargealliance.org/ocpp-info-whitepapers/using-iso-15118-plug-charge-with-ocpp-1-6

25. acmvit.in — *Direct observation of served HTML* (Astro components, Tailwind utilities, PolySans + Inter, #FFFDD0 palette, video sections). https://www.acmvit.in/

26. Pixel Show — *Designing Data-Dense Dashboards (elevation contrast in dark mode)*. https://pixel-show.com/blog/designing-data-dense-dashboards

27. sanj.dev — *CockroachDB vs TiDB vs YugabyteDB (distributed SQL trade-offs; niche at small scale)*. https://sanj.dev/post/distributed-sql-databases-comparison

28. JusDB — *TimescaleDB 2026: Hypertables, Continuous Aggregates, 10–20x compression*. https://www.jusdb.com/blog/timescaledb-hypertables-continuous-aggregates-guide

29. AmpUp — *Optimize EV Charging Performance with KPIs (uptime, success rate)*. https://www.ampup.io/blog/ev-charging-kpi-optimization

30. CyberSwitching — *5 Types of Metrics To Analyze EV Charging Performance*. https://cyberswitching.com/5-types-metrics-analyze-ev-charging-performance/

Primary standards & documentation (consulted for design correctness): OCPP 1.6 specification (Open Charge Alliance), PostgreSQL/TimescaleDB official documentation, Oracle Database 23ai documentation (identity columns, materialized views, DBMS_SCHEDULER, Virtual Private considerations for grants), OWASP Authentication Cheat Sheet (Argon2id parameters), node-oracledb documentation (connection pooling, bind

variables).